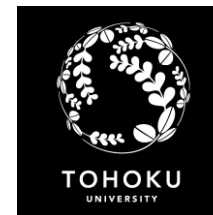


# SLAM(自己位置推定・地図構築)の 基礎と高性能化技術および実装・実応用のポイント



# 講師紹介



## 大河原 拓 [Project page URL](#)

|                   |                                          |
|-------------------|------------------------------------------|
| 2026年4月           | 産業技術総合研究所 スマートモビリティ研究グループ 研究員(常勤職員)      |
| 2025年4月 – 2026年3月 | 産業技術総合研究所 スマートモビリティ研究グループ 産総研特別研究員(ポスドク) |
| 2022年4月 – 2025年3月 | 東北大学 大学院工学研究科 航空宇宙工学専攻 博士課程              |
| 2020年4月 – 2022年3月 | 東北大学 大学院工学研究科 航空宇宙工学専攻 修士課程              |
| 2018年4月 – 2020年3月 | 首都大学東京 システムデザイン学部 航空宇宙システム工学コース          |
| 2013年4月 – 2018年3月 | 福島工業高等専門学校 機械工学科                         |
|                   | 福島県檜葉町出身                                 |

センサ融合に基づいた頑強性が高いSLAMの開発で博士論文(2025年3月)を執筆



# 本日本話しする内容

---

## 1章: SLAMの概要

2章: SLAMの基礎理論

3章: オドメトリ推定手法

4章: センサ融合によるSLAMの高性能化

5章: SLAMの実装技術とそのポイント

6章: SLAMの実例・最新研究動向の紹介



# 1. SLAMの概要: 自己位置推定とは

---



Q. 上の写真を撮った人はどこにいる？





# 1. SLAMの概要: 自己位置推定とは



Q. 上の写真を撮った人はどこにいる？



A. パリのルーブル美術館





# 1. SLAMの概要: 自己位置推定とは



自己位置推定には, 自分の位置を指し示すための地図が必要である



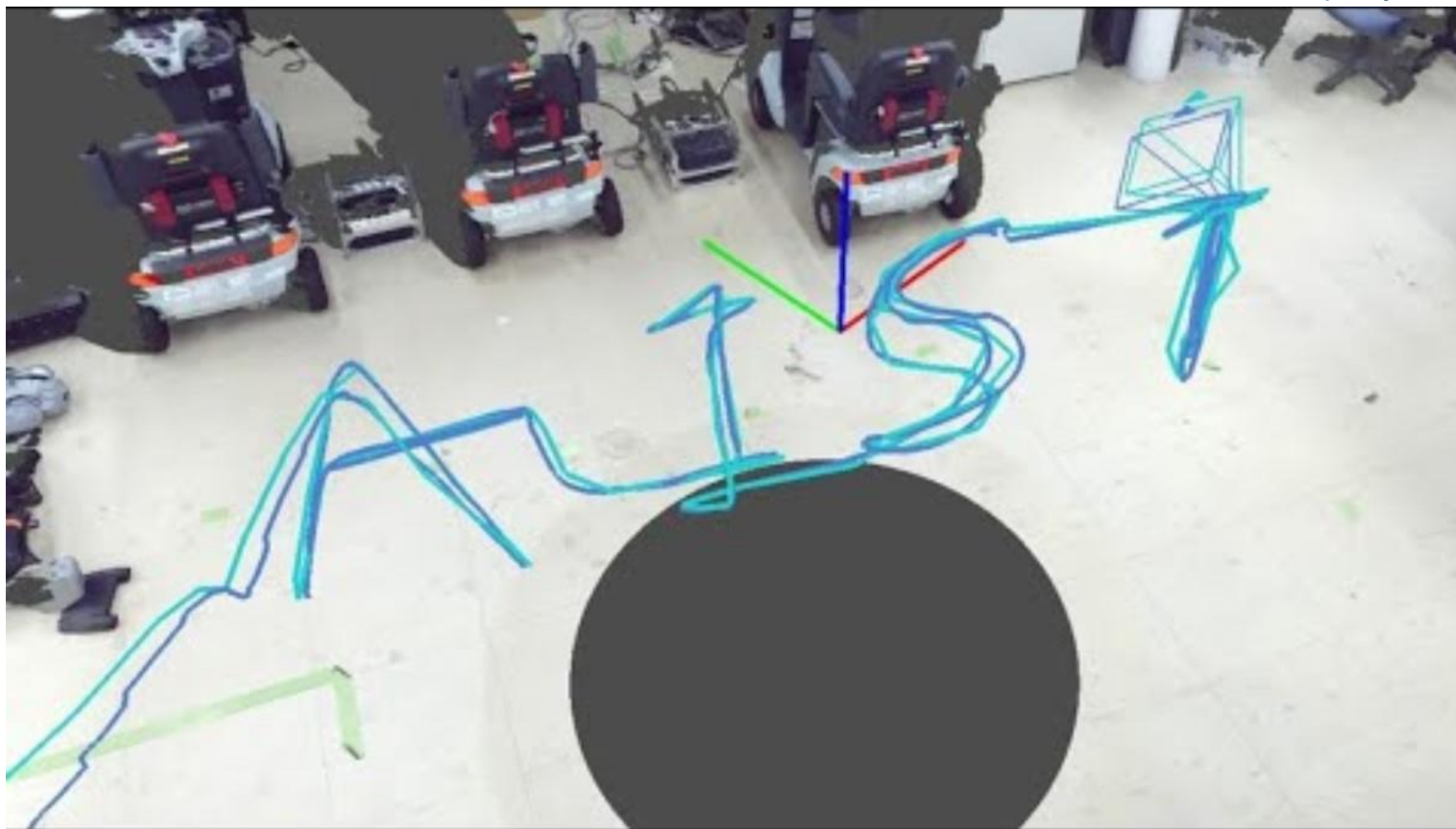


# 1. SLAMの概要: 自己位置推定とは

我々の研究グループの、カメラベースの自己位置推定

動画URL

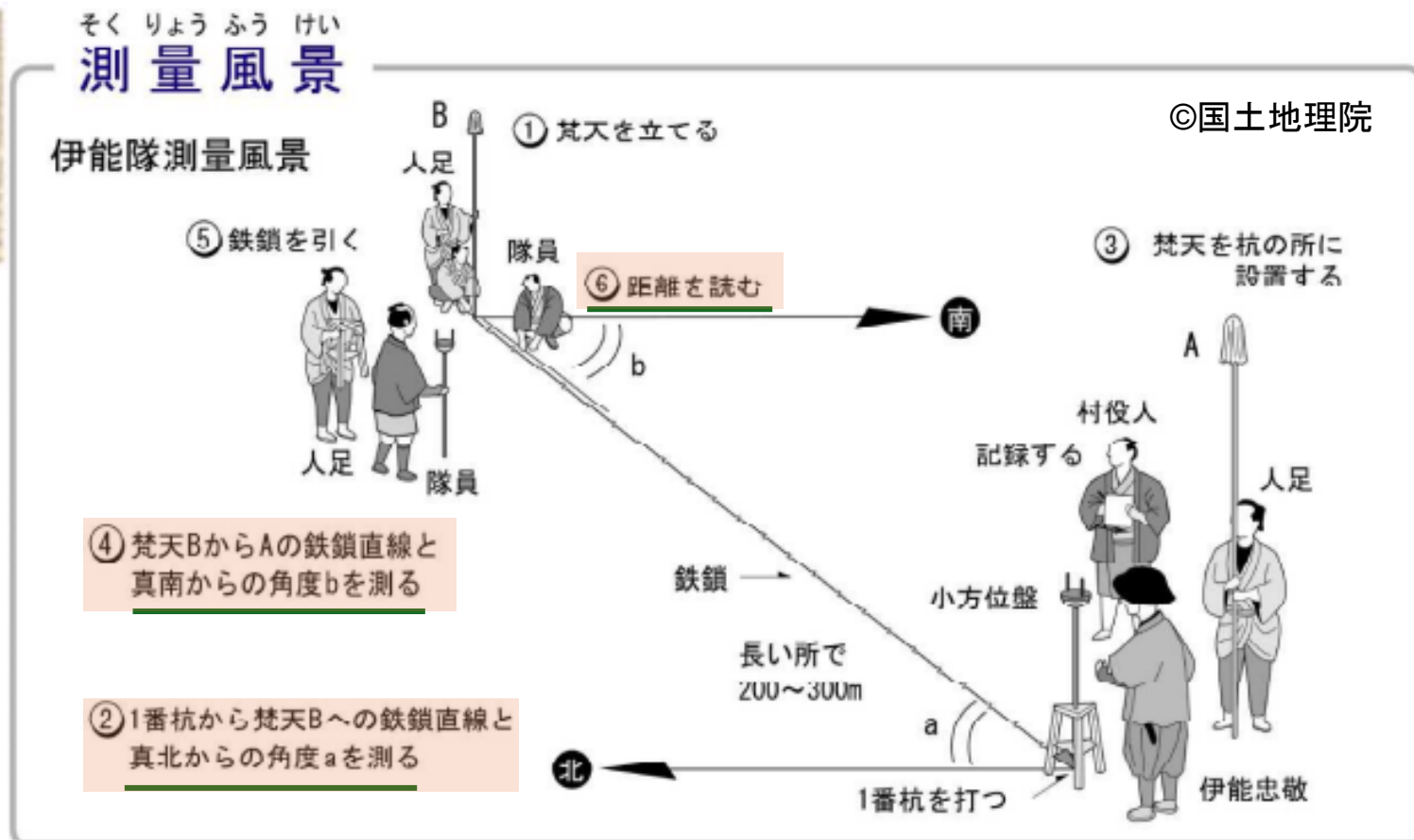
[https://youtu.be/1jdDb7\\_c1lc](https://youtu.be/1jdDb7_c1lc)





# 1. SLAMの概要: 地図構築とは

伊能忠敬による  
「大日本沿海輿地全図」



地図構築には、基準となる観測者(=人間, ロボットなど)の  
位置の逐次推定(計測)が必要である





# 1. SLAMの概要: 地図構築とは

我々の研究グループの、LiDARベースのSLAM(≒地図構築)

動画URL

<https://www.youtube.com/watch?v=p-G98jGfZb4>





# 1. SLAMの概要: SLAMとは

## 自己位置推定

「今の自分自身の位置は地図上のどこ？」

自己位置が先か？  
地図が先か？

## 地図構築

「今見ている風景や地形はどこを基準に見られたもの？」

自分自身の位置を推定すること(=Localization, 自己位置推定)と  
周囲環境の地図を作成すること(=Mapping, 地図構築)を同時に  
おこなう手法:

**S**imultaneous **L**ocalization **a**nd **M**apping (**SLAM**, 「スラム」)





# 1. SLAMの概要: 使用されるセンサ





# 1. SLAMの概要: 使用されるセンサ

---

## カメラ

- 2次元の画像を取得可能
- 解像度や波長は選択可能(可視光, 赤外線, etc)



©SONY



©GoPro



©Logicool



©Apple





# 1. SLAMの概要: 使用されるセンサ

## 3次元カメラ

- 3次元点群画像を取得可能
- 「深度カメラ」「Depthセンサ」とも
- 色情報(RGB)と深度情報(Depth)を同時に撮像可能なRGB-Dカメラが広く流通



©Mitsubishi Research Institute DCS



RealSense D435©Intel



# 1. SLAMの概要: 使用されるセンサ

## 3次元カメラ

- 3次元点群画像を取得可能
- 「深度カメラ」「Depthセンサ」とも
- 色情報(RGB)と深度情報(Depth)を同時に撮像可能なRGB-Dカメラが広く流通



RealSense D435©Intel



BellaBot  
©KAIYO ROBOTICS





# 1. SLAMの概要: 使用されるセンサ

## LiDAR (Light Detection And Ranging, ライダー)

- レーザーを発光して, 周囲環境からの反射光が受光されるまでの時間を計測して3次元点群を取得可能

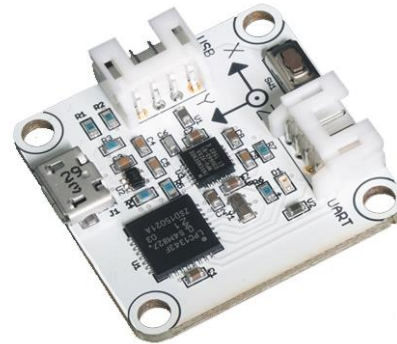
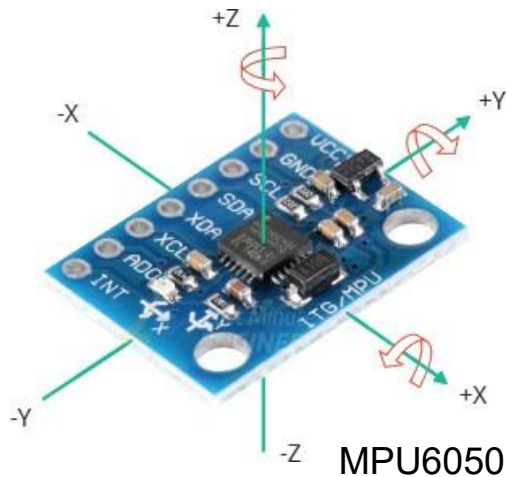




# 1. SLAMの概要: 使用されるセンサ

## IMU (Inertial Measurement Unit, 慣性計測装置)

- 角速度計 (ジャイロセンサ), 加速度計, 地磁気計などが一体となった計測ユニット



USB出力9軸IMUセンサモジュール  
©アールティ



©Nintendo



# 1. SLAMの概要: 使用されるセンサ

## エンコーダ（角度センサ）

- モータの回転数を計測するセンサ
- ロボットの場合は車輪の回転数や脚型ロボットの関節角度を取得するために利用される



©Universal Robots



©TOYOTA





# 1. SLAMの概要: SLAMの需要はなぜ？

- ロボットの自律移動には、地図や自己位置が重要
  - 地図が分からなければ、周囲の環境・目的地の位置などが分からない
  - 自己位置が分からなければ、ロボットが迷子になる

- 地図や自己位置を得る他の手段

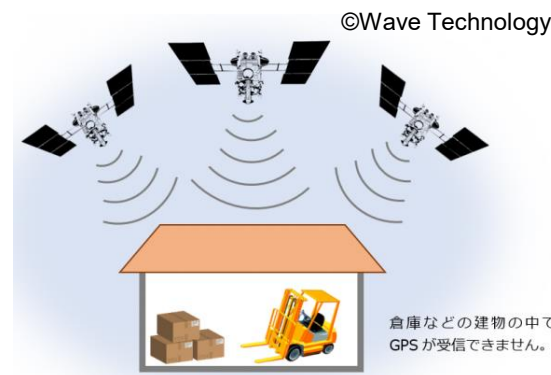
- 高価な測量機器
- GPS、マーカー、磁気テープ

☹️ 高価な機材や、インフラが必要

- SLAMの場合

- LiDAR/カメラを振り回すだけで地図ができる
- インフラがなくても、自己位置を把握できる

😊 お手軽に、多様な条件下でロボットの自律化を可能にするための基盤技術





# 1. SLAMの概要: 実装に必要な知識

- コンピュータビジョン:
  - 画像処理(カメラ) / 点群処理(LiDAR)
    - 2次元画像や3次元点群データを処理するSLAMの基盤技術
    - 画像や点群の幾何関係(最適化計算、数学)から、自己位置の推定が可能
    - 画像や点群を貼り合わせることで地図構築が可能
- 最適化計算(数理最適化):
  - ある関数を最小化(あるいは最大化)して所望の目的のために最良の値を計算
    - 複雑で手計算では解けない大規模問題をコンピュータで解くことが可能
  - SLAMは、最適化計算によって実行
    - 最適化計算は、本セミナーでのメイン
      - SLAMの高精度化には、必要不可欠
- ロボット工学、数学の基礎:
  - ロボット工学で用いる三角関数、行列計算、微分積分、確率統計などの基礎知識
  - ロボットの運動の記述や、最適化計算の処理で活用

自律ロボットを作るには、  
多様な知識が必要  
自律ロボットの一部であるSLAM  
においても、多様な知識が必要



# 本日本話しする内容

---

1章: SLAMの概要

**2章: SLAMの基礎理論**

3章: オドメトリ推定手法

4章: センサ融合によるSLAMの高性能化

5章: SLAMの実装技術とそのポイント

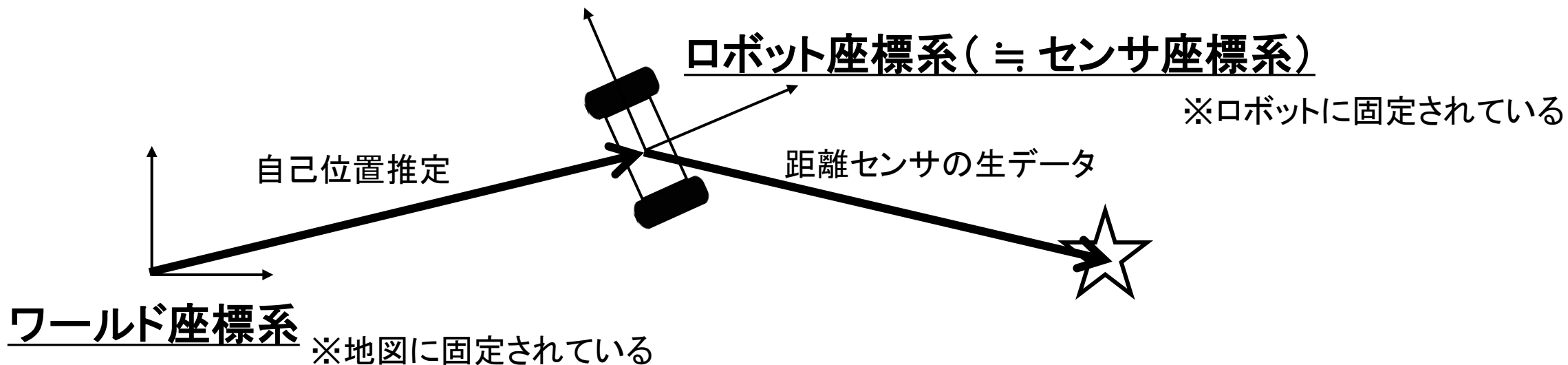
6章: SLAMの実例・最新研究動向の紹介





## 2. SLAM基礎: 座標系(ロボット工学の基礎)

- 座標系・座標変換は、**自己位置や地図を数式で表現するための基盤**
- 座標系とは
  - ロボットポーズやセンサデータを記述する際の**基準**
  - 例
    - *自己位置推定* → **ワールド座標系基準**のロボットのポーズを求める
    - *距離センサの生データ* → **距離センサ座標系基準**の物体までの位置ベクトルを測定する  
ロボットが動けば、センサデータの値が変わる

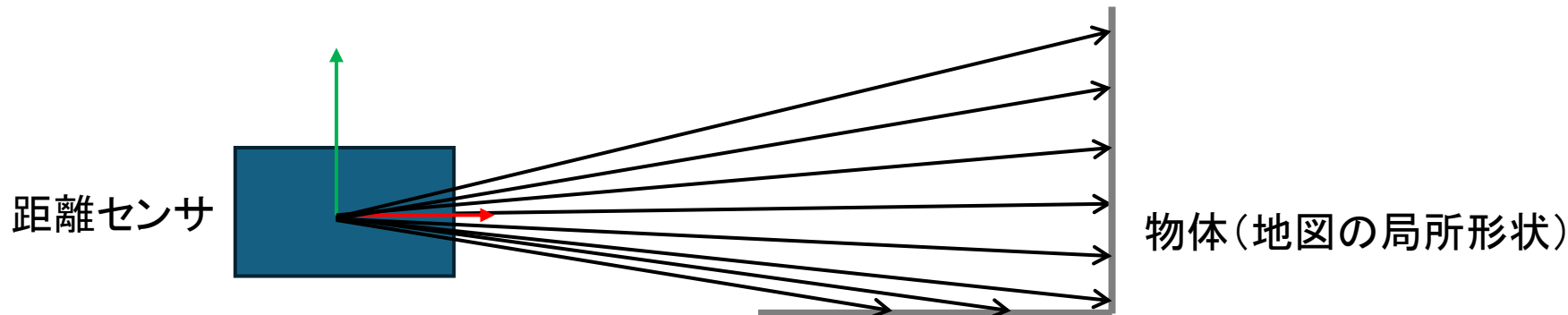




## 2. SLAM基礎: 座標変換(地図構築に関して)

- 距離センサのデータ

- センサ座標から見た物体までの位置ベクトル集合(=地図の局所形状)を計測



- 距離センサのデータから地図を作るには

- 距離センサのデータ(=地図の局所形状)をつなぎ合わせる操作が必要
- 座標変換が肝

↓ どのような座標変換？

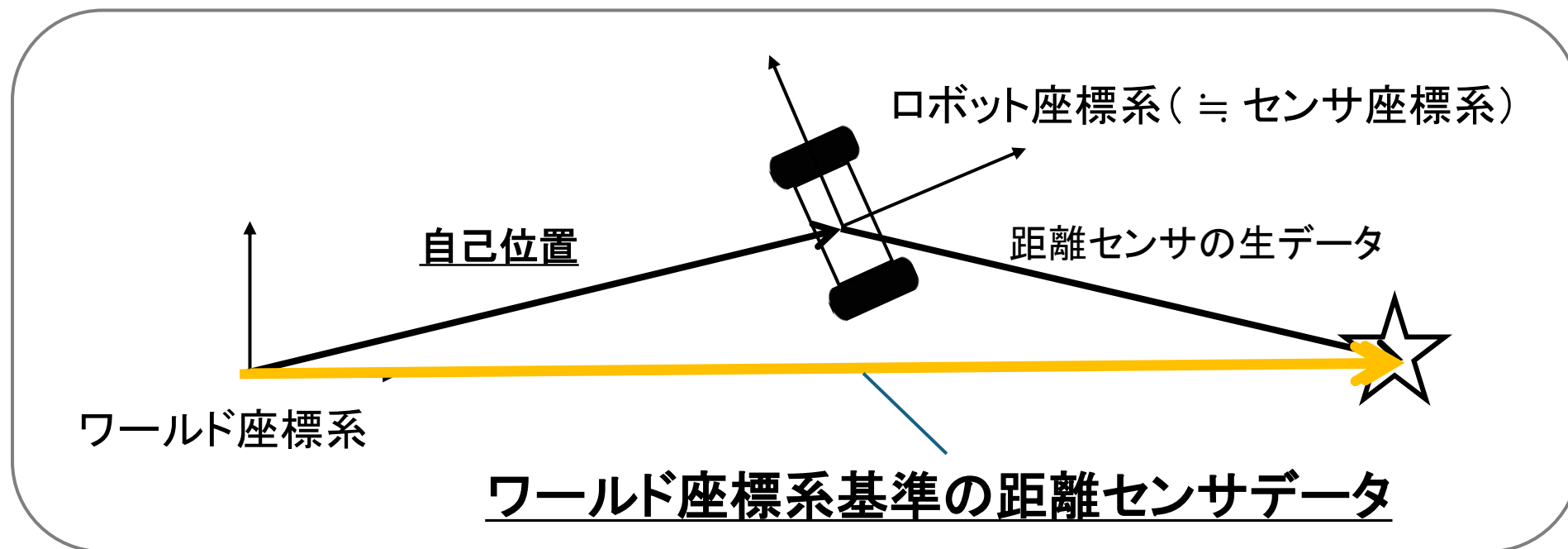


## 2. SLAM基礎: 座標変換(地図構築に関して)

- 地図構築に必要な座標変換

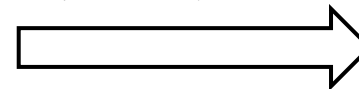
- **センサ座標系基準**の距離センサのデータを**ワールド座標系基準**に変換

→ ワールド座標系基準のセンサのポーズが分かれば座標変換(地図構築)が可能  
= 自己位置(自己位置推定の結果)



概念図

自己位置は  
どう記述するか?

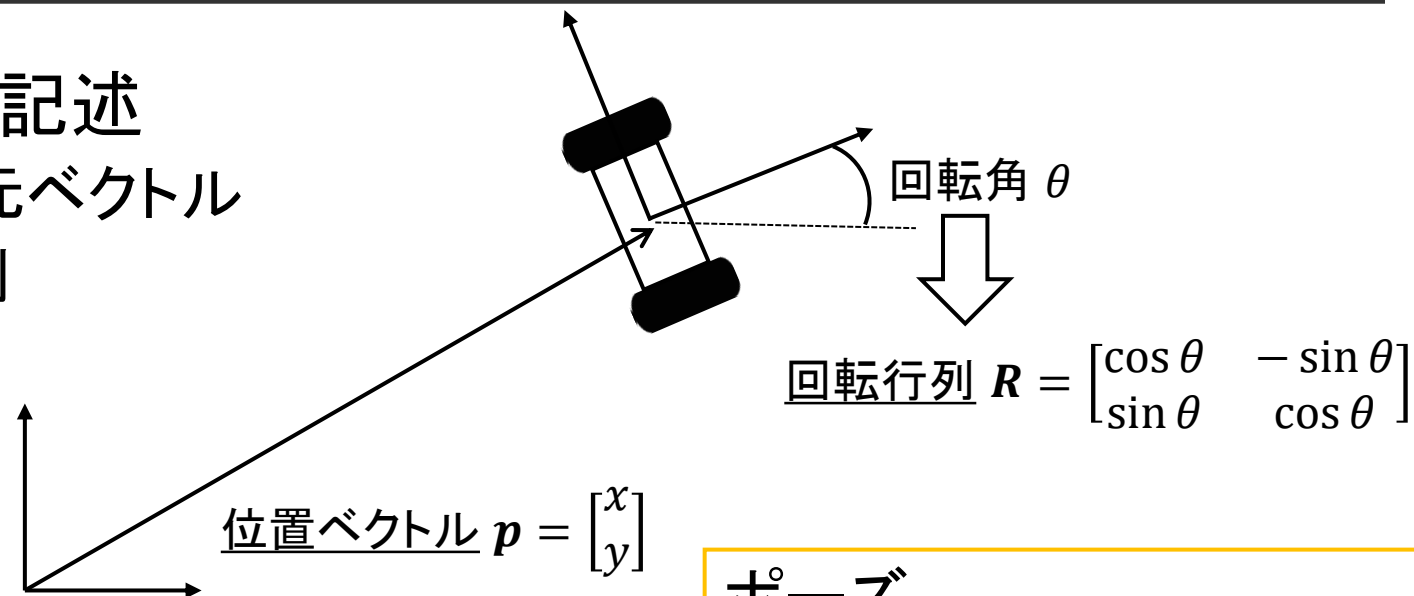






## 2. SLAM基礎: 座標変換(自己位置に関して)

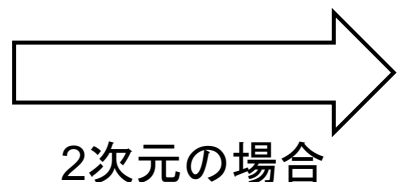
- 位置と姿勢による自己位置の記述
  - 位置  $p$ : 2次元ベクトル or 3次元ベクトル
  - 姿勢  $R$ :  $2 \times 2$ 行列 or  $3 \times 3$ 行列
    - 位置 & 姿勢 = ポーズ



ポーズ  
= 位置ベクトル & 回転行列

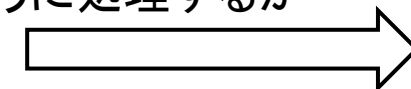
- 別のポーズの表現方法
  - 同次変換行列  $T$ : 位置と姿勢を1つの行列に集約

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}$$



$$T = \begin{bmatrix} \cos \theta & -\sin \theta & x \\ \sin \theta & \cos \theta & y \\ 0 & 0 & 1 \end{bmatrix}$$

SLAMの場合、自己位置推定をどのように処理するか



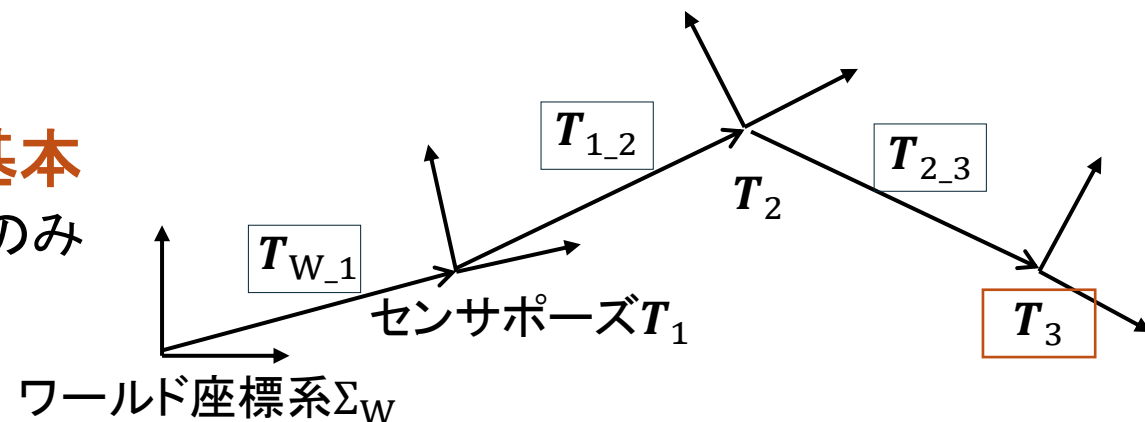


## 2. SLAM基礎: 座標変換(自己位置に関して)

- 自己位置推定の処理方法

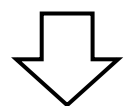
- SLAMではオドメトリ(移動量の積算)が基本**

- センサで取得できるのは、移動量( $T_{1\_2}$ など)のみ
- 自己位置( $T_3$ )は、移動量の積算で計算



- 同次変換行列による移動量の積算

- $T_3 = T_{W\_3} = T_{W\_1} T_{1\_2} T_{2\_3}$ 
  - 同次変換行列の合成、チェーン(間の1、2は消去されるイメージ)
- 各移動量が分かれば、自己位置が暫定的に計算できる
- 地図もまた暫定的に構築できる



センサデータの座標変換(≈ 地図構築)には、自己位置の情報が必要であった



## 2. SLAM基礎: 座標変換

### • 距離センサデータの座標変換(≈地図構築)

#### • 記号

- ワールド座標系基準のセンサポーズ:  $T_{W\_S3}$
- センサ座標系基準の距離センサから物体までの位置ベクトル:  $p_{S3}$

#### • ワールド座標系基準のセンサデータ

$$p_w = T_{W\_S3} p_{S3}$$

厳密には...

$$\begin{bmatrix} p_{w,x} \\ p_{w,y} \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & x \\ \sin \theta & \cos \theta & y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} p_{R3,x} \\ p_{R3,y} \\ 1 \end{bmatrix}$$

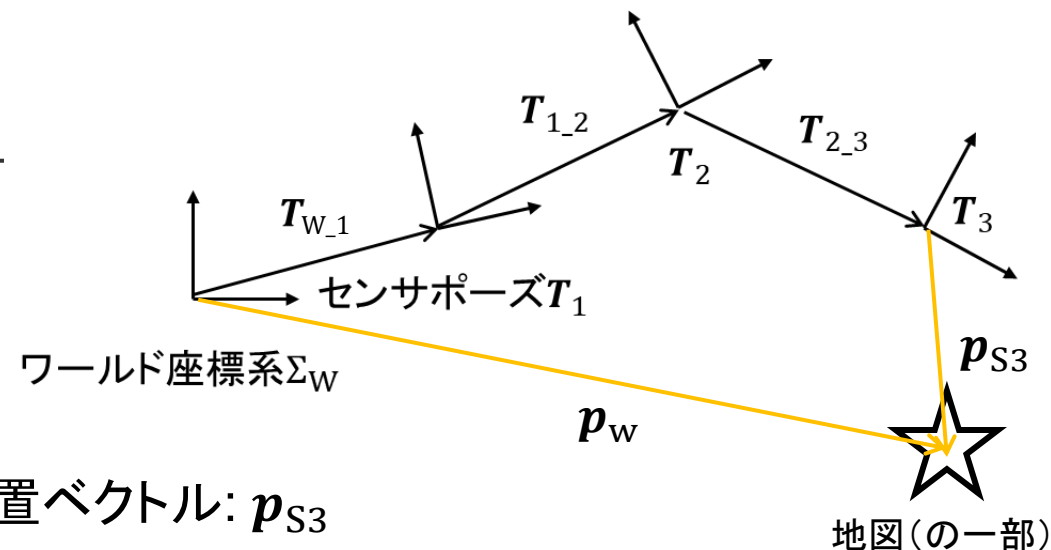
$p_w$                        $T_{W\_S3}$                        $p_{S3}$

同次変換行列での座標変換のための**辻褄合わせ**

### • 座標変換のルール

$$p_A = T_{A\_B} p_B$$

B座標系基準のベクトルをA座標系基準に変換するルール  
→ 同次変換行列 $T_{A\_B}$ があれば可能





## 2. SLAM基礎: 座標系・座標変換 まとめ

- ロボットのポーズ(位置 & 姿勢)は、同次変換行列 $T$ で表現
- 移動量の積算に基づく自己位置の記述
  - $T_3 = T_{W_3} = T_{W_1} T_{1_2} T_{2_3}$   $T$ : 同次変換行列(位置と回転を同時に表現)
  - 自己位置推定に関連
- センサ座標系基準のセンサデータ $p_{S3}$ をワールド座標系基準に座標変換
  - $p_W = T_{W_{S3}} p_{S3}$
  - 地図構築に関連
- 座標変換のルール
  - $p_A = T_{A_B} p_B$
- SLAMにおける座標系・座標変換の役割
  - 自己位置や地図を数式的に記述する際の基盤
    - センサやモデルに誤差が一切ない場合、座標変換の式だけで自己位置推定と地図構築が可能
    - 誤差は必ず含まれるので、それを対処して矛盾の無い自己位置推定・地図構築を行うのがSLAM





## 2. SLAM基礎: グラフベースSLAM

- グラフとは

- ノードとエッジの集合
  - ノード: 最適化変数
  - エッジ: 拘束

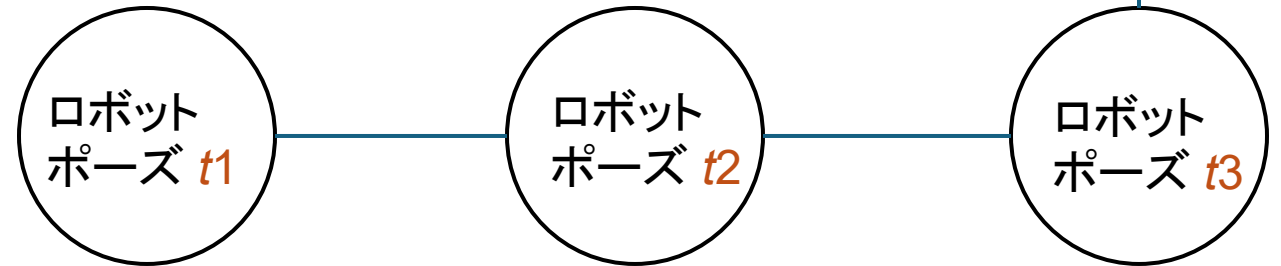
- グラフベースSLAM

- グラフによって表現されるSLAM、**最近の主流**
  - センサ情報を用いて、ノードに対してエッジ(拘束)を定義してグラフを構築
  - グラフを最適化することで、ノード(最適化変数: 各時刻のロボットポーズ)を推定する問題

### グラフ

ノード(最適化変数)

エッジ(拘束)

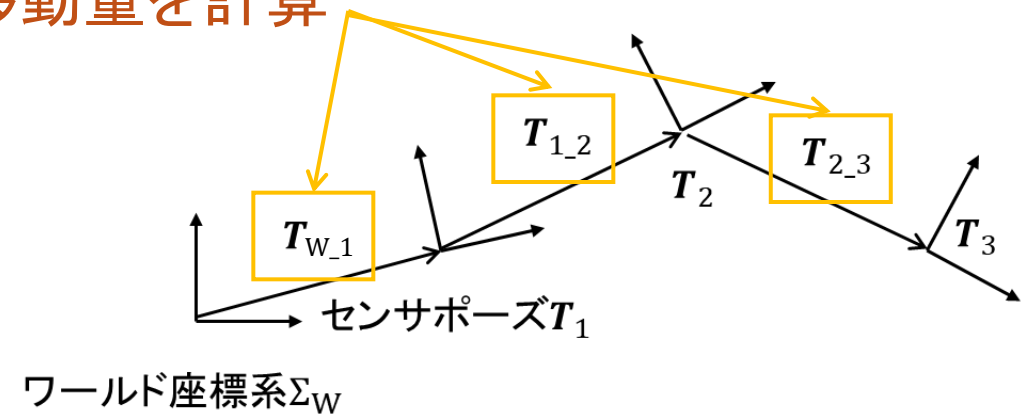




## 2. SLAM基礎: グラフベースSLAM

- エッジ(拘束)の取得方法

- オドメトリが基本→センサからロボットの移動量を計算
- オドメトリを得るためのセンサ
  - ホイールエンコーダ
  - IMU
  - カメラ
  - LiDAR

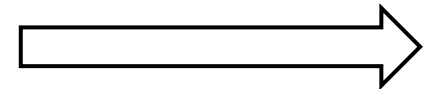


- オドメトリの問題点

- 移動量の誤差が蓄積

- 例:  $T_3 = T_{W_3} = T_{W_1} T_{1_2} T_{2_3}$ 
      - $T_3$ には、 $T_{W_1}$ ,  $T_{1_2}$ ,  $T_{2_3}$ の誤差が蓄積する
      - $T_{W_1}$ ,  $T_{1_2}$ ,  $T_{2_3}$ に誤差は必ず含まれる

ちなみに、地図構築  
はどうなるか？



- オドメトリだけでは、正確な自己位置推定が困難→SLAMのモチベーション



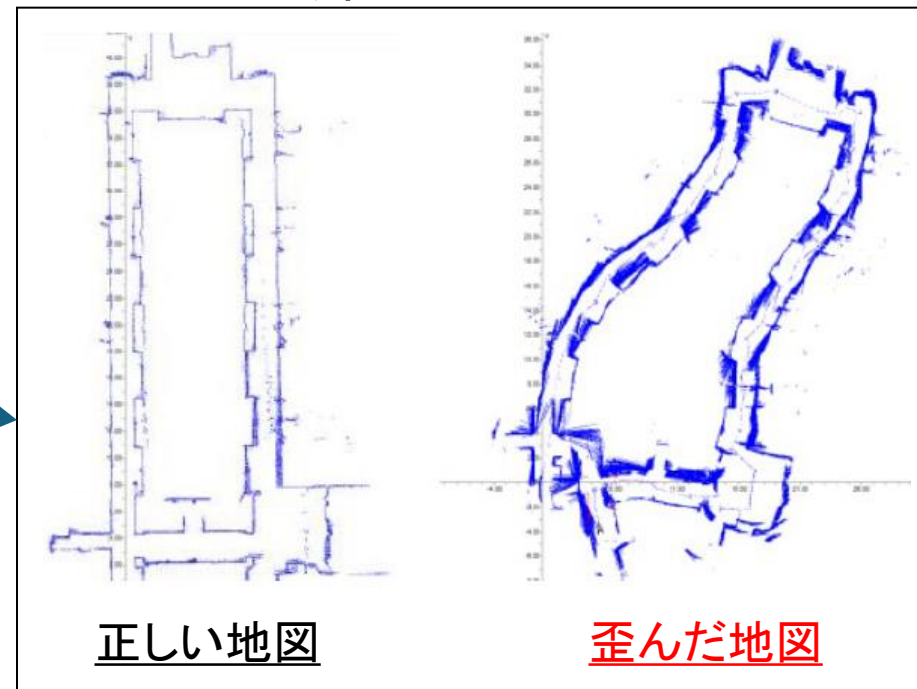
## 2. SLAM基礎: グラフベースSLAM

- オドメトリに基づいて地図構築した場合
  - ワールド座標系基準のセンサデータ(地図の一部)の計算
    - $T_3 = T_{W_3} = T_{W_1} T_{1_2} T_{2_3}$  (1章 26ページの式)
    - $p_w = T_{W_3} p_{S3}$ 
      - $p_{S3}$ にも誤差がある
  - $T_{W_3}$ には、 $T_{W_1}$ ,  $T_{1_2}$ ,  $T_{2_3}$ の誤差が蓄積
  - →  $p_w$ にも上記の蓄積誤差が作用、かつ支配的
    - → **地図が歪む**



- SLAMのモチベーション
  - 推定における誤差の蓄積を解消
  - → **一貫性のある自己位置・地図を推定**

廊下の地図



横山和ほか, “測域センサを用いた移動ロボットの環境地図構築及び歪み補正に関する検討”, 計測自動制御学会東北支部第274 回研究集会, No. 274-8, 2012.



## 2. SLAM基礎: グラフベースSLAM

- 蓄積誤差の解消方法

- 過去の情報を活用することが肝 → **ループ閉じ込み(ループ拘束)**
  - 時間的に離れた11と1(右図)の間の相対的な関係を直接定義することがカギ**
  - 11と1で**同一のランドマークを観測**することでループ拘束を定義可能

- オドメトリとループによる拘束の考え方**

- オドメトリ拘束: 局所的な整合性を保証
- ループ拘束: 大域的な整合性を保証
  - ループ拘束により、オドメトリの問題(蓄積誤差)を解消**

- オドメトリ拘束とループ拘束によるSLAMの定式化

- ポーズ調整(ポーズグラフ最適化)**という手法を使用



<https://www.youtube.com/watch?v=bWmjo9zONLE>

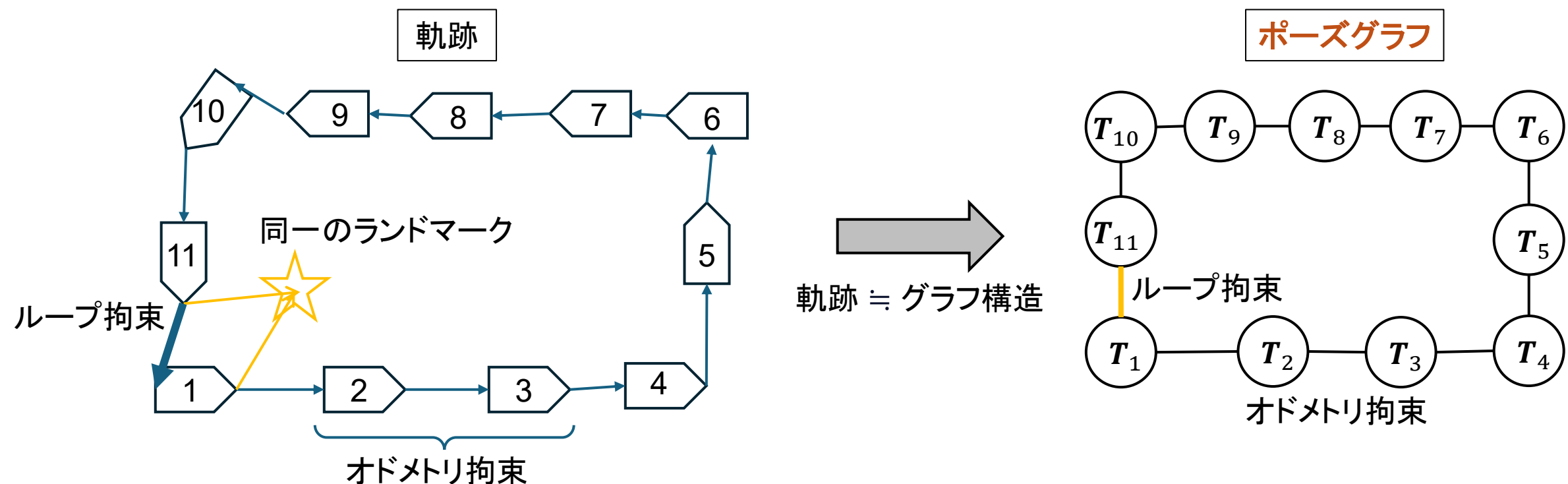






## 2. SLAM基礎: ポーズグラフ

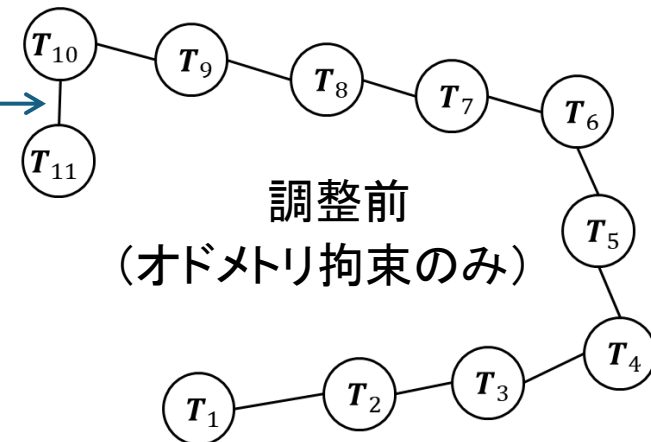
- ポーズグラフ: オドメトリ拘束とループ拘束のみに基づくグラフ
  - エッジ(拘束): オドメトリ拘束 ( $T_{1_2}, T_{2_3}, \dots, T_{10_11}$ )、ループ拘束 ( $T_{11_1}$ )
  - ノード(最適化変数): ワールド座標系基準の各時刻のポーズ ( $T_1, \dots, T_{11}$ )



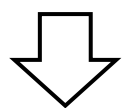


## 2. SLAM基礎: ポーズグラフ

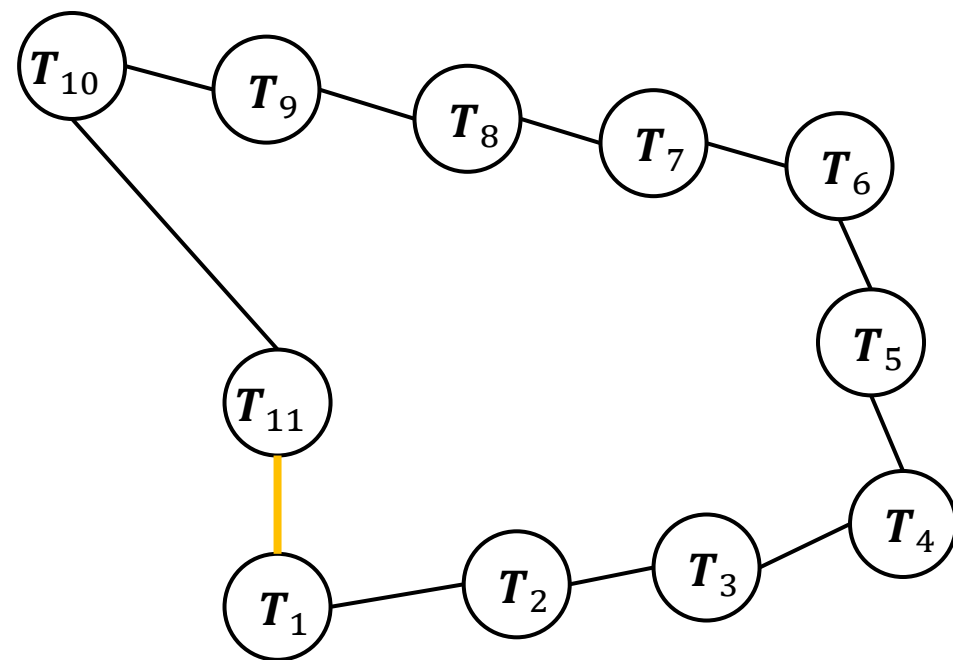
局所的関係  
は正しい



- ポーズグラフでSLAMを解く際のイメージ
  - オドメトリ拘束は、局所的な整合性を保つ(右図参照)
  - ループ拘束は、大域的な整合性を保つ
    - $T_{11_1}$ は、のような相対ポーズを主張
- 両方の拘束を考慮するには？
  - 両方の主張を良い感じに取り入れる
  - → **最適化**が使われる
    - 何かの基準に基づくことで、両方の拘束を考慮可能
  - → **局所的、大域的な整合性を保つSLAMが実現**



ポーズグラフを記述する際の一般的な仮定



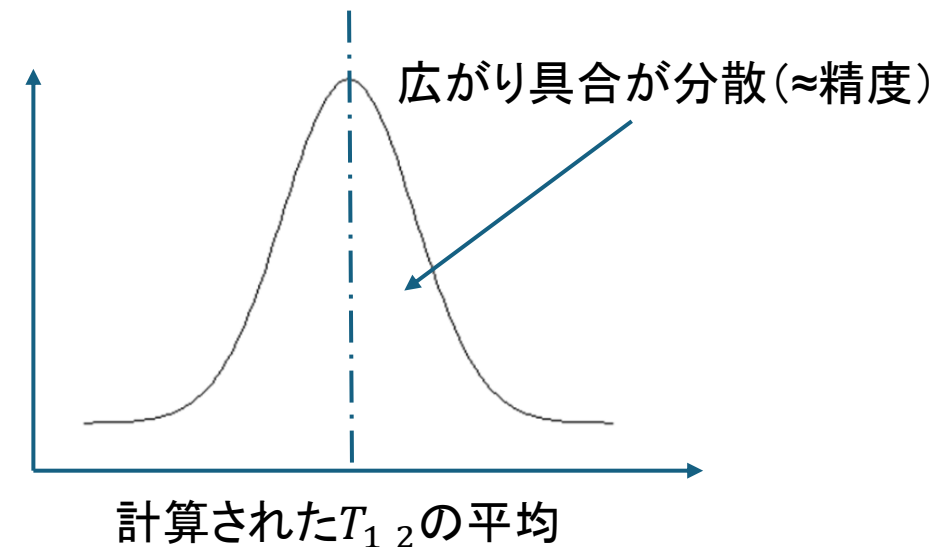
グラフ構造とロボットポーズを合わせた図  
→ グラフ構造  $\equiv$  軌跡



## 2. SLAM基礎: ポーズグラフによるグラフベースSLAM

- SLAMにおける誤差に関する仮定

- 全ての観測(例:  $T_{1\_2}$ )には誤差がある
  - センサに誤差があり、そのセンサで $T_{1\_2}$ を求めたから。
- 誤差は、正規分布と仮定することが一般的
  - 真の $T_{1\_2}$  = 計算された $T_{1\_2}$  + 正規分布に基づく誤差
- → 観測など(例:  $T_{1\_2}$ )は、正規分布として扱う
  - 計算された $T_{1\_2}$ は平均
  - その平均の精度は分散(共分散行列)



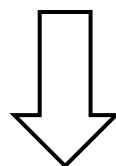
- SLAMにおけるエッジやノードの扱い

- 全ての誤差を正規分布として扱う
  - 正規分布としたエッジを定義
  - 最適化変数も正規分布として計算



## 2. SLAM基礎: ポーズグラフによるグラフベースSLAM

- ポーズグラフによるSLAMの最適化
  - 最適化とだけ説明したが、何を最適化するのか？
  - → 誤差関数 = 最適化変数と観測の差分 を最適化
    - 最適化問題: 誤差関数が最小になるように最適化変数(今回は、ロボットポーズ)を調整
    - → 「ポーズ調整(Pose adjustment)」という技術 (Pose Graph Optimizationとも呼ばれる)
    - → オドメトリ拘束(局所的な整合性)+ループ拘束(大域的整合性)を適切に調整するイメージ



ポーズ調整の具体的な説明の前に...

- ロボットポーズ( = 同次変換行列  $T$  )を最適化する際の面倒さ
  - 誤差関数はポーズの差分だが、単なる引き算では表現不可能 (NG:  $T_{1_2} - \hat{T}_{1_2}$ )
    - $T$  は、回転行列 $R$ と位置ベクトル $p$ から形成される
    - → 色々な制約をもつ回転行列 $R$ が原因

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}$$





## 2. SLAM基礎: ポーズグラフによるグラフベースSLAM

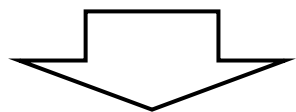
- 回転行列の色々な制約(=回転行列の定義)

- 列ベクトル( $v_1, v_2, v_3$ )はそれぞれ**正規直交**

- $v_1, v_2, v_3$ のノルムはそれぞれ1
    - $v_1, v_2, v_3$ のそれぞれの内積は0

$$R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

$v_1 \quad v_2 \quad v_3$



- **$R$ の引き算には物理的な意味がない(回転行列ではない)**

- 回転行列の定義から逸脱するから
  - → 同様に、 $T$ の引き算も物理的な意味がない
  - → **ポーズの差分には、引き算は不適となる**

$R_1 - R_2$ は、回転行列の定義を満たさない  
→  $R_1 - R_2$ は、**回転行列ではない**  
→  $T_1 - T_2$ も、**回転行列ではない**

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}$$



## 2. SLAM基礎: ポーズグラフによるグラフベースSLAM

- ポーズに関する差分の適切な表現方法

- $T_{1\_2}$ と $\hat{T}_{1\_2}$ の適切な差分の表現方法:  $T_{1\_2}^{-1}\hat{T}_{1\_2}$

- ポーズの差分の表現には、逆行列を使う

- 差分が小さいと、 $T_{1\_2}^{-1}\hat{T}_{1\_2}$ は単位行列に近づく

- ポーズグラフによるSLAMの誤差関数の表現方法

- $E_{1,2} = T_{1\_2}^{-1}\hat{T}_{1\_2} = (T_1^{-1}T_2)^{-1}\hat{T}_{1\_2}$

最適化変数相当

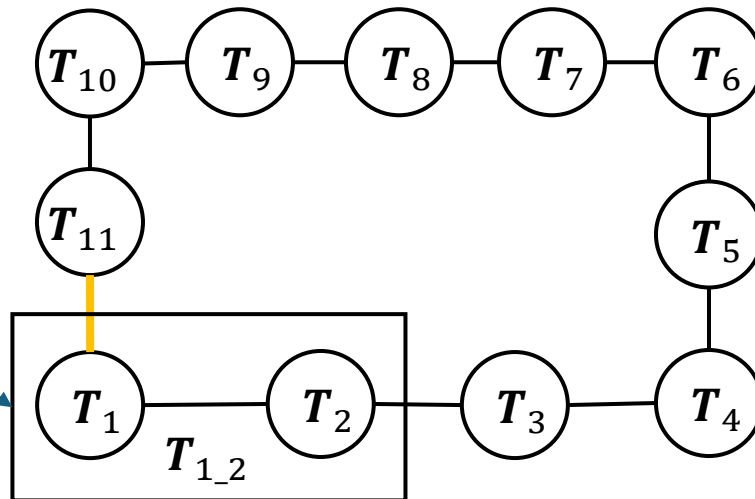
計測値

最適化変数 $T_1$ 、 $T_2$ で表現

実際には最適化変数ではないので変形

誤差関数の物理的意味  
最適化変数から定義される相対ポーズ、  
センサから計測された相対ポーズの差分

最後にもう一山  
式変形を行う





## 2. SLAM基礎: ポーズグラフによるグラフベースSLAM

- Log mapの導入

- 相対ポーズに関する誤差関数:  $E_{1,2} = T_{1\_2}^{-1} \hat{T}_{1\_2} = (T_1^{-1} T_2)^{-1} \hat{T}_{1\_2}$ 
  - $E_{1,2}$ は $3 \times 3$  (二次元平面の場合)、もしくは  $4 \times 4$  (三次元空間の場合)の行列
  - → **ポーズが冗長に表現されていて、最適化が大変**
    - 二次元の場合、 $E_{1,2}$ は9個 ( $3 \times 3$ ) のパラメータだが、ポーズは3個のパラメータで表現可能
  - → **行列 (冗長なパラメータ) をベクトル (最低限なパラメータ) に写像したい**

- Log map

- 同次変換行列をベクトルに写像する操作 (二次元平面:  $\mathbb{R}^3$ 、三次元空間:  $\mathbb{R}^6$ )

- Log mapによって,  $E_{1,2}$  (同次変換行列) をベクトルとして表現

- $E_{1,2} = \log \left( (T_1^{-1} T_2)^{-1} \hat{T}_{1\_2} \right)$  ← 行列ではなくベクトルとして誤差を表現

- → 誤差の大きさを  $E_{12}^T E_{12}$  で表現可能

- → **誤差の大きさが評価関数**

- 近年のSLAMの論文でよく見る表現

Log mapを正面から理解しようとする、結構時間がかかるので、冗長なポーズの表現を、最低限の次元で表現して、最適化が楽になるぐらいの粒度の説明に留めます。



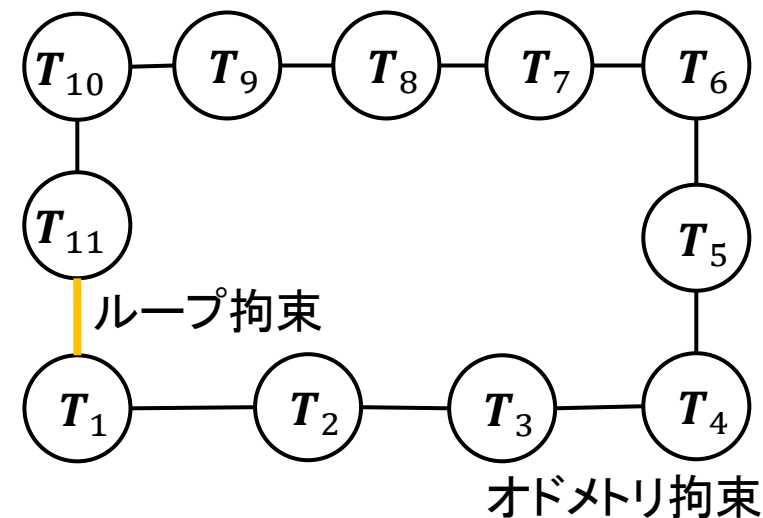
## 2. SLAM基礎: ポーズグラフによるグラフベースSLAM

- ポーズグラフによるSLAMのおさらい
  - オドメトリ拘束、ループ拘束の両方を最適化
  - 最適化するのは評価関数(誤差の大きさ相当)

- **ポーズ調整の評価関数  $E$**

$$E = \underbrace{\sum E_{i-1,i}^T \Sigma_{i-1,i}^{-1} E_{i-1,i}}_{\text{オドメトリ拘束}} + \underbrace{\sum E_{s,t}^T \Sigma_{s,t}^{-1} E_{s,t}}_{\text{ループ拘束}}$$

全体の誤差の大きさ



- $E$ は、全てのオドメトリ拘束、ループ拘束の和として定義(→ 両方の拘束を考慮可能)
- $\Sigma_{i-1,i}^{-1}$ 、 $\Sigma_{s,t}^{-1}$ は共分散行列
  - 重みに相当する(例: 分散が小さい→精度が高い→重みが大きい。逆も同様)
  - 拘束(エッジ)は正規分布として表現するのだった





## 2. SLAM基礎: ポーズグラフによるグラフベースSLAM

- ポーズ調整 (Pose Graph Optimization) まとめ
  - 評価関数 $E$  (全てのオドメトリ拘束、ループ拘束の和)を最小化する問題
    - 局所的な整合性・大域的な整合性を保証した、矛盾の無い自己位置推定・地図構築が可能
    - → **これこそがSLAMの本質(5章でデモを通して詳しく説明)**
- 具体的に、どのように最適化するのか？
  - 非線形最小二乗問題として定式化
    - 評価関数 $E$ は最小二乗の形
    - 姿勢(回転行列)は三角関数から定義される→非線形
  - →ガウスニュートン法などで解くことが可能
    - 繰り返し計算に基づく数値計算

この最適化の過程で、log mapの恩恵を得る。

Log mapを使わないと、冗長なパラメータのポーズを最適化することになる。

Log mapを使うと、必要最低限のパラメータのポーズを最適化すればOK。

※ 詳細は省略

ポーズ調整の評価関数  $E$

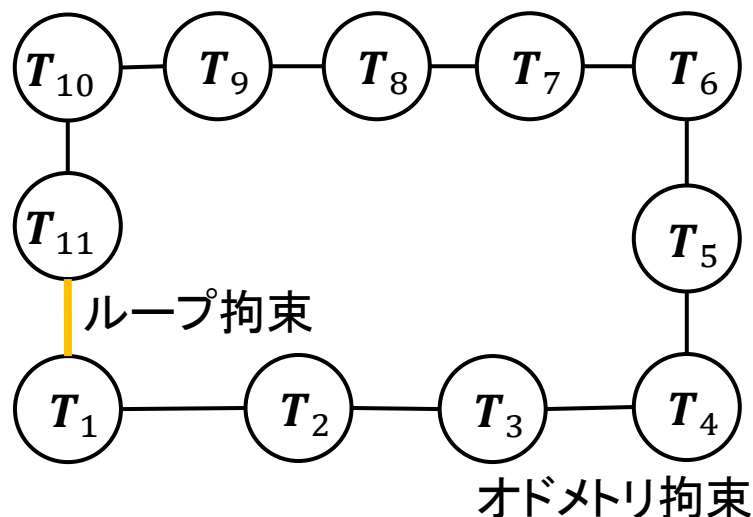
$$E = \underbrace{\sum E_{i-1,i}^T \Sigma_{i-1,i}^{-1} E_{i-1,i}}_{\text{オドメトリ拘束}} + \underbrace{\sum E_{s,t}^T \Sigma_{s,t}^{-1} E_{s,t}}_{\text{ループ拘束}}$$



## 2. SLAM基礎: ポーズグラフによるグラフベースSLAM まとめ

1. オドメトリに基づく相対ポーズ $\hat{T}_{1_2}, \hat{T}_{2_3}, \dots, \hat{T}_{i_{i+1}}$ を逐次取得
2. 対応するオドメトリ拘束を逐次構築
3. 現在の観測において、過去に観測したランドマークと同一のランドマークを観測した場合、ループ拘束 $\hat{T}_{s_t}$ を構築
4. 評価関数 $E$ を最適化することで、自己位置、地図を推定(ポーズ調整)

ポーズグラフ



ポーズ調整の評価関数

$$E = \underbrace{\sum E_{i-1,i}^T \Sigma_{i-1,i}^{-1} E_{i-1,i}}_{\text{オドメトリ拘束}} + \underbrace{\sum E_{s,t}^T \Sigma_{s,t}^{-1} E_{s,t}}_{\text{ループ拘束}}$$

$E$ : 誤差ベクトル(同次変換行列にlog mapを適用)  
 $\Sigma$ : 共分散行列

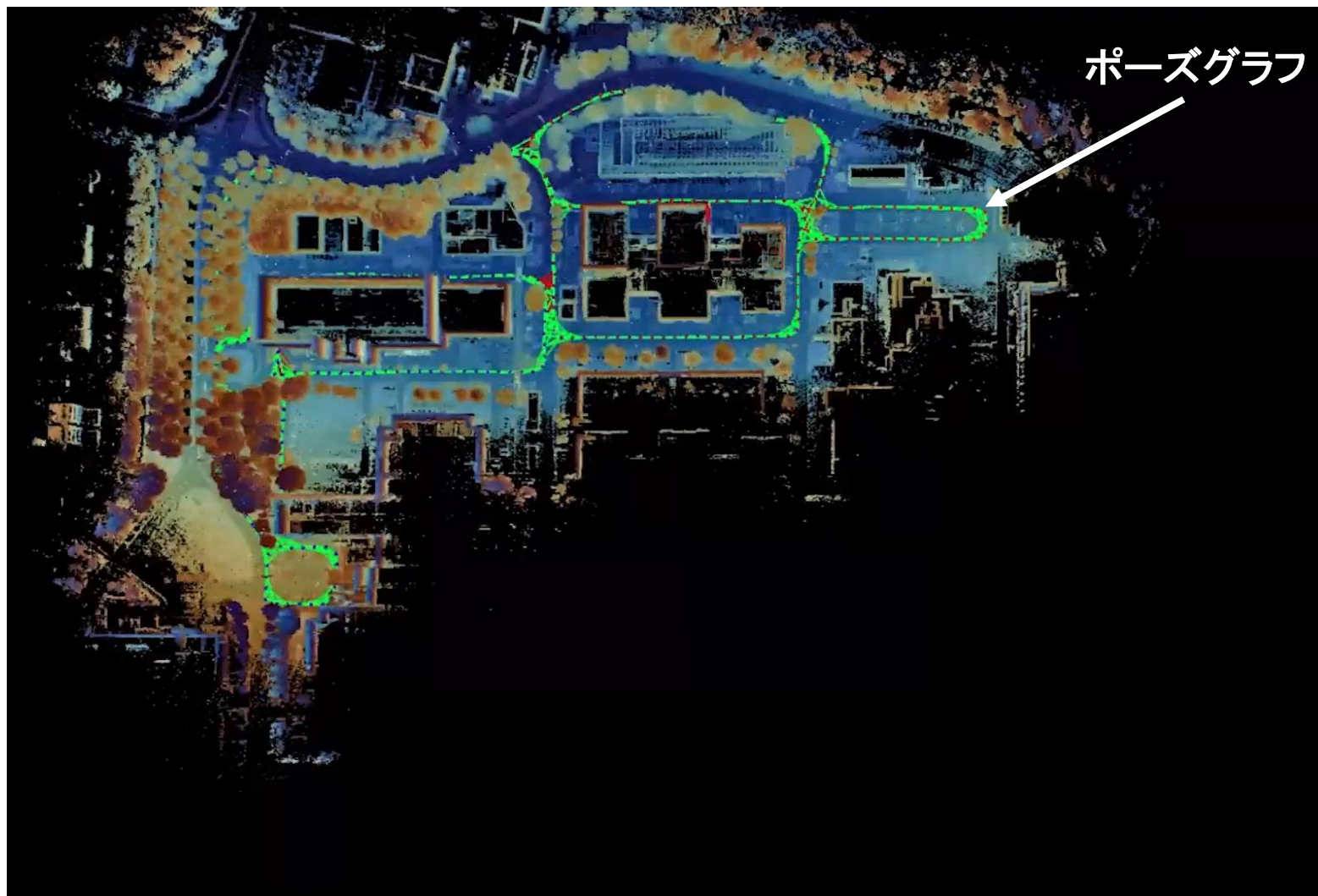


## 2. SLAM基礎: ポーズ調整によるグラフベースSLAMの例

### ポーズ調整による3D SLAMの例

ポーズ調整については、5章でサンプルプログラムを通して詳細に説明します。

<https://www.youtube.com/watch?v=bWmjo9zONLE>



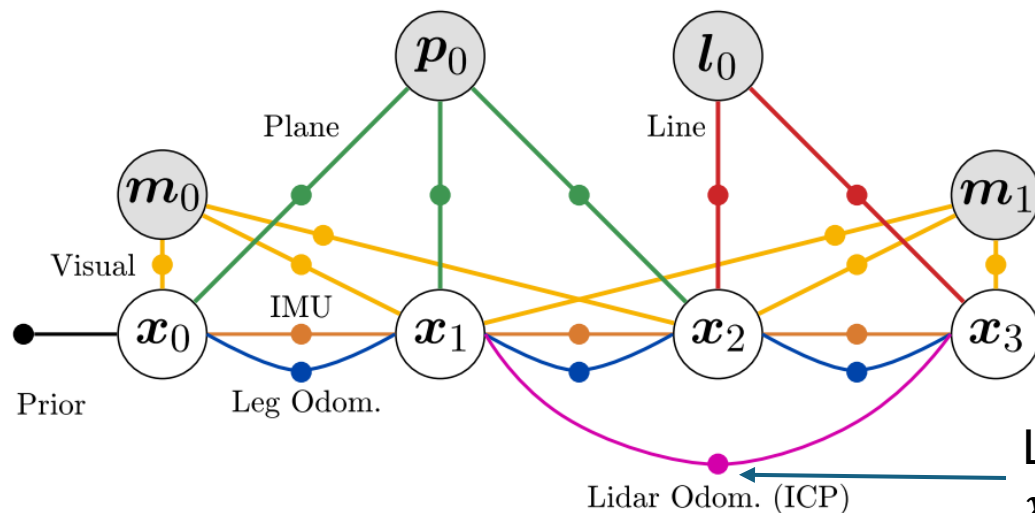
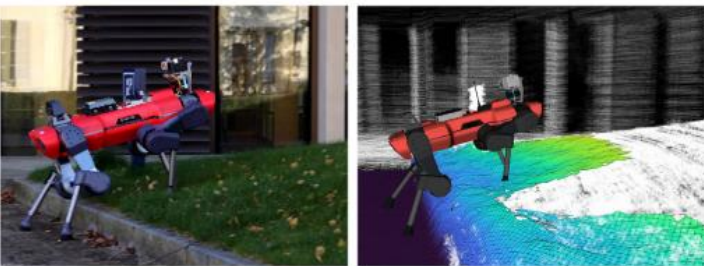


## 2. SLAM基礎: ファクタグラフ

### • ポーズグラフを一般化させたグラフ

- ノード(最適化変数): ポーズ以外でもOK(速度、滑りなどの未知パラメータなど)
- エッジ(拘束): 相対ポーズ以外でもOK(相対速度、未知パラメータ同士の関係など)
  - 複雑なシステム(例: センサ融合のSLAM)を作るときに便利
- エッジ上の丸い点: 関連する最適化変数に関する誤差関数(ファクタ)を表現

VILENSという手法の  
ファクタグラフ



ファクタグラフの特徴

- ロボット速度も推定
- IMUの誤差(バイアス)も推定
- 速度に関する誤差関数も定義
- その他多くの誤差関数を定義

LiDAR Odomの誤差関数は、  
 $x_1$ と $x_3$ の最適化変数から定義される

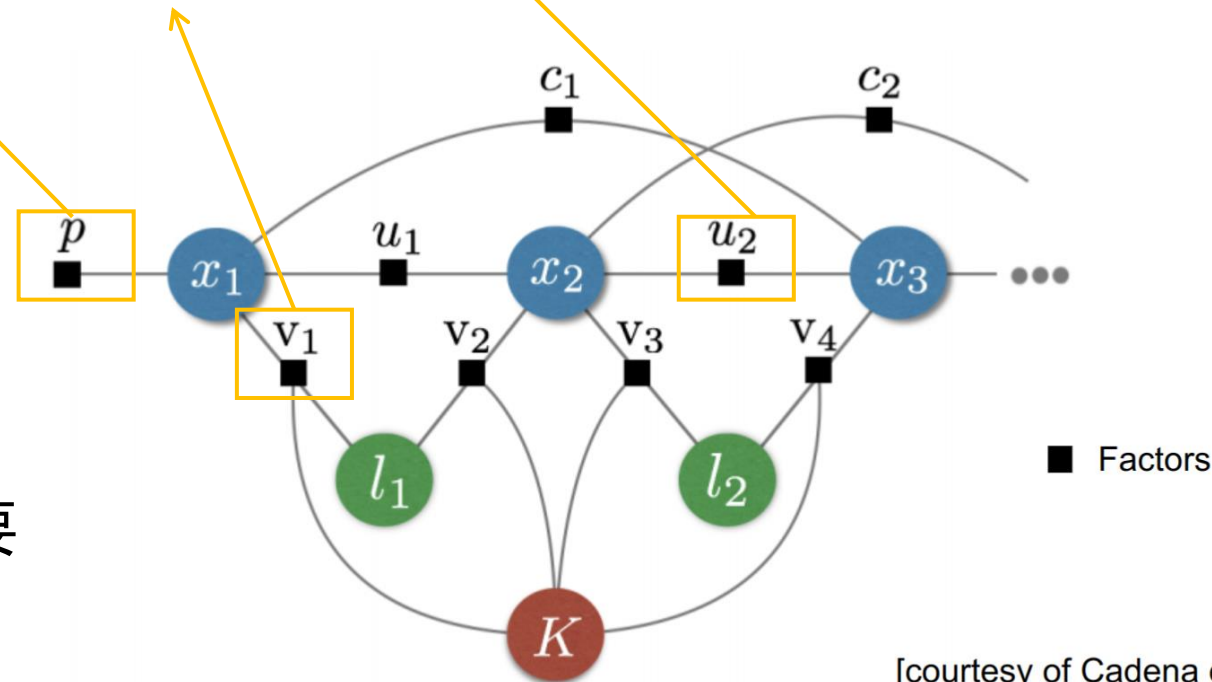
Wisth, David, et.al., "VILENS: Visual, inertial, lidar, and leg odometry for all-terrain legged robots." IEEE Transactions on Robotics 39.1 (2022): 309-326.



## 2. SLAM基礎: ファクタグラフ

- 別なファクタグラフを見してみる
- ファクタの見方、解釈方法の復習
  - $u_2$ :  $x_2$ ,  $x_3$ という最適化変数から定義されるファクタ
  - $v_1$ :  $x_1$ ,  $l_1$ ,  $K$ という最適化変数から定義されるファクタ
  - $p$ :  $x_1$ のみから定義されるファクタ

- 近年のSLAMの動向
  - 多くの手法がファクタグラフを採用
  - ファクタグラフを解釈できることは重要



[courtesy of Cadena et al.]





## 2. SLAM基礎: ファクタグラフ



- ファクタグラフとポーズグラフの違い
  - ポーズグラフは、ファクタグラフの一種
  - 拘束・最適化変数が全てポーズのファクタグラフ = ポーズグラフ
    - つまり、本質的な違いはない



- ファクタグラフは、ポーズグラフと似たような評価関数を使用
  - ポーズグラフと同じように最適化可能
  - **ポーズグラフが理解できれば、ファクタグラフも同様に理解可能**
- ファクタグラフの実装方法
  - GTSAMという最適化ライブラリを使えば容易に実装可能(5章、サンプルプログラム)
    - **GTSAMはファクタグラフ最適化を前提**としているため



## 2. SLAM基礎: まとめ

---

- 座標系、座標変換
  - 自己位置、地図を数式で表現する際の基盤
- ポーズグラフ、ポーズ調整
  - オドメトリ拘束、ループ拘束から評価関数を定義
  - 評価関数は、ガウスニュートン法などの最適化手法で最小化
- ポーズグラフを一般化したファクタグラフ
  - 複雑なSLAMを構築する際に便利
  - 近年のSLAMでは頻繁に使用
- 5章のデモ
  - ポーズグラフによるポーズ調整のデモを行います。
  - オドメトリ拘束・ループ拘束に、GTSAMのコードを通して実際に触れてもらいます。



# 本日本話しする内容

---

1章: SLAMの概要

2章: SLAMの基礎理論

**3章: オドメトリ推定手法**

4章: センサ融合によるSLAMの高性能化

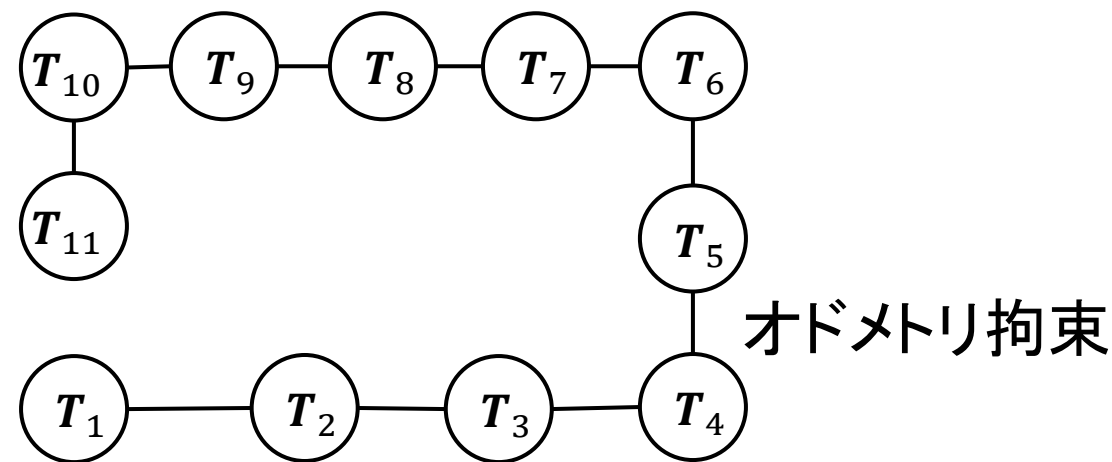
5章: SLAMの実装技術とそのポイント

6章: SLAMの実例・最新研究動向の紹介



### 3. オドメトリ: SLAMにおけるオドメトリ拘束

- ポーズグラフにおいて、オドメトリ拘束としてよく使われる手法
  - エンコーダを用いたオドメトリ(例: ホイールオドメトリ)
  - IMUを用いたオドメトリ
  - カメラを用いたVisual Odometry
  - LiDARを用いたLiDAR Odometry
    - Visual Odometry, LiDAR Odometryの基礎理論はループ拘束にも関連





# 3. オドメトリ: エンコーダによるオドメトリ

## • ホイールオドメトリ

- 車輪型ロボットやクローラ型ロボットでのオドメトリ
- 車輪の回転量からロボットの移動量を計算
  - 車輪の回転量は、エンコーダから計測可能

## • レッグオドメトリ

- 脚型ロボット、ヒューマノイドでのオドメトリ
- 関節の動きからロボットの移動量を計算
  - 関節の動きはエンコーダから計測可能

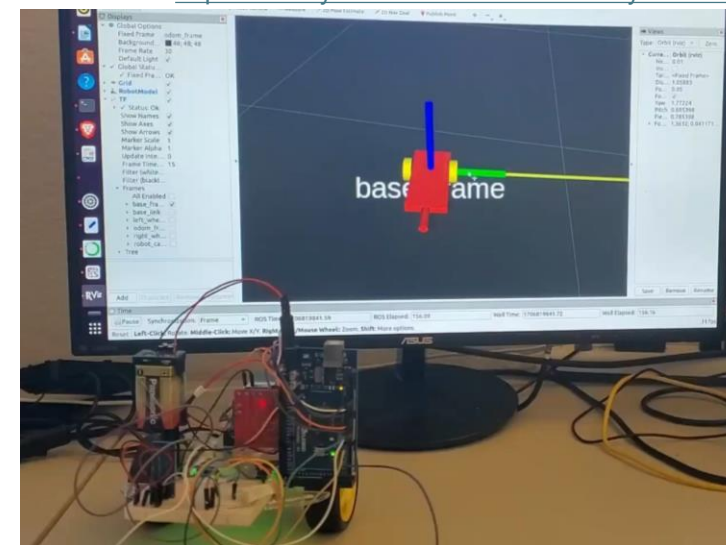
## • 長所・短所

- アルゴリズム単純 & 計算コスト軽い
- カメラやLiDARが苦手な環境でも問題なし
  - 暗闇、煙などでもOK
- 車輪や脚先の滑りの影響を大きく受ける
  - 研究課題になってるので、6章で対処方法を紹介する



©Unitree

<https://www.youtube.com/watch?v=mkycD7tiiFo>



ホイールオドメトリ



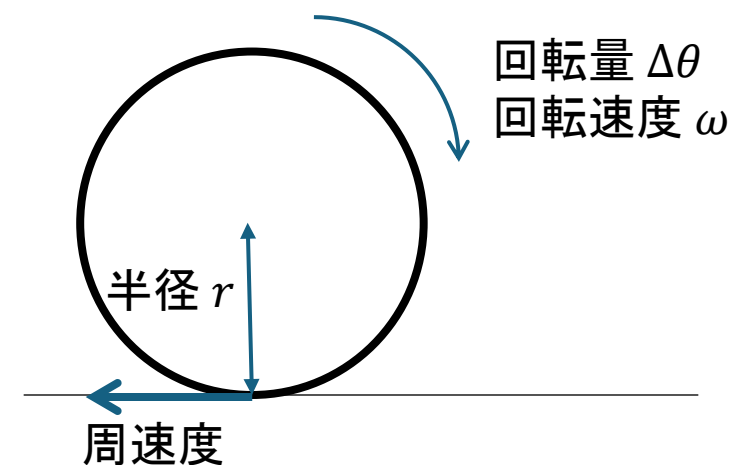


### 3. オドメトリ: ホイールエンコーダによるホイールオドメトリ

- アルゴリズムの根本

- 車輪が回ったら、回った分だけ進む (高校物理の円運動)

- 周速度  $= r\omega$
    - 移動量  $= r\Delta\theta$



- ホイールオドメトリに必要な値

- 車輪速 or 車輪の回転量を計測するホイールエンコーダ
  - ロボットの車輪半径、ホイールベース (車幅)



### 3. オドメトリ: ホイールエンコーダによるホイールオドメトリ

- 差動二輪ロボット

- 車輪が2個、かつ独立に駆動可能な構成
  - 例: ルンバ、タートルボット、電動車いす
- ホイールオドメトリの式
  - ロボット工学の教科書によく掲載されてる



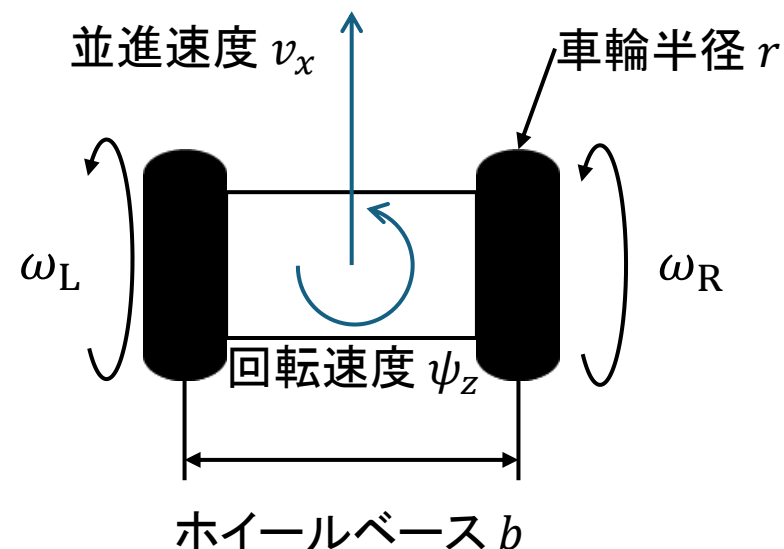
©iRobot



©セリオ

- 差動二輪ロボットの場合の速度

- $v_x = \frac{R(\omega_L + \omega_R)}{2}$
- $\psi_z = \frac{R(\omega_L - \omega_R)}{b}$ 
  - 横方向の移動は発生しないと仮定
  - 2次元平面上の運動を仮定
  - 車輪は滑らないと仮定





### 3. オドメトリ: ホイールエンコーダによるホイールオドメトリ

- ホイールオドメトリの高性能化



説明の前に...

- **高性能化＝改良 → 改良ということは、「理論の欠点を克服すること」**

- 欠点を知ることが重要となるが、その欠点を知るには？

- 1. その手法の理論・考え方をマスターし、不足してる箇所を発見
    - 2. **実際に動かしてみて、微妙な点を発見** → 理論を見直し、**微妙な点**が生じる原因を考察・解明
      - ロボットの実機を使う研究開発だと、2点目が効く可能性が高い
      - 理論が難しい場合も、2点目が効く可能性が高い(自分の卒業研究)
      - もちろん、最もスマートなのは1点目

- 高性能化には、理論の欠点を特定し、それを克服する必要がある

- それでは、ホイールオドメトリの理論では、どのような原因で誤差が発生するのか・・・？

Kojima, Hirohisa, Taku Okawara, and Pavel M. Trivailo. "Ellipse detection-based visual servo control for capturing upper-stage rocket body." Acta Astronautica 182 (2021): 295-309.



### 3. オドメトリ: ホイールエンコーダによるホイールオドメトリ

#### ・ホイールオドメトリで誤差が発生する原因

- ・ エンコーダのノイズ
  - ・ エンコーダのスペックシートから取得可能
    - ・ エンコーダのノイズはどのようにホイールオドメトリに影響？ → 誤差伝播で計算可能
- ・ 車輪径やホイールベースなどの機械的誤差
  - ・ 加工誤差や取り付け誤差などにより、CADでの寸法に誤差はつきもの
    - ・ キャリブレーションで誤差を補正
- ・ 車輪の滑り
  - ・ 車輪は滑らないと仮定したが、実際は滑る
    - ・ 滑り率や滑り角の推定
- ・ 地面の凹凸
  - ・ ホイールオドメトリでは、滑らかな2次元平面上の運動を仮定

研究例を6章で  
解説します



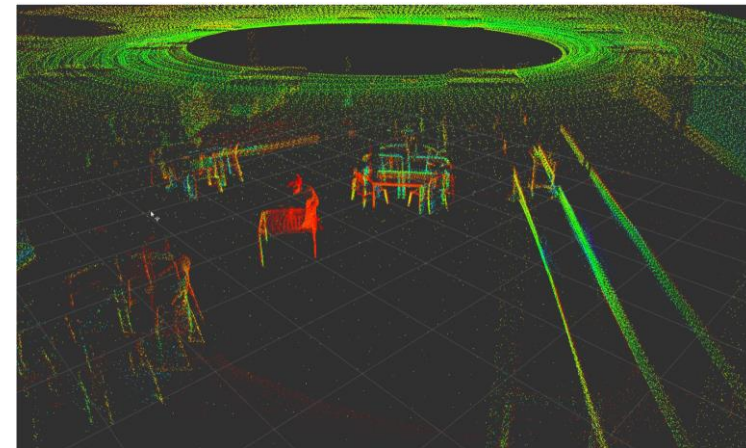
これらを考慮すれば、ホイールオドメトリの高性能化が可能



### 3. オドメトリ: LiDARでSLAMの拘束を定義

- LiDARとは: センサから物体までの3次元ベクトルを計測
- LiDARによるオドメトリ
  - 点群から、ロボットの移動量を推定
  - ある時刻A、別の時刻Bでの点群ペアから $T_{A\_B}$ を推定

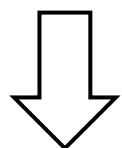
<https://proc-cpuinfo.fixstars.com/2023/01/livox-mid360-ros1-ros2/>



LiDARから計測した3次元点群

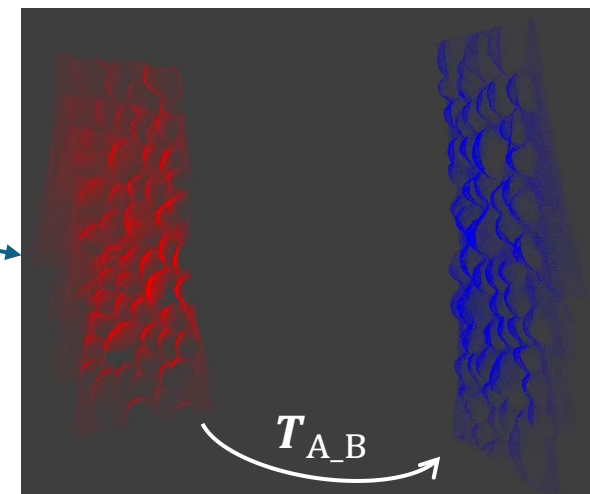
- 移動量 $T_{A\_B}$ を推定する方法
  - スキャンマッチング

- 点群A・点群Bをマッチングし、最適化することで $T_{A\_B}$ を推定する問題
  - 例: ICP



スキャンマッチングは、  
どのようにSLAMと関連するのか

赤い点群→青い点群へのスキャンマッチング  
= 赤い点群と青い点群間の移動量を計算







### 3. オドメトリ: スキャンマッチング

#### • オドメトリ拘束を得る手順

- $T_1$ での点群、 $T_2$ での点群を取得
- スキャンマッチングで $\hat{T}_{1_2}$ を推定
- $\hat{T}_{1_2}$ を観測として、オドメトリ拘束を定義

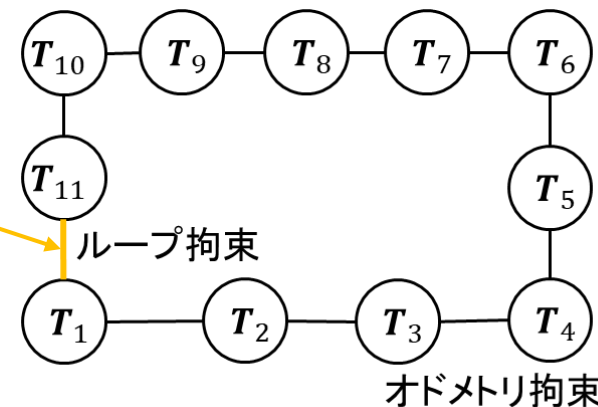
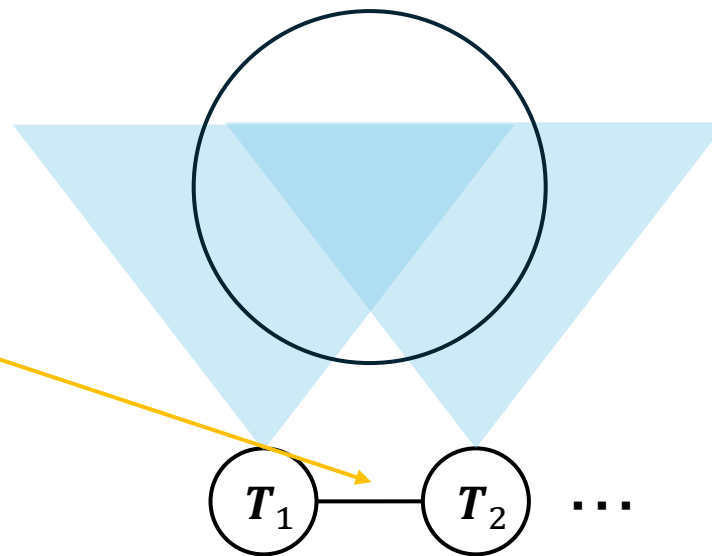
#### • ループ拘束

- ループ拘束も同様
  - $T_1$ 、 $T_{11}$ での点群にオーバーラップ存在
  - スキャンマッチングで $\hat{T}_{1_11}$ を推定
  - $\hat{T}_{1_11}$ を観測として、ループ拘束を定義

#### • スキャンマッチングの役割

- オドメトリ拘束、ループ拘束ともに定義可能
- →スキャンマッチングはLiDAR SLAMの要

点群のオーバーラップ → スキャンマッチング実行可能





### 3. オドメトリ: LiDARによるICP

---

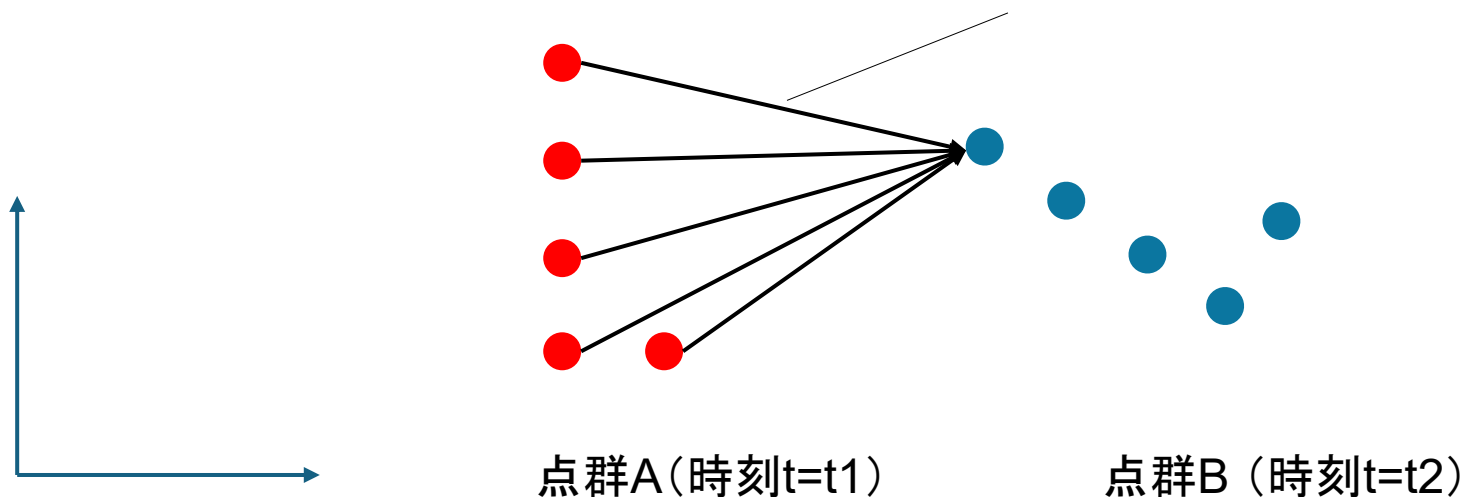
- スキャンマッチングの代表的な手法: ICP
- ICP(**Iterative** Closest Point)の手順まとめ
  1. 点群A、点群Bでマッチング(点群の間での対応点探索)
    - ・ 単にClosest point(最近傍点)をペアとするだけ
  2. マッチングペアから評価関数を定義
  3. 評価関数を最適化し、 $\hat{T}_{A\_B}$ を計算
  4. 評価関数の変化が微小になったら終了, **変化が大きければ1に戻る(iterative)**



### 3. オドメトリ: LiDARによるICP (Iterative **Closest Point**)

- 1. 点群A(時刻 $t_1$ で取得)、点群B(時刻 $t_2$ で取得)でマッチング(対応点探索)
  - 単に**Closest point(最近傍点)**をペアとするだけ
    - 各点において、距離に関する最近傍点を求める
  - シンプルな方法なので、下図のような場合、明らかに誤マッチングとなる

最初のイテレーションでは、誤マッチングの可能性大  
(例: 点群Aの全ての対応点は、点群Bの一番左の点)



### 3. オドメトリ: LiDARによるICP

#### ・2. マッチングペアから評価関数を定義

$$E_{\text{ICP}} = \sum_{i=1}^N \frac{\|d\|^2}{\text{対応点の距離}}$$

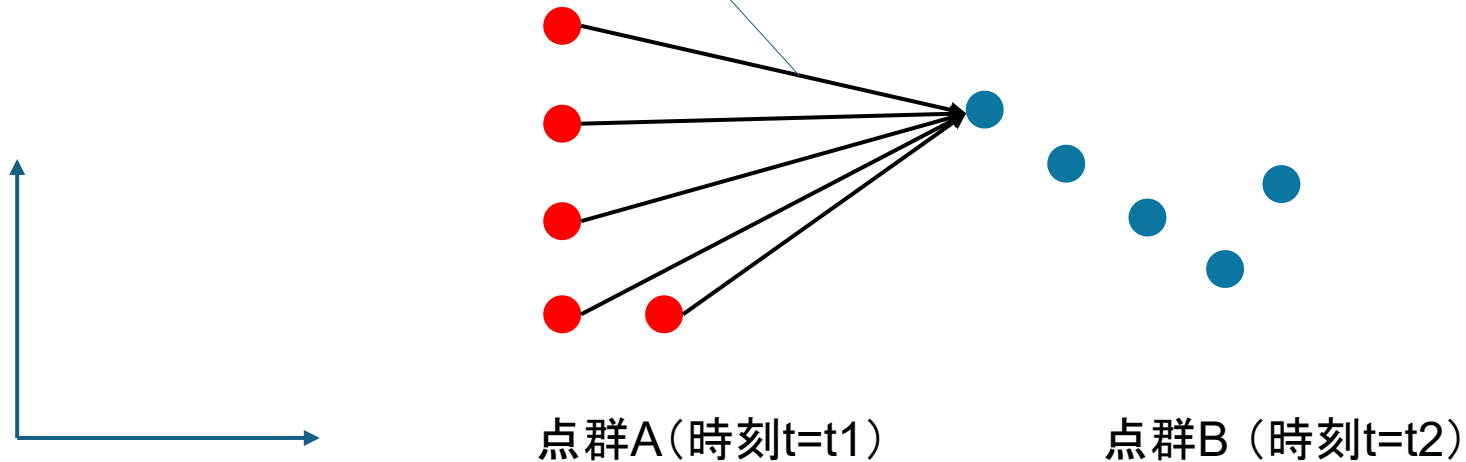
全マッチングペアの和

完全に点群が一致した場合...

→ 対応点の距離  $\|d\|^2$  は0になる

→ ICPでは、 $\|d\|^2$ を小さくするように点群を処理

→ 繰り返し計算で徐々に $\|d\|^2$ を小さくしていく



# 3. オドメトリ: LiDARによるICP

---

## • 2-1. 点の距離に基づく評価関数の式変形

$$E_{\text{ICP}} = \sum_{i=1}^N \|d_i\|^2$$

$\hat{T}_{A\_B}$ が式にないので、  
 $\hat{T}_{A\_B}$ を最適化できない





### 3. オドメトリ: LiDARによるICP

#### • 2-1. 点の距離に基づく評価関数の式変形

$$E_{\text{ICP}} = \sum_{i=1}^N \|d_i\|^2 = \sum_{i=1}^N \left\| \overset{\text{点群Aの座標系}}{\underset{\text{点群Bの}i\text{番目の点の位置ベクトル}}{^A p_{B_i}}} - \overset{\text{点群Aの座標系}}{\underset{\text{点群Aの}i\text{番目の点の位置ベクトル}}{^A p_{A_i}}} \right\|^2$$

点群Aの座標系での点群Bの*i*番目の点の位置ベクトル

点群Aの座標系での点群Aの*i*番目の点の位置ベクトル

基準となる座標系は、点群Aの座標系で統一

しかし、 $^A p_{B_i}$ は直接取得できない

直接取得できるのは、点群Bの座標系基準の点群Bでの $^B p_{B_i}$



$\hat{T}_{A_B}$ によって、 $^B p_{B_i}$ を点群Aの座標へと座標変換する

$$^A p_{B_i} = \hat{T}_{A_B} ^B p_{B_i}$$



# 3. オドメトリ: LiDARによるICP

## • 2-1. 点の距離に基づく評価関数の式変形

$$E_{\text{ICP}} = \sum_{i=1}^N \|d_i\|^2 = \sum_{i=1}^N \|\mathbf{{}^A p_{B_i}} - \mathbf{{}^A p_{A_i}}\|^2 = \sum_{i=1}^N \|\hat{\mathbf{T}}_{A_B} \mathbf{{}^B p_{B_i}} - \mathbf{{}^A p_{A_i}}\|^2$$

点群Bの座標系での点群Bの  $i$  番目の点の位置ベクトル  
= 点群Bの生データ

点群Aの座標系での点群Aの  $i$  番目の点の位置ベクトル  
= 点群Aの生データ

# 3. オドメトリ: LiDARによるICP

## • 2-1. 点の距離に基づく評価関数の式変形

$$E_{\text{ICP}} = \sum_{i=1}^N \|d_i\|^2 = \sum_{i=1}^N \|\mathbf{{}^A p_{B_i}} - \mathbf{{}^A p_{A_i}}\|^2 = \sum_{i=1}^N \|\hat{\mathbf{T}}_{A_B} \mathbf{{}^B p_{B_i}} - \mathbf{{}^A p_{A_i}}\|^2$$

点群Bの座標系での点群Bの  $i$  番目の点の位置ベクトル  
= 点群Bの生データ

点群Aの座標系での点群Aの  $i$  番目の点の位置ベクトル  
= 点群Aの生データ

## 式変形に関する事の発端

$$E_{\text{ICP}} = \sum_{i=1}^N \|d_i\|^2 \quad \leftarrow \text{最適化すべきポーズの移動量 } \hat{\mathbf{T}}_{A_B} \text{ が、評価関数の式の中ないので、最適化できない}$$

# 3. オドメトリ: LiDARによるICP

## • 2-1. 点の距離に基づく評価関数の式変形

$$E_{\text{ICP}} = \sum_{i=1}^N \|d_i\|^2 = \sum_{i=1}^N \|\mathbf{{}^A p_{B_i}} - \mathbf{{}^A p_{A_i}}\|^2 = \sum_{i=1}^N \|\hat{\mathbf{T}}_{A_B} \mathbf{{}^B p_{B_i}} - \mathbf{{}^A p_{A_i}}\|^2$$

点群Bの座標系での点群Bの  $i$  番目の点の位置ベクトル  
= 点群Bの生データ

点群Aの座標系での点群Aの  $i$  番目の点の位置ベクトル  
= 点群Aの生データ

## 式変形に関する事の発端

$$E_{\text{ICP}} = \sum_{i=1}^N \|d_i\|^2 \quad \leftarrow \text{最適化すべきポーズの移動量 } \hat{\mathbf{T}}_{A_B} \text{ が、評価関数の式の中ないので、最適化できない}$$

評価関数の式の中最適化変数があるので、 $\hat{\mathbf{T}}_{A_B}$  に関する最適化が可能



### 3. オドメトリ: LiDARによるICP

- 3. 評価関数を最適化し、 $\hat{T}_{A\_B}$ を計算
  - $E_{ICP}$ は最小二乗の形
  - → **ポーズ調整と同様、ガウスニュートン法などで解ける**
    - 評価関数が最小となる $\hat{T}_{A\_B}$ が解となる
      - **評価関数は、対応点間の距離  $\|d_i\|^2$**

ICPの評価関数: 
$$E_{ICP} = \sum_{i=1}^N \|d_i\|^2 = \sum_{i=1}^N \|\hat{T}_{A\_B}^B p_{B\_i} - p_{A\_j}\|^2$$

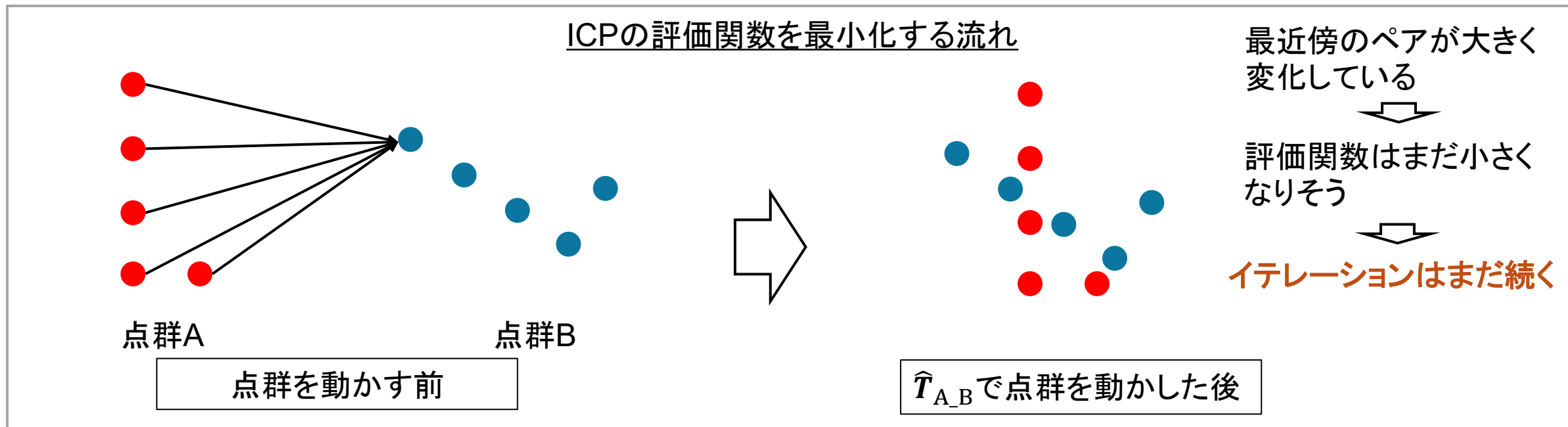
対応点間の距離  $\|d_i\|^2$ に最適化変数  $\hat{T}_{A\_B}$  が含まれるように式変形  
→ **オリジナルなSLAMを作る際に重要な考え方**

→  
 $\hat{T}_{A\_B}$ が計算されたら、  
ICPの処理はどう進む？



### 3. オドメトリ: LiDARによるICP

- $\hat{T}_{A\_B}$  が計算された後の処理
  - $\hat{T}_{A\_B}$  で点群を動かす(下図)



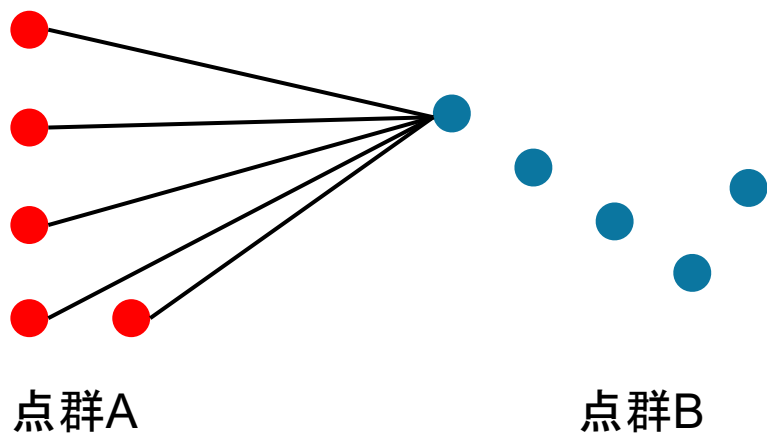
- 4. 評価関数  $E_{ICP}$  の変化が微小であれば終了、変化が大きければ1に戻る
  - 上記のケースだと、まだ続く



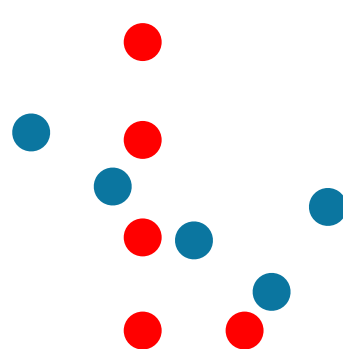
### 3. オドメトリ: LiDARによるICP

- ICP(Iterated Closest Point)の手順まとめ

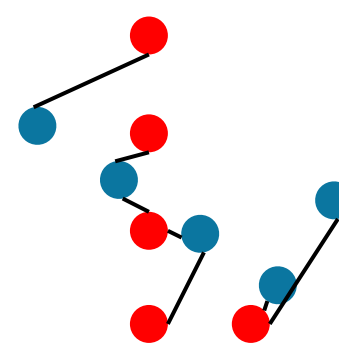
- 点群A、点群Bでマッチング(対応点探索)
  - 単にClosest point(最近傍点)をペアとするだけ
- マッチングペアから評価関数  $E_{ICP}$  を定義
- 評価関数を最適化し、 $\hat{T}_{A\_B}$ を計算
- 評価関数の変化量が微小であれば終了、大きければ1に戻る(Iterative)



1回目の最近傍探索



$\hat{T}_{A\_B}$ で点群を動かした後

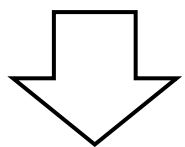
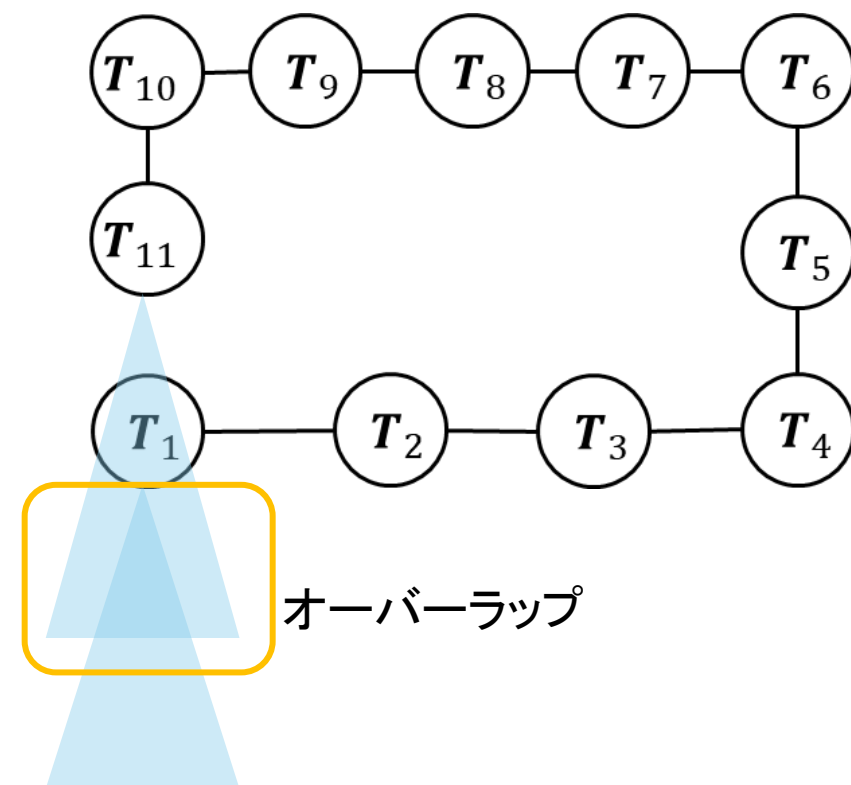


2回目の最近傍探索



### 3. オドメトリ: ICPによるSLAM(ループ拘束)

- ICPによるループ拘束
  - ICPが成り立つ条件:
    - 2種類の異なる点群間でオーバーラップが存在
    - →連続するフレームでは、たしかにありそう
  - $T_1$ と $T_{11}$ 間でループ拘束を作る条件:
    - $T_1$ と $T_{11}$ の点群間でオーバーラップが存在
    - $T_1$ と $T_{11}$ の点群でICPを実行し、 $\hat{T}_{1,11}$ が推定可能



SLAMにおけるICPの役割とは？

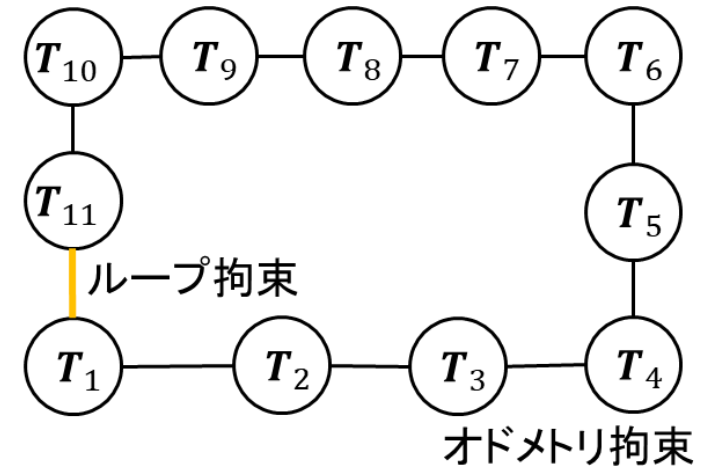


### 3. オドメトリ: ICPとSLAM

- SLAMにおけるICPの役割
  - オドメトリ拘束もループ拘束も作る

↓ ポーズ調整の復習

- オドメトリ、ループによる $\hat{T}_{1_{11}}$ の表現方法の違い
  - オドメトリ:  $\hat{T}_{1_2}, \hat{T}_{2_3}, \dots, \hat{T}_{10_{11}}$ のように、移動量の連鎖で表現 (ICPの誤差が蓄積する)
  - ループ:  $\hat{T}_{1_{11}}$ を直接的に表現可能 (ICPの誤差は蓄積しない)
  - → どちらもICPで構築されるが、 $T_1$ と $T_{11}$ 間の相対ポーズの精度 (主張加減) が違う
- $T_1$ と $T_{11}$ 間の相対ポーズについて、ポーズ調整の時に意識したいイメージ
  - オドメトリでも、ループでも、 $T_1$ と $T_{11}$ 間の相対ポーズを表現可能
  - オドメトリだけでは $\hat{T}_{1_{11}}$ の精度がイマイチである一方で、ループでは高精度で表現可能
    - 誤差の蓄積量の違いが原因
  - ポーズ調整では、ループ拘束による観測の主張が優先されるように最適化





# 3. オドメトリ: LiDARによるICP まとめ

## • ICP (= LiDAR SLAM) の高性能化

### • 精度面

#### • 評価関数の改良

- 今回使用したのは、点と点の距離 (ICP)
- → 平面の拘束を考慮する手法
  - NDT
  - GICP
  - VGICP
- → 点群の色も導入
  - Color ICP

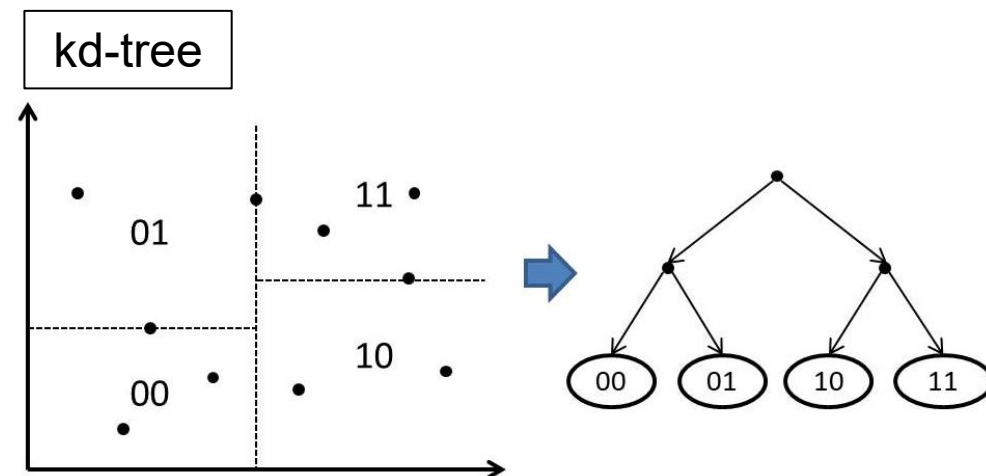
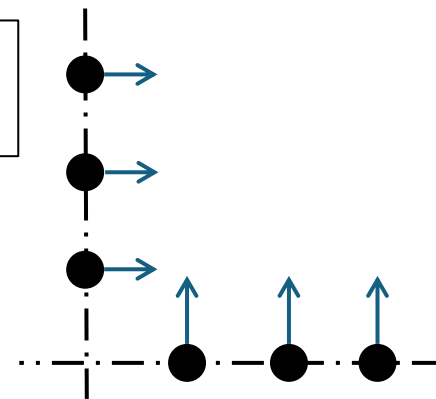
### • 頑強性

- マッチングでの外れ値除去
  - 距離  $\|d_i\|^2$  が大きすぎるマッチングは除去

### • 高速性

- kd-tree や ikd-tree による最近傍探索の高速化
- GPU実装
  - 点群の数は膨大なので、GPUとの相性が良い

各点の法線情報を活用  
→ 平面による拘束を構築



<http://www.sanko-shoko.net/note.php?id=75jb>



### 3. オドメトリ: LiDARによるICPの活用・実装

- 我々の研究グループで開発されたオープンソースのICP

- gtsam\_points

- ICP
- GICP
- VGICP
- ...

- 使用方法の例(右の動画)

- demo\_matching\_cost\_factorsを実行
- LiDARを持ってなくても即テスト可能

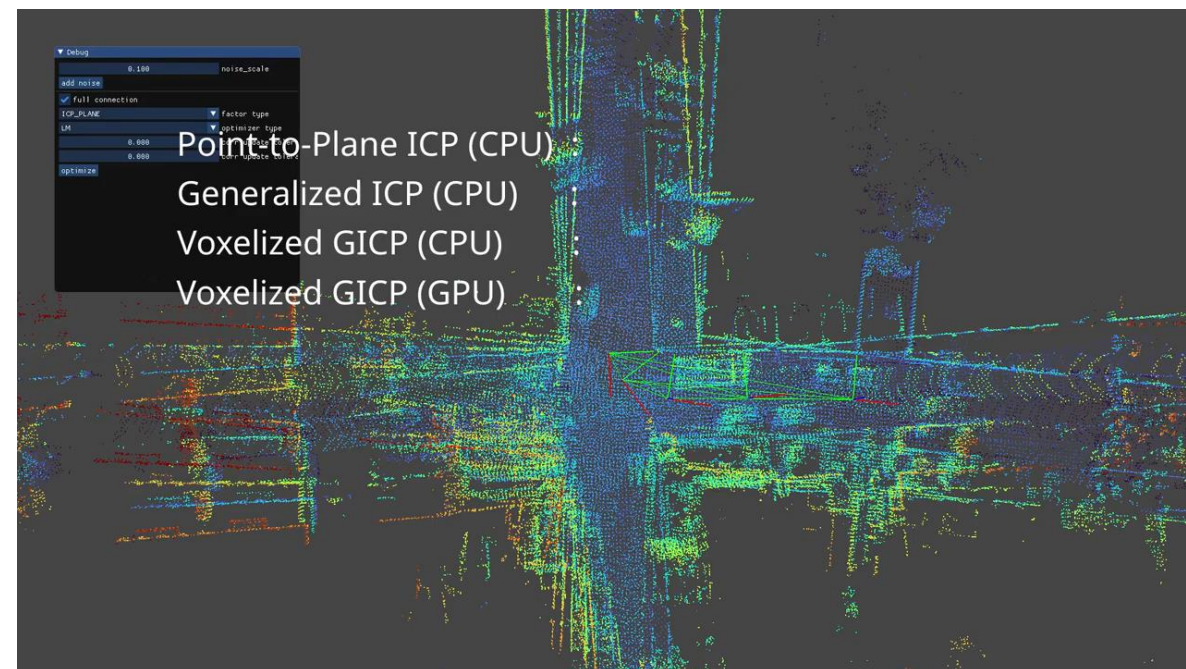
- コンパイルなしでも使用可能

- 使用手順マニュアル

- ICPをSLAMの拘束として実装可能

- demo\_matching\_cost\_factorsを参考にすれば、ICPによる拘束を簡単に実装可能

[https://www.youtube.com/watch?v=HCXCWlx\\_VOM](https://www.youtube.com/watch?v=HCXCWlx_VOM)



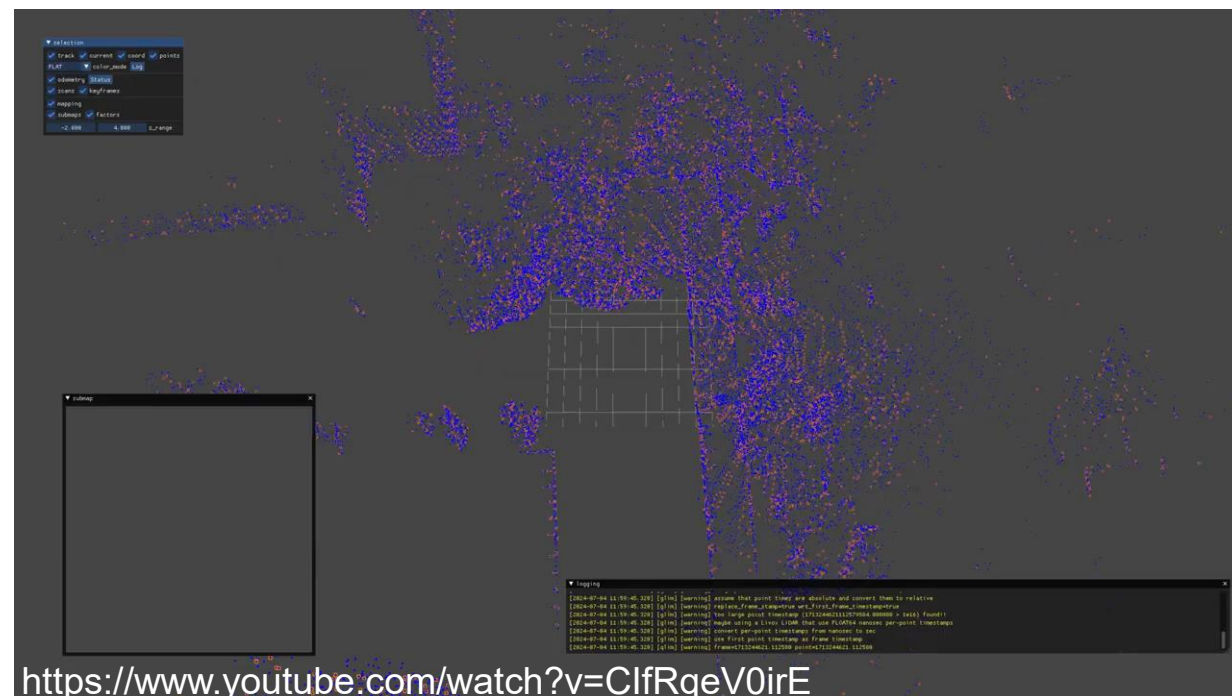
gtsam\_pointsによる様々なスキャンマッチングの実行





### 3. オドメトリ: ICPを用いたLiDAR SLAM

- GLIM (ROS1/ROS2対応)
  - gtsam\_pointsによるICPを拘束とした我々の研究グループが開発したオープンソースのLiDAR-IMU SLAM
  - IMUがついたLiDAR (Livox MID-360やLivox AVIAなど)で簡単に実行可能
    - コンパイルなしでも使用可能
    - 使用手順マニュアル
- Livox MID-360でのGLIM実行例



Koide, Kenji, et al. "GLIM: 3D range-inertial localization and mapping with GPU-accelerated scan matching factors." Robotics and Autonomous Systems (2024): 104750.



### 3. オドメトリ: カメラによるSLAMの拘束

- カメラの種類

- 単眼カメラ

- いわゆる普通のカメラ。スマホのような画像を取得
    - → 2次元画像を取得

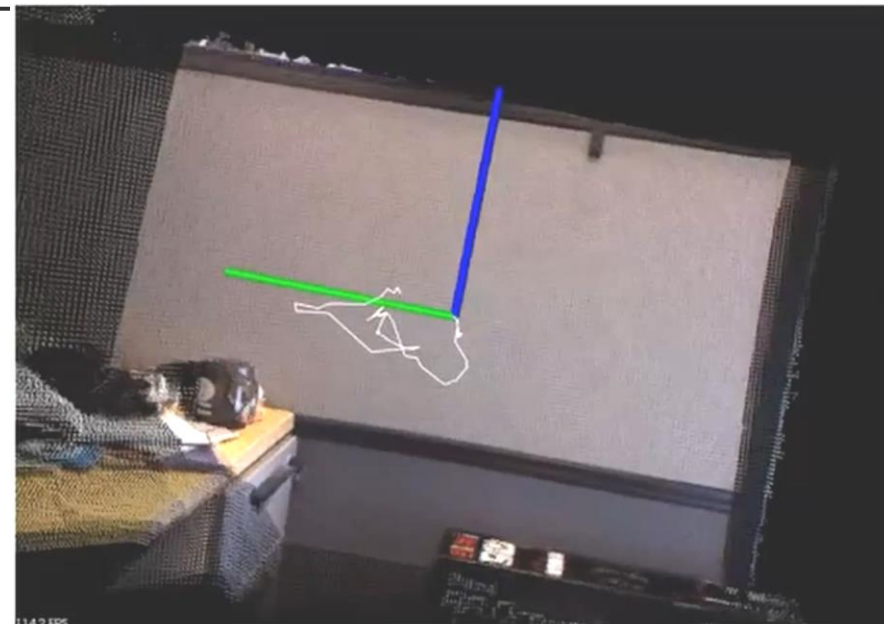
- ステレオカメラ

- 単眼カメラを2個備えたカメラ
    - 2個のカメラ画像を処理することで、3次元情報を取得

- デプスカメラ

- 3次元点群を直接取得
    - **色付き点群**が取得可能なRGB-Dカメラ

- 以降の説明では、RGB-Dカメラの使用を前提とする



<https://www.youtube.com/watch?v=Nm2ggyAW4rw>



Realsense D435 ©Intel



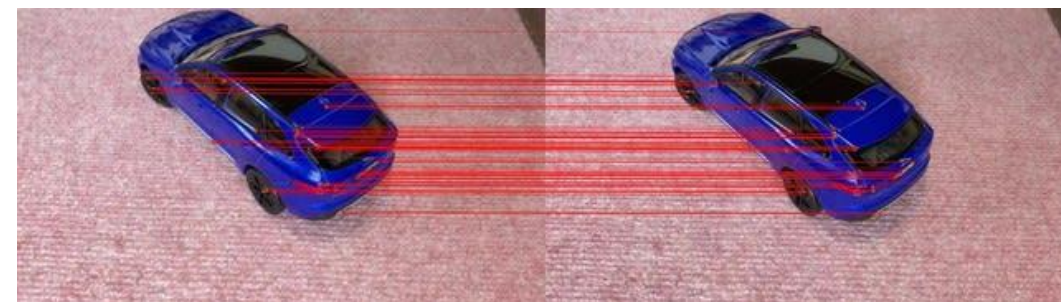
### 3. オドメトリ: カメラによるSLAMの拘束

- カメラによるSLAMの拘束の作り方の概要
  - 再投影誤差の最小化(バンドル調整)が肝
  - 画像のマッチング・カメラ幾何に基づき、移動量を推定
    - SLAMにおける立ち位置は、LiDARのICPとほとんど同じ
    - オドメトリ拘束・ループ拘束を構築可能



#### 再投影誤差の最小化の手順

- 画像Aと画像Bの特徴マッチング
  - 画像Aと画像Bから、それぞれ特徴点を抽出
  - 特徴点に対して特徴量記述子を計算
  - 特徴量記述子でマッチングし、対応点を取得
- 対応点から評価関数(再投影誤差)を定義
- 評価関数を最適化し、移動量 $\hat{T}_{A\_B}$ を推定



特徴マッチングの例

<https://whoopsidaies.hatenablog.com/entry/2013/12/07/135810>  
<https://blog.goo.ne.jp/yoossh/e/3ba797002ce9fbd08fc9e23fa9356ae5>



### 3. オドメトリ: カメラによる再投影誤差の最小化

- 以降では、「再投影誤差の最小化」の理論を少し丁寧に説明
  - 再投影誤差(評価関数)の最小化によって、カメラ画像からロボットの移動量を計算可能
  - = SLAMでは必須の機能
- 再投影誤差の最小化の理論を学ぶモチベーション
  - 評価関数である再投影誤差の考え方の習得
    - 考え方:「理論式」と「観測値」の差分を最小化するように最適化変数を求める
    - SLAMの理論において、重要な考え方を分かりやすく表現した形になっている
      - 個人的な意見です。
  - この考え方を丁寧に伝えるために、再投影誤差を少し丁寧に説明
    - まずは、カメラ座標系と画像座標系の間の「カメラ幾何」の概要から説明



再投影誤差の前に、  
カメラ幾何の基礎から説明



### 3. オドメトリ: カメラによる再投影誤差の最小化

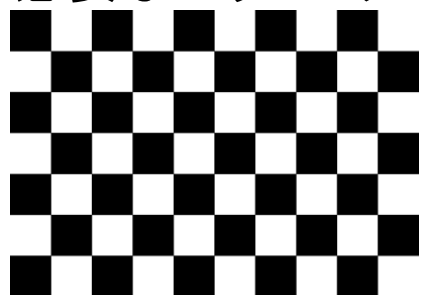
- カメラの幾何の基本

- カメラ画像上の情報は、**3次元空間上の点が2次元画像平面上に投影**されたもの
  - 3次元空間上の点: **カメラ座標系**を基準
  - 2次元画像平面上的点: **画像座標系**を基準
- カメラ幾何は、visual SLAMの基盤

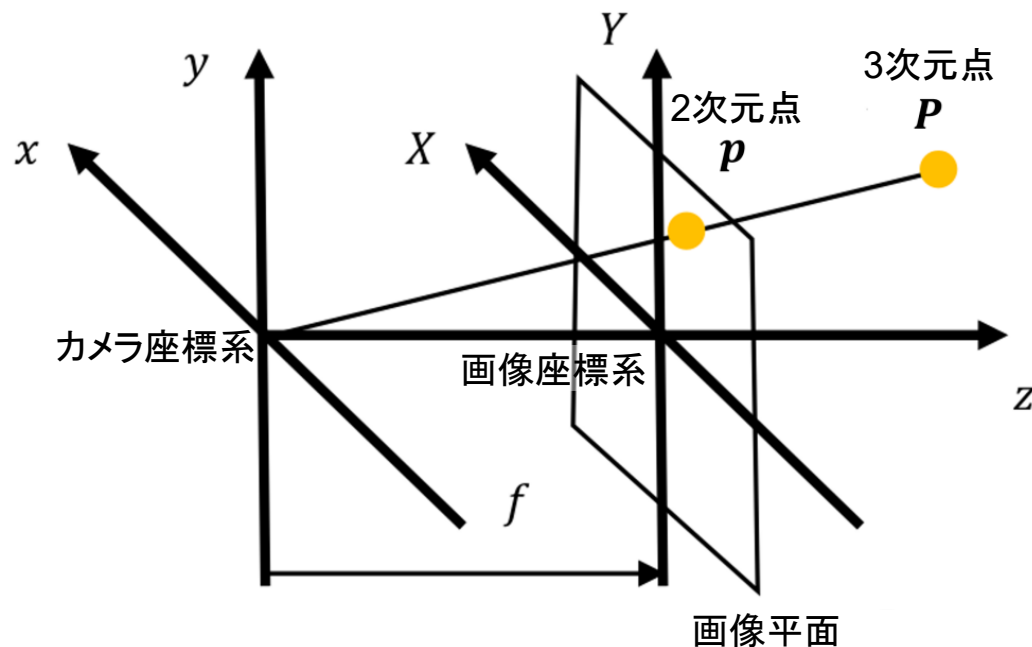
- カメラ座標と画像座標の関係の記述方法

- カメラ座標系・画像座標系の座標変換**

- 必要なパラメータは、キャリブレーションで取得



キャリブレーションボード  
を使って実施可能

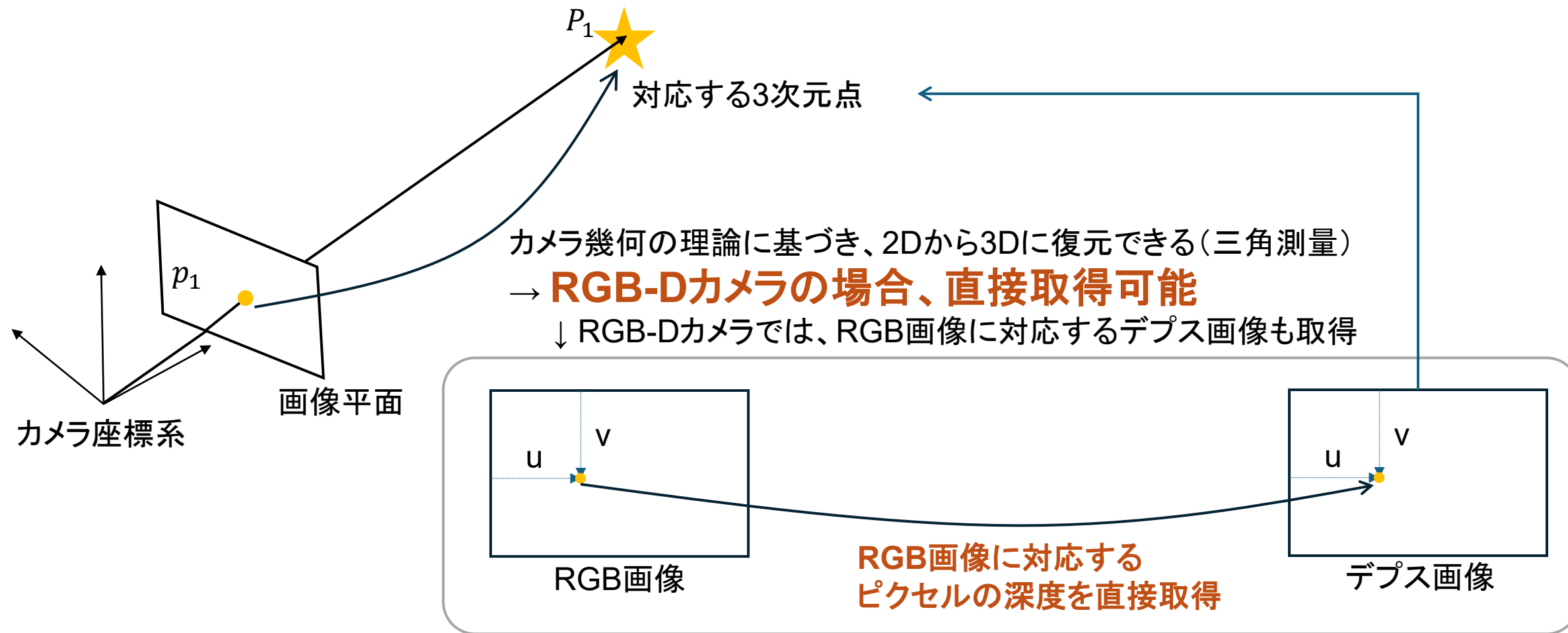






### 3. オドメトリ: カメラによる再投影誤差の最小化

- 基礎: カメラ画像上の2次元点から、対応する3次元点を取得するには？





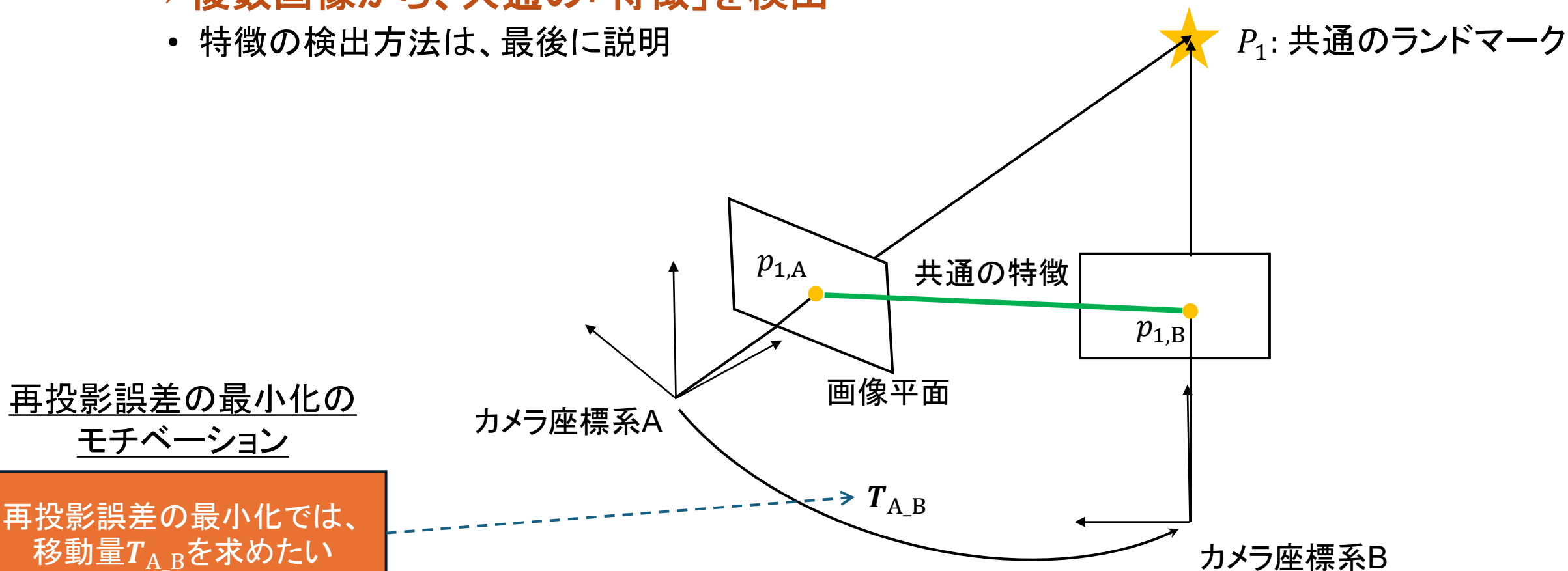


### 3. オドメトリ: カメラによる再投影誤差の最小化

- 手順 1: 複数のカメラ画像から共通のランドマークを検出

- 複数画像から、共通の「特徴」を検出

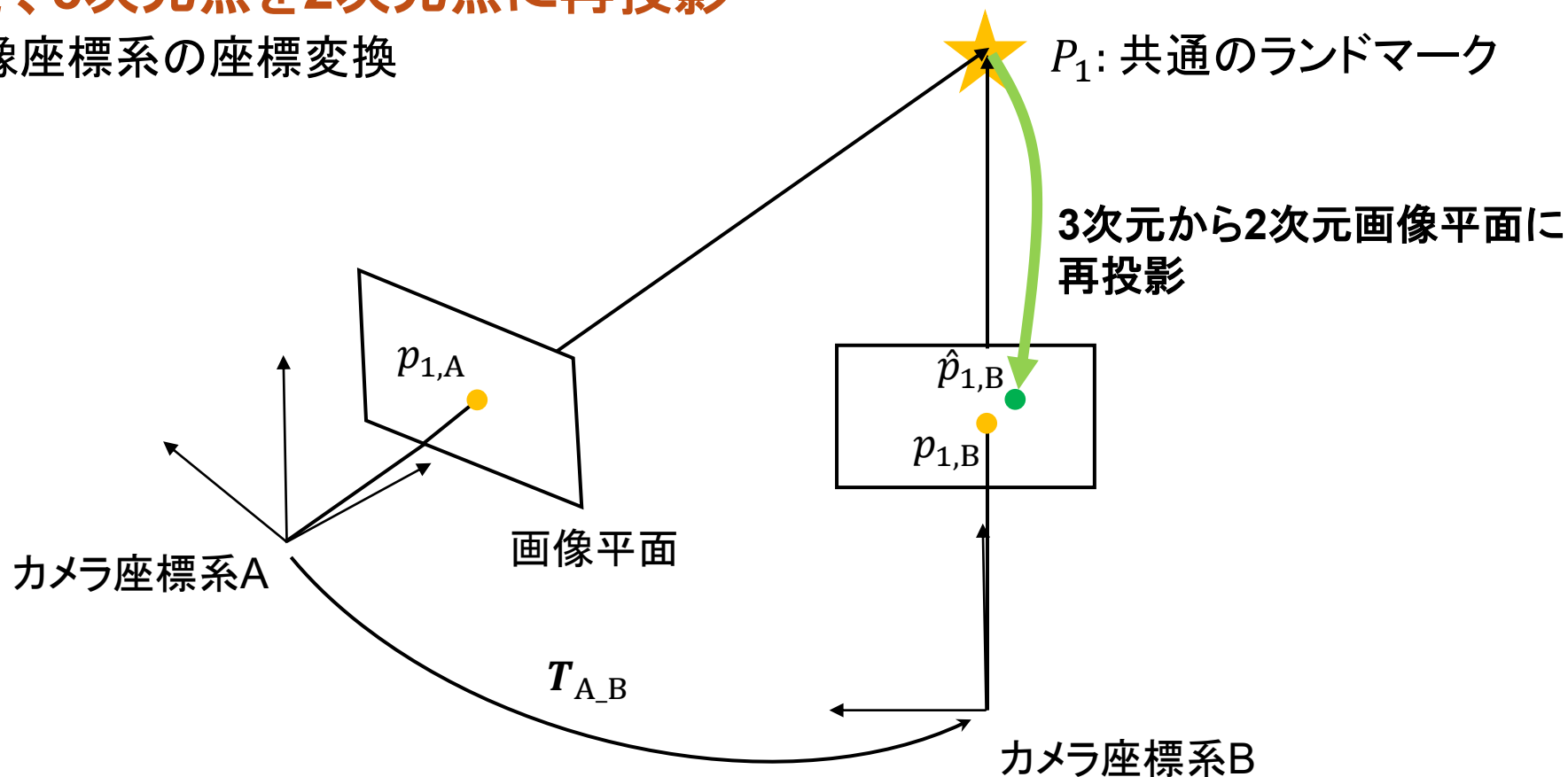
- 特徴の検出方法は、最後に説明





### 3. オドメトリ: カメラによる再投影誤差の最小化

- 手順 2: 3次元上の点を2次元画像に再投影
  - カメラ幾何に基づき、3次元点を2次元点に再投影  
= カメラ座標系と画像座標系の座標変換





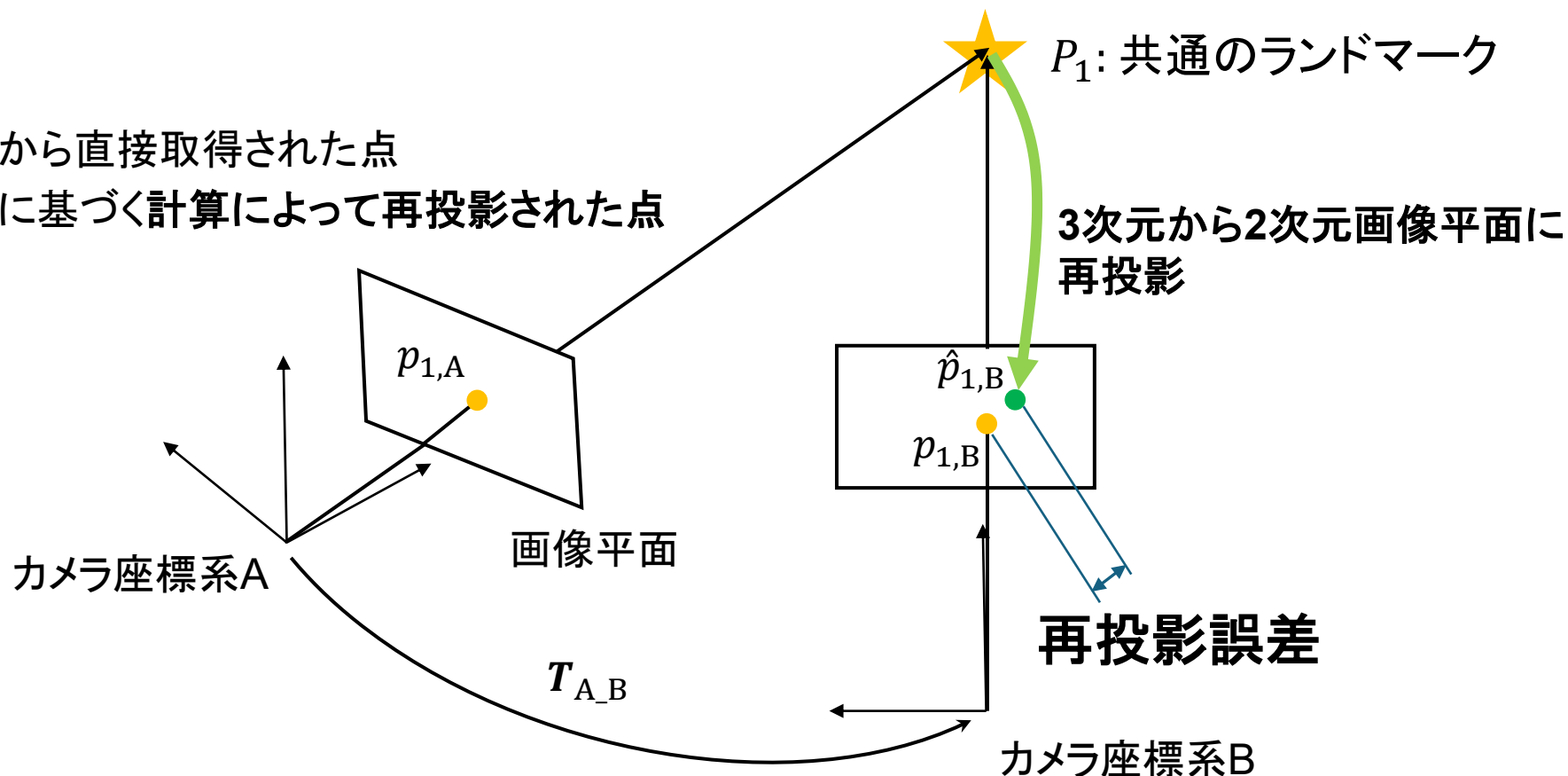
### 3. オドメトリ: カメラによる再投影誤差の最小化

- 手順 3: 移動量  $T_{A\_B}$  の最適化の評価関数「再投影誤差」を定義

- 再投影誤差

- $\|\hat{p}_{1,B} - p_{1,B}\|^2$

- $p_{1,B}$ : カメラ画像から直接取得された点
- $\hat{p}_{1,B}$ : カメラ幾何に基づく計算によって再投影された点

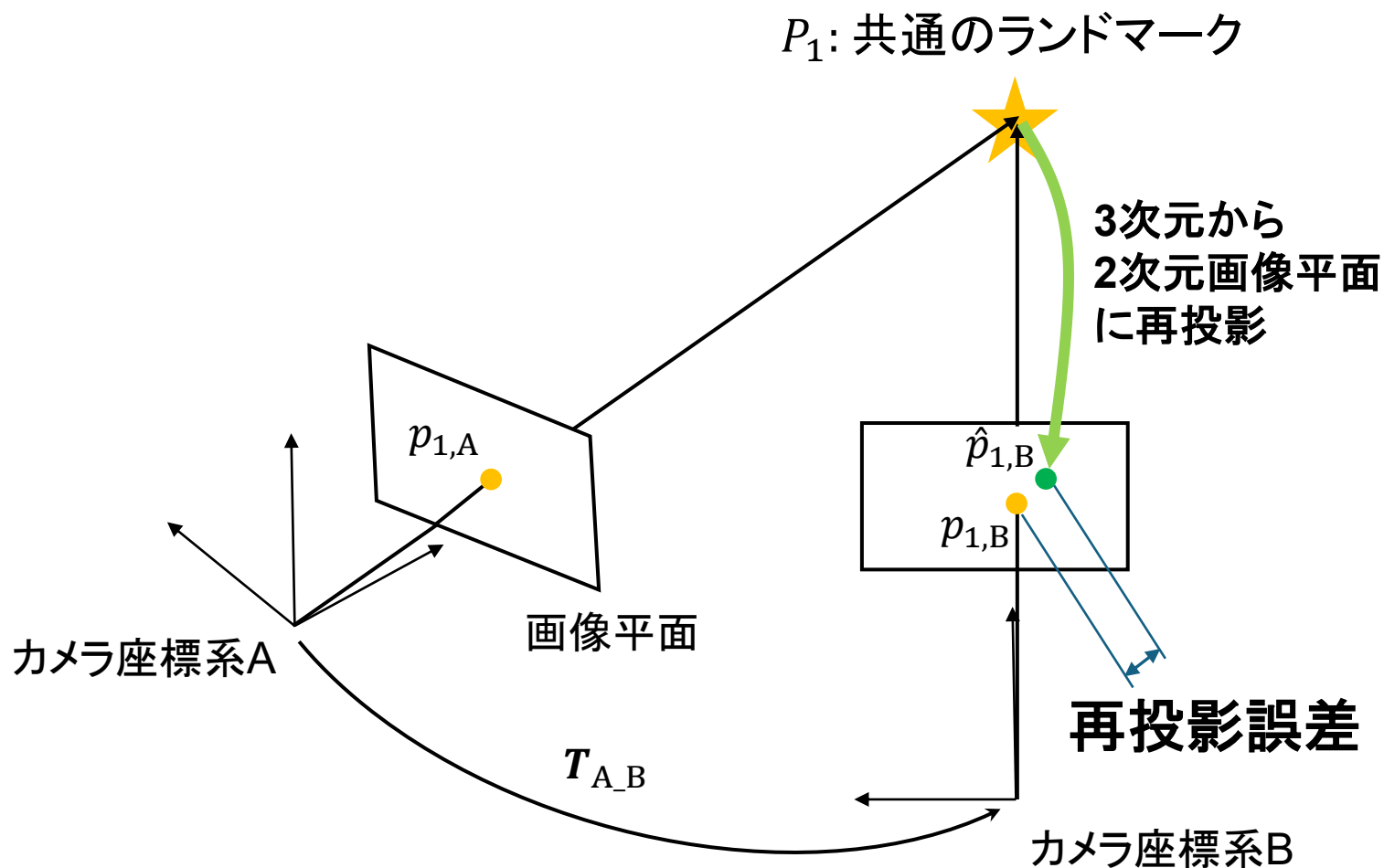




### 3. オドメトリ: カメラによる再投影誤差の最小化

#### • 手順 4: 再投影誤差の最小化

- $\|\hat{p}_{1,B} - p_{1,B}\|^2$ 
  - $\hat{T}_{A\_B}$  が式に含まれていない
  - $\rightarrow \hat{T}_{A\_B}$  の推定ができない
- $\rightarrow \|\hat{p}_{1,B}(\hat{T}_{A\_B}) - p_{1,B}\|^2$ 
  - $\hat{p}_{1,B}$  を  $\hat{T}_{A\_B}$  の関数として変形
    - カメラ幾何 (透視投影変換) に基づく式変形で導出可能
  - $\rightarrow \hat{T}_{A\_B}$  の最適化が可能





### 3. オドメトリ: カメラによる再投影誤差の最小化

- 再投影誤差の最小化 まとめ

- 再投影点  $\hat{p}_{1,B}$  は  $\hat{T}_{A\_B}$  の関数
- $\hat{T}_{A\_B}$  を変化させると  $\hat{p}_{1,B}$  も変化
- **再投影誤差が小さくなるように  $\hat{T}_{A\_B}$  を調整**

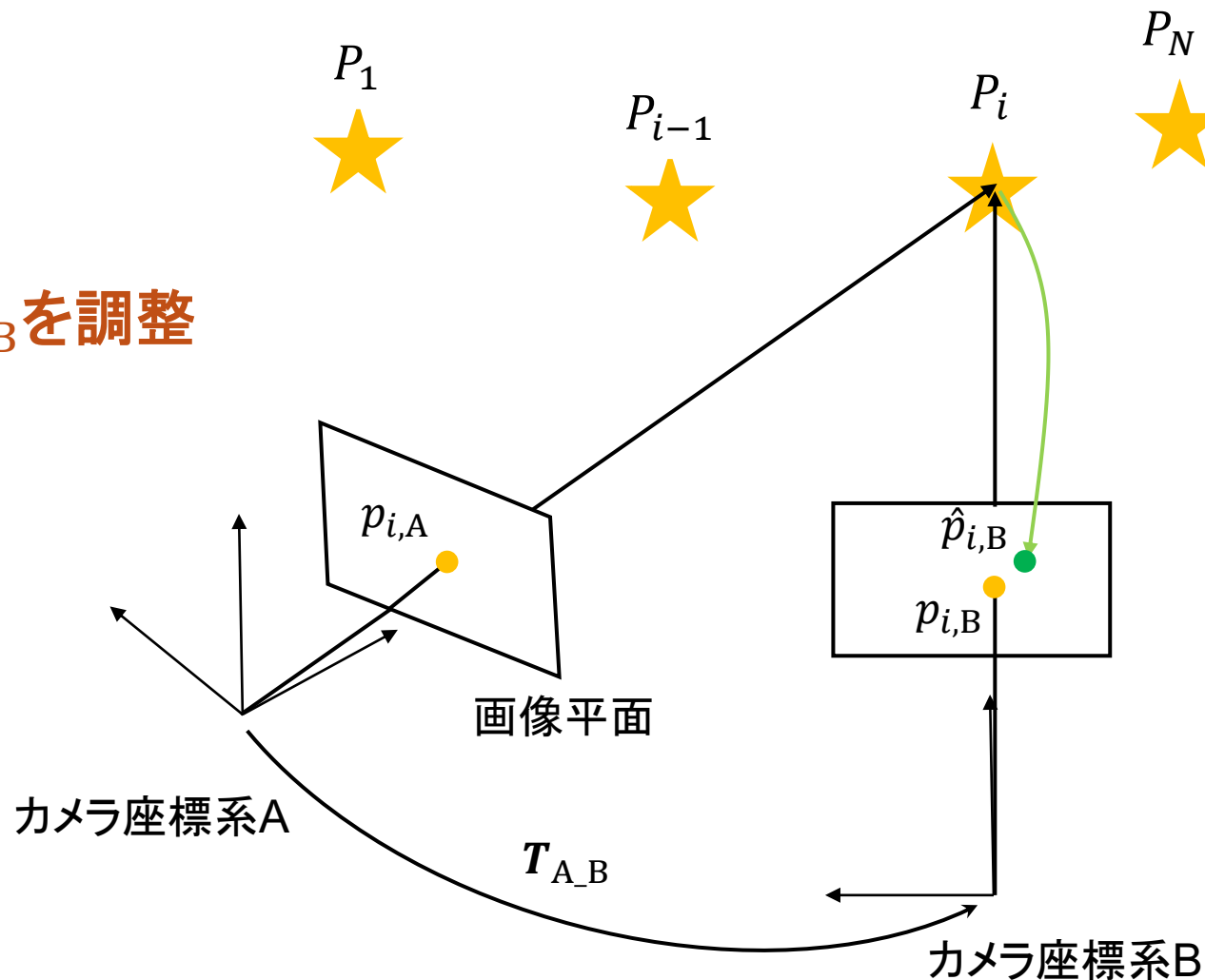
実際に得られた観測との辻褄が合うように

- 最終的な評価関数

$$E = \sum_{i=1}^N \|\hat{p}_{i,B}(\hat{T}_{A\_B}) - p_{i,B}\|^2$$

**$N$ 個の共通のランドマークの再投影誤差の総和**

- $E$  を最小とする  $\hat{T}_{A\_B}$  を推定



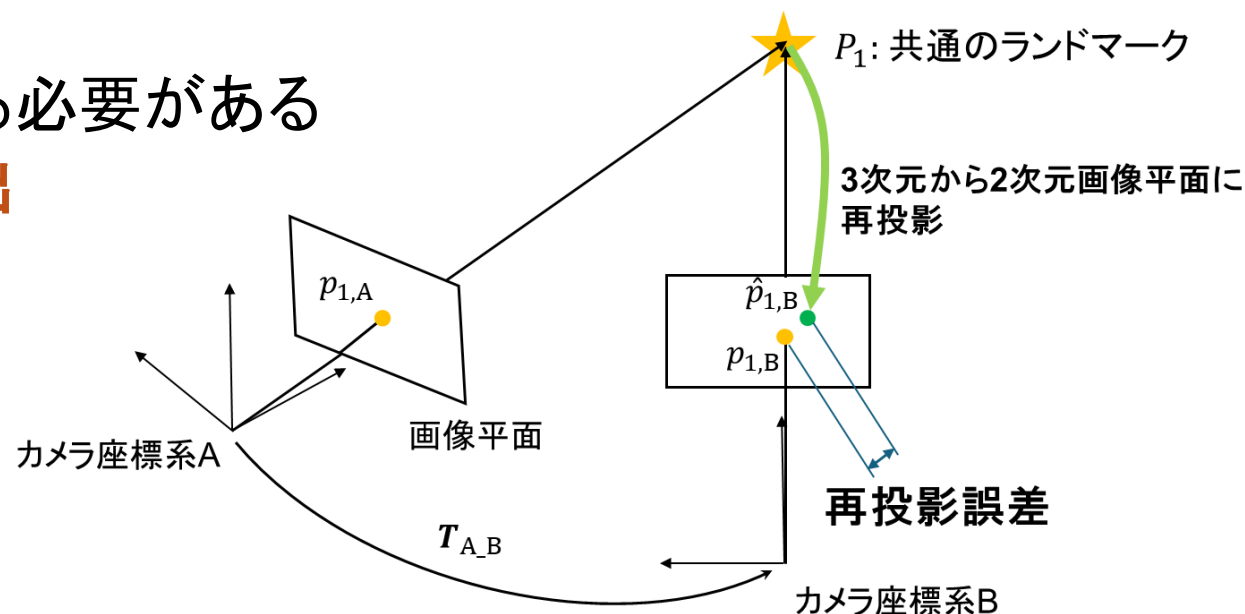


### 3. オドメトリ: カメラによる再投影誤差の最小化

- 再投影誤差  $E$  を最小とする  $\hat{T}_{A\_B}$  を推定するには
  - ガウスニュートン法などの最適化手法で解ける
  - 非線形最小二乗問題の形なので

$$E = \sum_{i=1}^N \|\hat{p}_{i,B} - p_{i,B}\|^2$$

- 再投影誤差を計算するには？
  - 画像間で共通のランドマークを見つける必要がある
  - **画像Aと画像B内で、同一な点を検出**
    - 特徴マッチングという技術**
      - 特徴点検出
      - 特徴量記述
      - 画像間の特徴のマッチング
  - これができる初めて最適化が行える







# 3. オドメトリ: カメラによる再投影誤差の最小化

## • 1. 特徴点検出: 画像から特徴を抽出

### • 特徴とは？

- 画像において、安定で継続的にトラッキング可能な領域
- →カメラを動かしたり、照明条件を変えても、定常的に画像に写る目印のような領域
- →**具体的には画像上のエッジやコーナー(特にコーナー)**
  - つまり、画像上の輝度値の勾配が大きい箇所

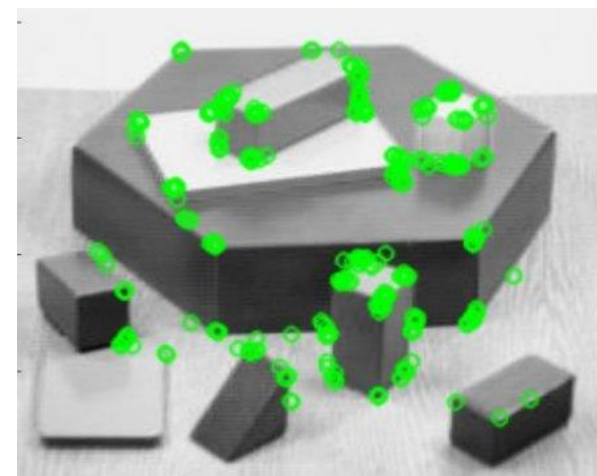
### • 特徴点検出の手法

- GFTT
- ORB
- SIFT
- その他多数

右図のように**特徴点の2D位置を出力**

### • 実装方法

- OpenCVという画像処理ライブラリに画像を入力するだけ
- 簡単に様々な手法を実装可能



検出された特徴点の例

[https://labs.eecs.tottori-u.ac.jp/sd/Member/oyamada/OpenCV/html/py\\_tutorials/py\\_feature2d/py\\_orb/py\\_orb.html](https://labs.eecs.tottori-u.ac.jp/sd/Member/oyamada/OpenCV/html/py_tutorials/py_feature2d/py_orb/py_orb.html)



### 3. オドメトリ: カメラによる再投影誤差の最小化

---

- 2. 特徴量記述子: 特徴点に対して**特徴量を計算**
  - 特徴点検出では、特徴点の位置だけ計算する処理
  - **特徴量記述子は、特徴点周辺を特徴ベクトル or バイナリコードとして記述する処理**
    - 特徴点近傍の画素値の輝度勾配などから特徴ベクトル or バイナリコードを定義できる
      - **重要なのは、カメラ視点が多少変化しても同じ値となるようなアルゴリズム**
    - 様々なアルゴリズムが提案されている
      - SIFT: 128次元の実数の特徴ベクトル
      - SURF: 64次元の実数の特徴ベクトル
      - BRIEF: 数十～数百の0 or 1のバイナリコード
      - その他多数
- 実装方法
  - OpenCVという画像処理ライブラリに特徴点と画像を入力するだけ
  - 簡単に様々な手法を実装可能



### 3. オドメトリ: カメラによる再投影誤差の最小化

- 3. 特徴マッチング: 特徴量記述子に基づきマッチングし、対応点を取得

- 特徴量は、多少視点が変化しても不変であった

- → **各特徴ペアにおいて、ある一致度に基づき特徴マッチング**

- 一致度には何が使われる?

- 特徴ベクトル → 実数ベクトル間の距離

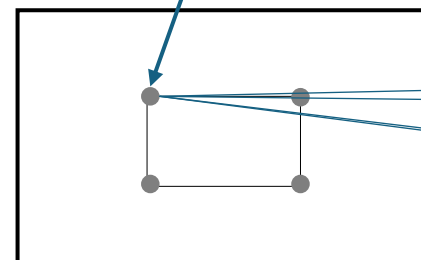
- バイナリコード → ハミング距離

- マッチングの手順(例: 全探索)

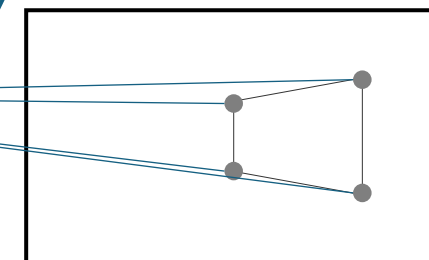
1. 画像Aから特徴を一つ選択
2. 上記の特徴と画像Bの全特徴の間で一致度を計算
3. 最も一致度が高いペアを対応点とする
4. 1~3を全ての特徴に適用
5. 終了

1点選択

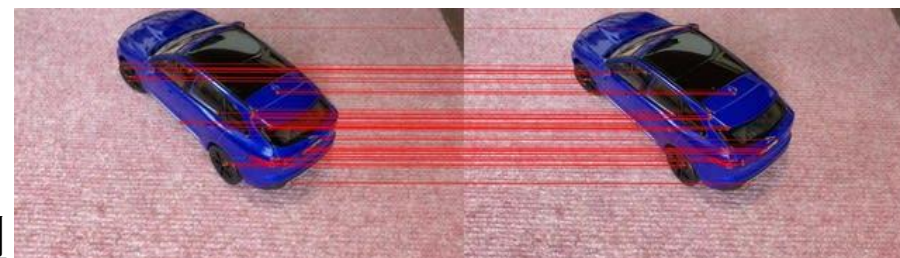
全ての対応点を考慮  
→ 一致度が最高のペアを決定



画像A



画像B



特徴マッチング結果の例

<https://blog.goo.ne.jp/yoossh/e/3ba797002ce9fbd08fc9e23fa9356ae5>



### 3. オドメトリ: カメラによる再投影誤差の最小化 まとめ

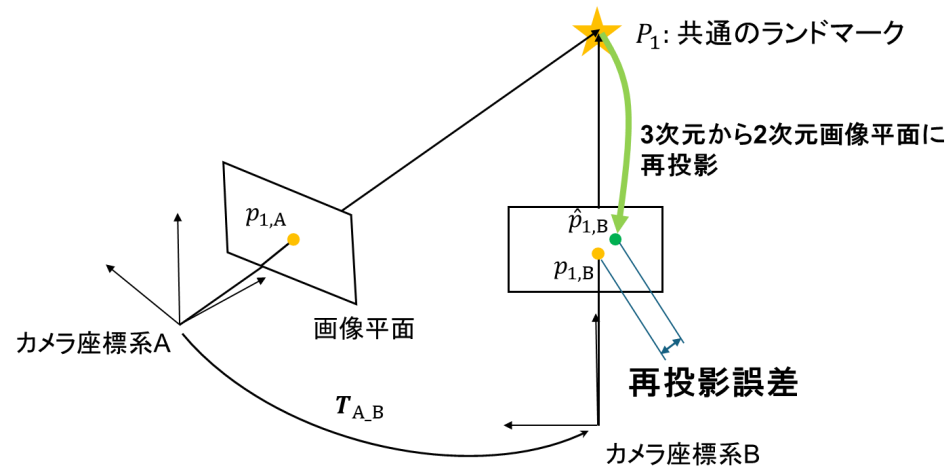
#### • 再投影誤差の最小化の手順

##### 1. 画像Aと画像Bの特徴マッチング

- 画像Aと画像Bから、それぞれ特徴点を抽出
- 特徴点に対して特徴量記述子を計算
- 特徴量記述子でマッチングし、対応点を取得

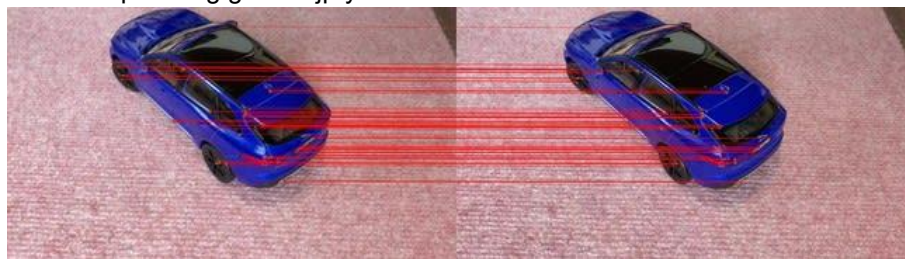
##### 2. 対応点から評価関数(再投影誤差)を定義

##### 3. 評価関数を最小化するような移動量 $\hat{T}_{A\_B}$ を推定



再投影誤差の概念図

<https://blog.goo.ne.jp/yoossh/e/3ba797002ce9fbd08fc9e23fa9356ae5>



画像A・画像B間の特徴マッチングの例

再投影誤差最小化  
の評価関数

$$E = \sum_{i=1}^N \|\hat{p}_{i,B} - p_{i,B}\|^2$$



### 3. オドメトリ: カメラによる再投影誤差の最小化 まとめ

- SLAMを改良するにあたり、再投影誤差の表現で意識すること

- 再投影誤差:  $\|\hat{p}_{1,B}(\hat{T}_{A_B}) - p_{1,B}\|^2$

- $p_{1,B}$ : カメラ画像から直接取得された点 → センサの観測値
  - $\hat{p}_{1,B}(\hat{T}_{A_B})$ : カメラ幾何に基づき再投影された点 → 観測関数

最適化変数(ロボットポーズなど)に基づく、センサデータの予測モデル

- SLAMでよく使われる評価関数

- 「観測関数」と「センサの観測値」の差分
- → この差を小さくするように、最適化
- → センサデータとの整合性がとれるような最適化変数を推定
  - SLAMを拡張するような新しい拘束を作るときに必要な考え方
  - → SLAMの論文の式展開が理解できるようになる(個人的意見です)

長々と説明したが、  
一番伝えたいのは、この考え。  
バンドル調整は単なる例。



# 3. オドメトリ: カメラによるSLAM まとめ

- Visual SLAMの高性能化の一例

- 特徴マッチングの外れ値除去

- エピポーラ幾何など

- 特徴マッチングが不適切だと、バンドル調整 (Visual SLAM) は破綻

- 機械学習との組み合わせ

- 3次元地図上の物体に意味をもたせたsemantic mapping

- 学習ベースの特徴マッチング

- カメラの狭い視野の改善

- 視野が狭いことのデメリット: 画像内に有益な特徴の数が不十分になる可能性が高くなる

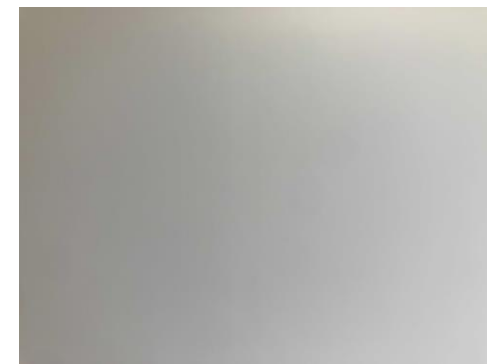
- 複数カメラの使用

- 広い視野を持つ「全方位カメラ」や「魚眼カメラ」の使用

- Active SLAMの導入

- Active SLAM: SLAMの不確実性を最小化 or 情報量を最大化するようなロボットの動作を計画

- → 動作計画+SLAMの統合: 特徴の無い画像の取得を避けたり、効率的に地図構築する自律動作







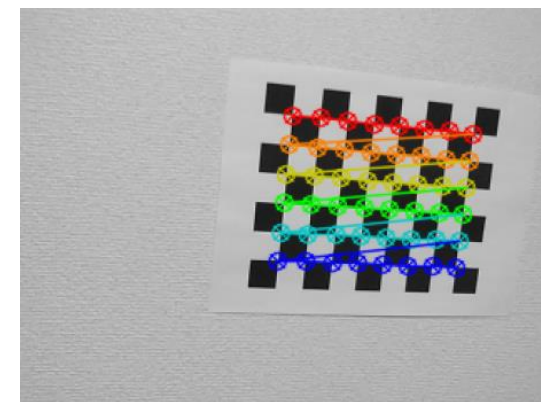
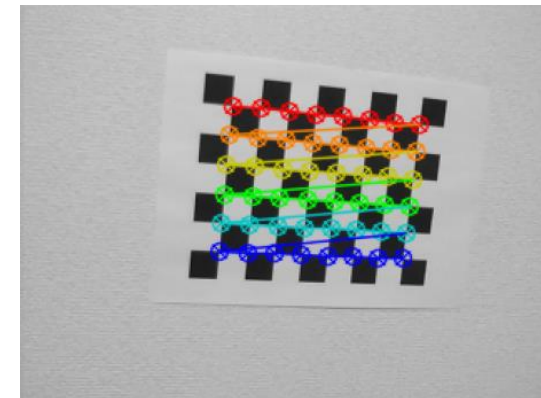
### 3. オドメトリ: 再投影誤差の最小化の応用

- カメラのキャリブレーション

- カメラ座標系と画像座標系の座標変換パラメータを取得
- 3次元空間の点を2次元画像に再投影する際に必須
- 手順
  1. キャリブレーションボードを用意
  2. キャリブレーションボードを写した画像を複数枚準備
  3. 複数画像でバンドル調整
    - →再投影誤差を最小化するように座標変換パラメータを推定
    - OpenCVで手軽に試せます

- カメラキャリブレーション+Visual SLAM

- どちらも「再投影誤差の最小化」で同時に解ける



カメラのキャリブレーション



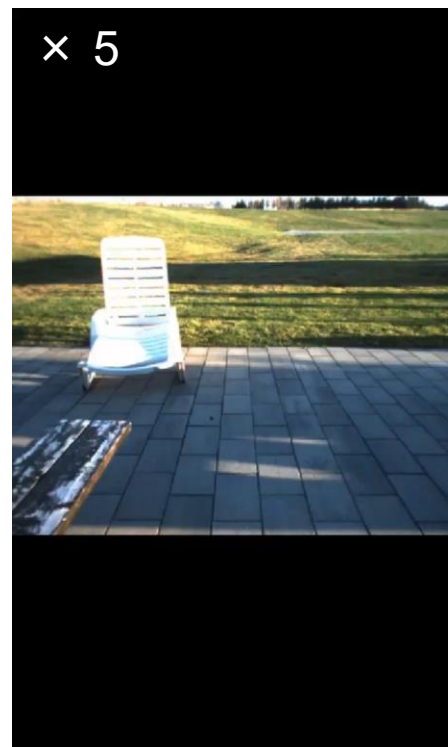
### 3. オドメトリ: Visual SLAM

- RTAB-Map (ROS1/ROS2対応)
  - 手軽に実行可能なVisual SLAM
    - コンパイルなしでも実行可能
    - 下記のコマンドだけでインストール完了
      - `sudo apt install ros-$ROS_DISTRO-rtabmap-ros`
  - 密な色付き点群を構築可能
  - RealsenseやKinectなどのRGB-Dカメラでよく使われる
    - ステレオカメラやLiDARにも対応

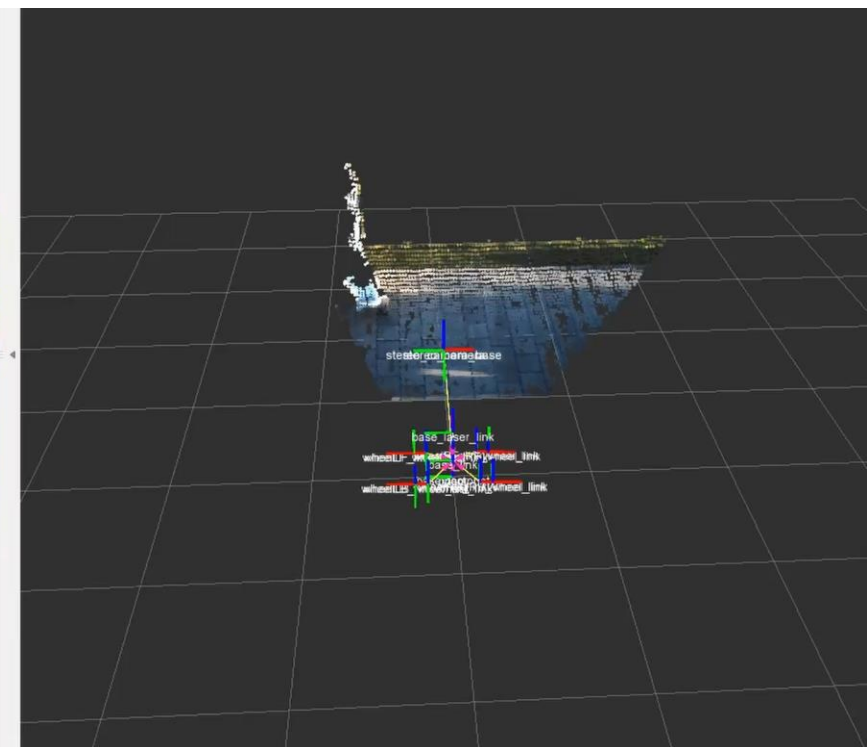
©Intel



©Microsoft



<https://www.youtube.com/watch?v=qpTS7kg9J3A>



RTAB-Mapを用いた移動ロボットによるSLAM

Labbé, Mathieu, et.al., "RTAB-Map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation." Journal of field robotics 36.2 (2019): 416-446.



## 3. オドメトリ: まとめ

---

- SLAMにおけるオドメトリ拘束について説明
  - エンコーダを用いたオドメトリ拘束
    - ホイールオドメトリ
  - LiDARを用いたオドメトリ拘束
    - ICP
    - デモ #1 ([gtsam\\_points](#))
    - デモ #2 ([GLIM](#))
  - カメラを用いたオドメトリ拘束
    - バンドル調整(再投影誤差の最小化)
    - デモ #3 ([RTAB-Map](#))



# 本日本話しする内容

---

1章: SLAMの概要

2章: SLAMの基礎理論

3章: オドメトリ推定手法

**4章: センサ融合によるSLAMの高性能化**

5章: SLAMの実装技術とそのポイント

6章: SLAMの実例・最新研究動向の紹介



## 4. センサ融合: SLAMで使われるセンサ

- **センサ融合**: 複数のセンサを組み合わせ、各センサの長所を活かし、短所を補う
- 外界センサ (SLAMのメイン)
  - カメラ
    - **長所**: 色情報、高密度な情報
    - **短所**: 視野の狭さ、**視覚的特徴が不十分だと破綻**
  - LiDAR
    - **長所**: 高精度な3次元位置を直接取得可能、360度視野
    - **短所**: 色情報なし、点群の間隔がスパース、**幾何特徴が不十分だと破綻**
- 内界センサ (メインの補助)
  - IMU
    - **長所**: 高速な動作にも対応可能、**環境の特徴に依存せずに移動量推定が可能**
    - **短所**: 並進方向の移動量には加速度の二階積分が必要、計測値のバイアスの考慮
  - エンコーダ
    - **長所**: IMUより安定した並進移動量の推定が可能、**環境の特徴に依存せずに移動量推定が可能**
    - **短所**: 運動学モデルの誤差の影響

©Intel



© Livox

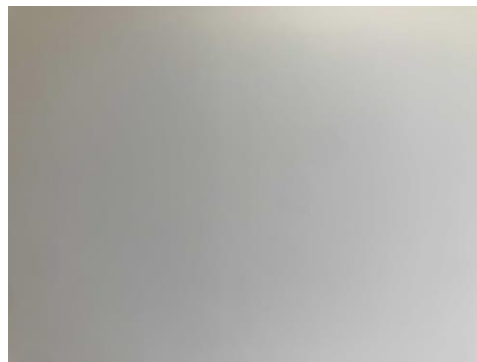




## 4. センサ融合: SLAMが失敗する例

### • Visual SLAMの場合

- 視覚**特徴**が検出できない
  - 白い壁など
- **特徴**マッチングが難しい
  - 連続パターンなど



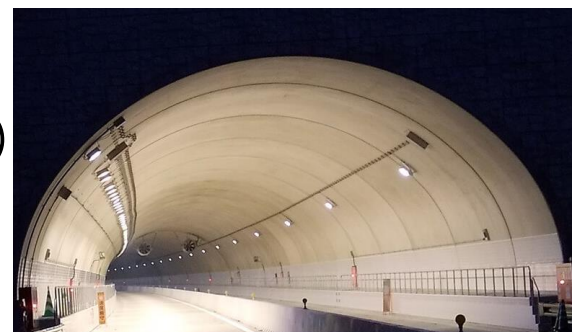
特徴が検出できない例: 白い壁



マッチングが難しい例: 連続パターン

### • LiDAR SLAMの場合

- 幾何**特徴**のマッチングが困難(**点群の縮退**)
  - 広い水平面だけの環境
  - トンネル(自動運転の課題)



幾何特徴が不十分な例: トンネル

360度視野であっても、  
かまぼこのような点群が  
常に取得される

↓  
**長手方向の位置が不定**

- → **環境の特徴に依存しないエンコーダやIMUとの融合が有効**





## 4. センサ融合: SLAMの高性能化の2つのアプローチ

※GICPはICPの改良版

### 1. SLAMにおける個々の要素を高性能化(例: ICP → GICP)

- 有効だが、**原理的な限界**がある(例: GICPの場合も、幾何特徴がなければ結局破綻)

### 2. センサ融合

- 複数のセンサを組み合わせ、各センサの長所を活かし、短所を補う

- 例

- LiDAR – カメラ

- LiDARで高精度に地図構築、カメラで地図の色付け
    - LiDARで幾何的な特徴、カメラで視覚的な特徴を取得

- LiDAR – IMU

- 高速動作において、IMUでLiDARの点群の歪みを補正
    - LiDARの拘束で、IMUデータのバイアスを推定

- LiDAR – エンコーダ

- LiDARで車輪や脚先の滑りを推定
    - エンコーダによるオドメトリで、LiDARによる拘束の破綻を補間

センサ融合によるSLAMは、**ポーズグラフ・ファクタグラフ**での実装が可能

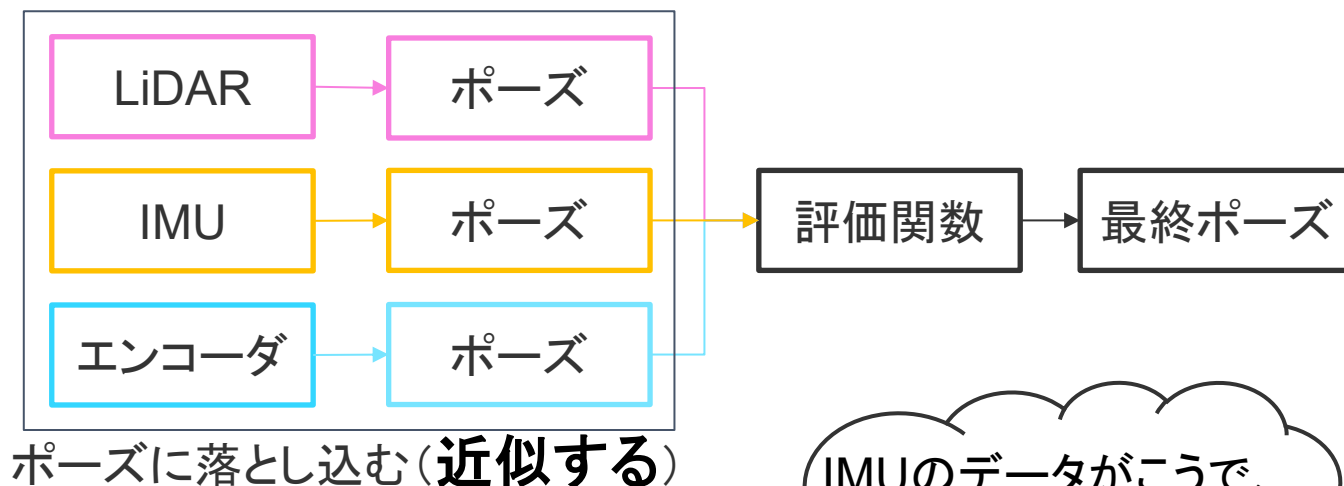
どのようなシステムになるか？



## 4. センサ融合: ルーズカップリングとタイトカップリング

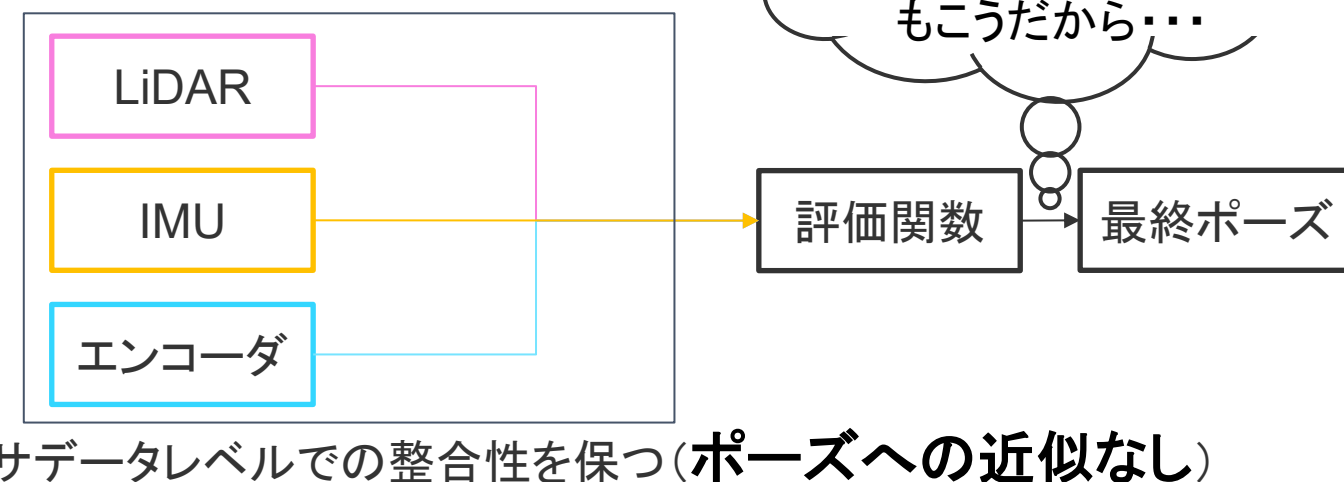
- ルーズカップリング

- 各観測からポーズを得る
- 各ポーズから評価関数を定義
- 評価関数を最適化して最終ポーズを取得
- **シンプルなシステムで実装可能**
  - ポーズグラフによるポーズ調整と等価



- タイトカップリング

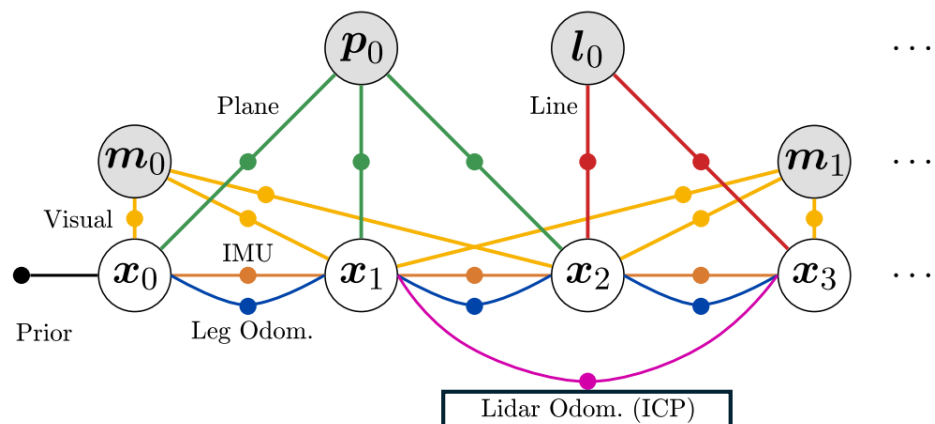
- 各観測から評価関数を直接定義
- 評価関数を最適化して最終ポーズを取得
- **複雑だが、精度や頑強性は高い**
  - ポーズグラフでは記述不可能
  - ファクタグラフで記述可能



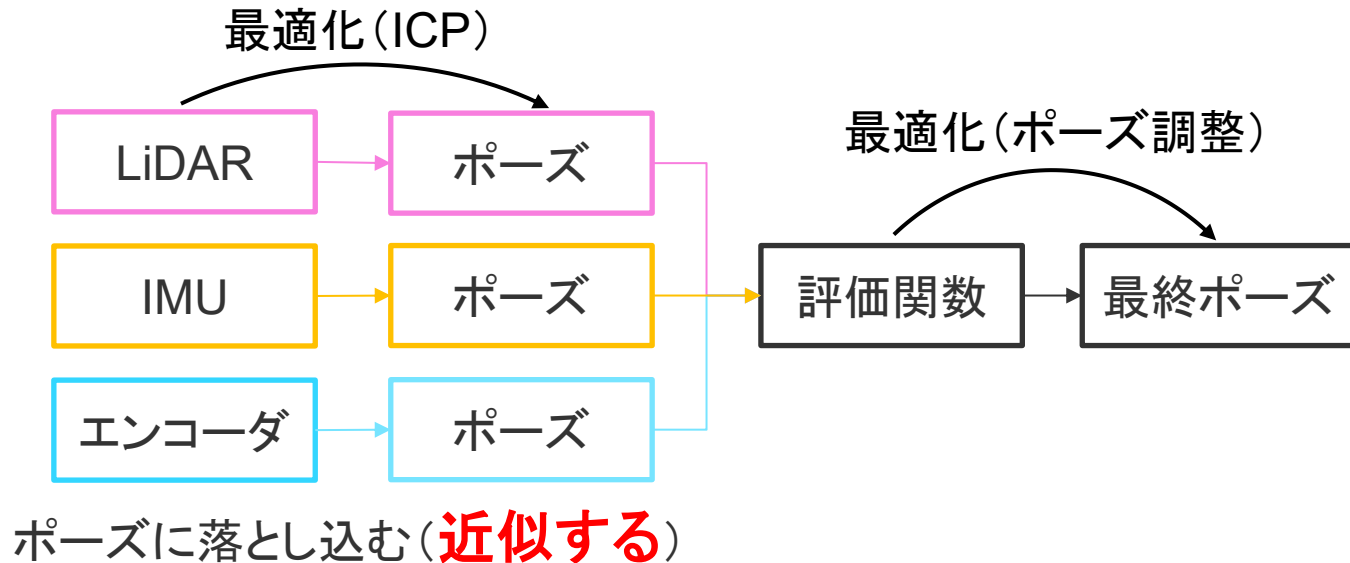


## 4. センサ融合: ルーズカップリングの例

- VILENSのICP(下図のピンクのファクタ)
  - ファクタとして記述しているが、VILENSでは相対ポーズを直接的に拘束としている
    - $x_1$ と $x_3$ の間の相対ポーズをICPで事前に計算し、観測としている
  - →  $x_1$ と $x_3$ 間の点群でICP(最適化)を実行し、さらにそこから全体的な最適化を実行
    - 最適化が二回行われ、一回目の最適化で、点群の情報はポーズの情報に圧縮される
      - システムが単純化される



VILENSという手法のファクタグラフ



Wisth, David, et.al., "VILENS: Visual, inertial, lidar, and leg odometry for all-terrain legged robots." IEEE Transactions on Robotics 39.1 (2022): 309-326.



## 4. センサ融合: タイトカップリングの例

- GLIMのMatching cost factor (ICPによる拘束) の例

- ICPの拘束を相対ポーズに圧縮せず、点群のまま最適化

- 点群の重なり of 整合性がとれるようなSLAM (最適化) が可能

- 矛盾の無い3次元地図の構築が可能

- 矛盾の無い地図 → 地図上の壁が二重になってたりしない

- 点群のまま最適化することの大変さ

- 全体の最適化の繰り返し計算の度にICPを実行

- 最近傍点探索など諸々全ての処理

- ICPの対象は、1ペアの点群だけでない

- たしかにシステムが複雑

- ルーズカップリングの場合、ICPは1回だけ

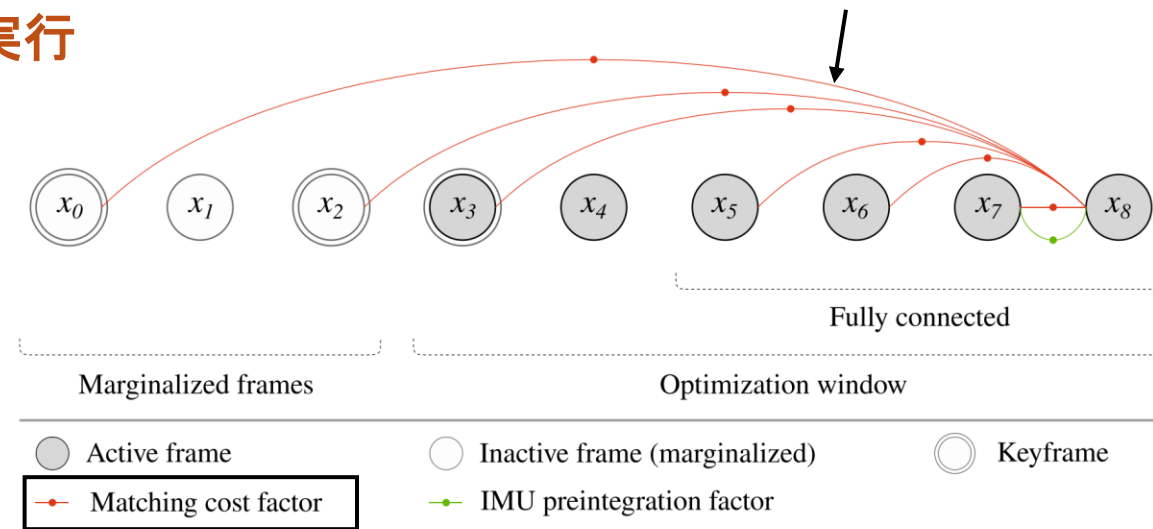
- GLIMでは、GPUによる高速化で実現可能

地図上で点群が二重



点群の重なり of 整合性を考慮しないと発生する齟齬

Matching cost factor (ICPによる拘束)



Koide, Kenji, et al. "GLIM: 3D range-inertial localization and mapping with GPU-accelerated scan matching factors." Robotics and Autonomous Systems (2024): 104750.

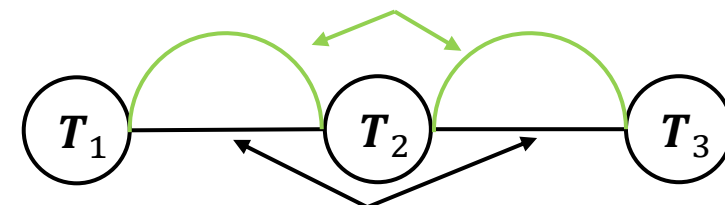


## 4. センサ融合: ルーズカップリングとタイトカップリングの実装

### • ルーズカップリングの実装例

- 各センサからロボットの移動量を計算
- **それらの移動量を並列なエッジとしてポーズグラフ構築**
- 各エッジから定義されるポーズグラフを最適化
  - GTSAMで容易に実装可能 ( BetweenFactor<Pose> )

ホイールオドメトリによる相対ポーズ



LiDARのICPによる相対ポーズ

### • タイトカップリングの実装例

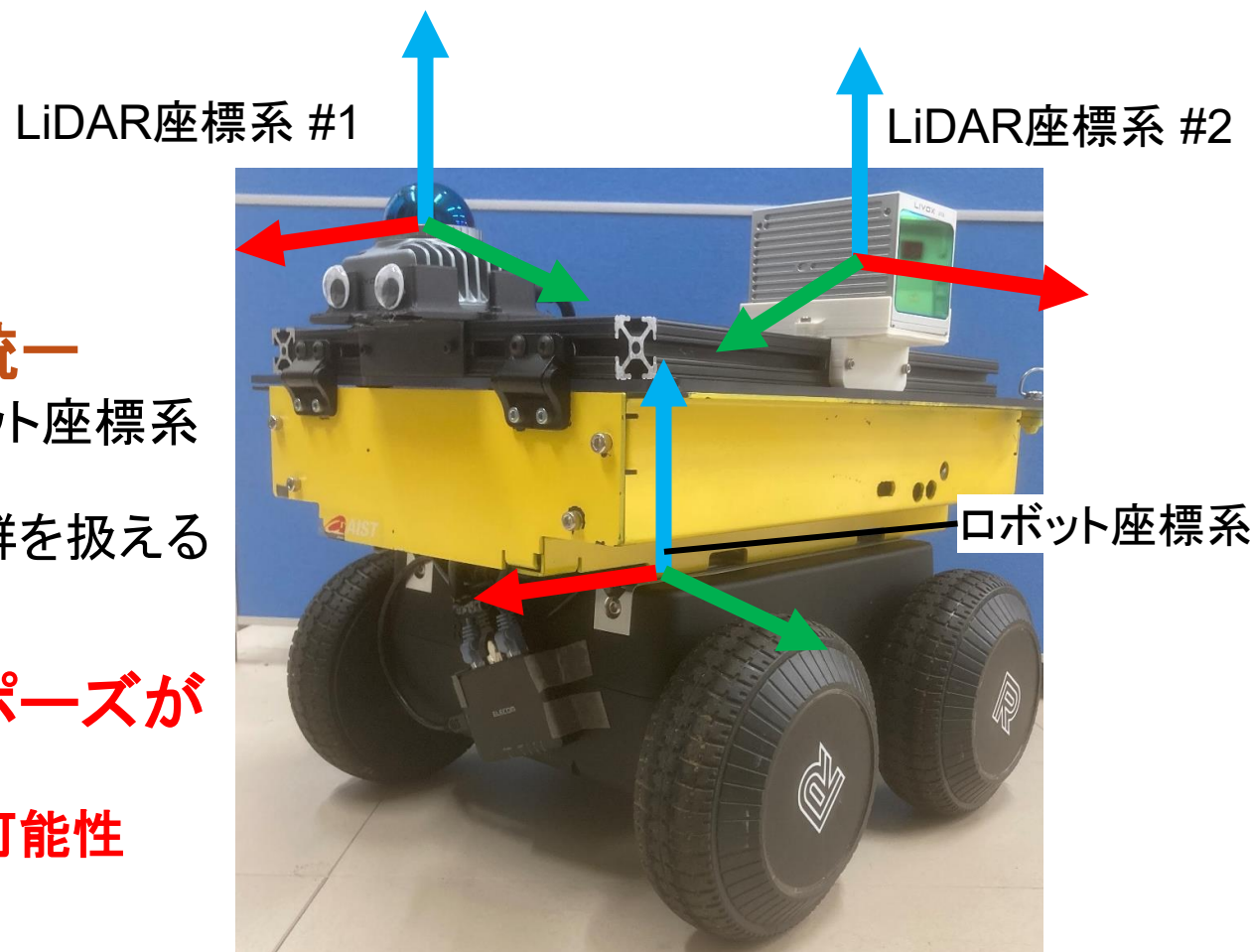
- **事前準備: 各センサの評価関数(ファクタ)を設計・実装**
- 各センサからの評価関数を統合し、最適化 (基本はルーズカップリングと同じ)
  - よく使われるファクタの場合、GTSAMで用意されている
    - 公式: IMUやカメラの拘束など
    - 有志: LiDARの拘束、滑りなどの誤差を保証したホイールオドメトリの拘束
      - [gtsam\\_points](#)
      - [full\\_linear\\_wheel\\_odometry\\_factor](#) ← ※URLです
- **新しいファクタを作る場合、自身で評価関数を設計・実装しなければならない**
  - ポーズグラフやファクタグラフ最適化の理論 (特にヤコビ行列) の理解が必要





## 4. センサ融合: センサ融合の実装で気を付ける事

- 各センサの座標系の統合
  - 各センサごとに座標系をもつ
    - ホイールオドメトリ: ロボット座標系
    - 点群: LiDAR座標系
  - **センサ融合では、各センサの座標系を統一**
    - 例: LiDAR座標系で記述された点群をロボット座標系に変換
    - → LiDARがロボット座標系にあるように点群を扱える
- 座標系を統一する際の難しさ
  - **ロボット座標系とLiDAR座標系の相対ポーズが正確でなければならない**
    - **センサ融合が推定結果に悪影響を及ぼす可能性**
    - 3D CADでの値では不十分な可能性
      - 加工誤差や取り付け誤差
  - **相対ポーズのキャリブレーションが重要**





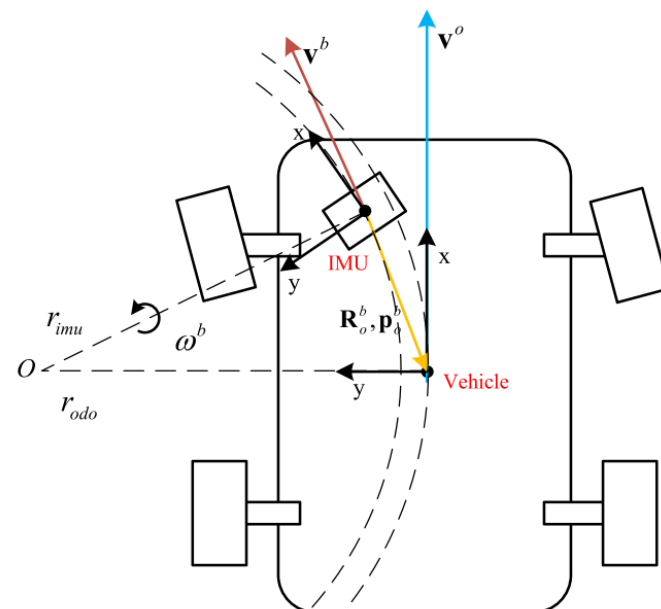


## 4. センサ融合: センサ融合を実装する際の注意

- センサポーズのキャリブレーション(外部パラメータキャリブレーション)
  - 各センサの相対ポーズを推定する問題(SLAMに関する研究テーマの一つ)
    - オフラインキャリブレーション
      - ロボット座標系とセンサ座標系の相対ポーズは不変の場合、オフラインで実行可能
    - オンラインキャリブレーション
      - SLAMと同時に、センサ間の相対ポーズをオンラインで推定する問題
        - → センサ間の相対ポーズを最適化変数とする
        - → 全ての観測で整合性がとれる相対ポーズを推定する
      - ファクタグラフ(GTSAM)で容易に実装可能

# Kalibr

[kalibr](#) (IMUとカメラ間のオフラインキャリブレーション)



ロボット座標系とIMU座標系のオンラインキャリブレーション

Zhao, Hang, et.al.. "Vehicle-Motion-Constraint-Based Visual-Inertial-Odometer Fusion With Online Extrinsic Calibration." *IEEE Sensors Journal* (2023).



## 4. センサ融合: まとめ

---

- センサ融合の重要性
- ルーズカップリング/タイトカップリングについて
  - 概念
  - 例
  - 実装方法



# 本日本話しする内容

---

1章: SLAMの概要

2章: SLAMの基礎理論

3章: オドメトリ推定手法

4章: センサ融合によるSLAMの高性能化

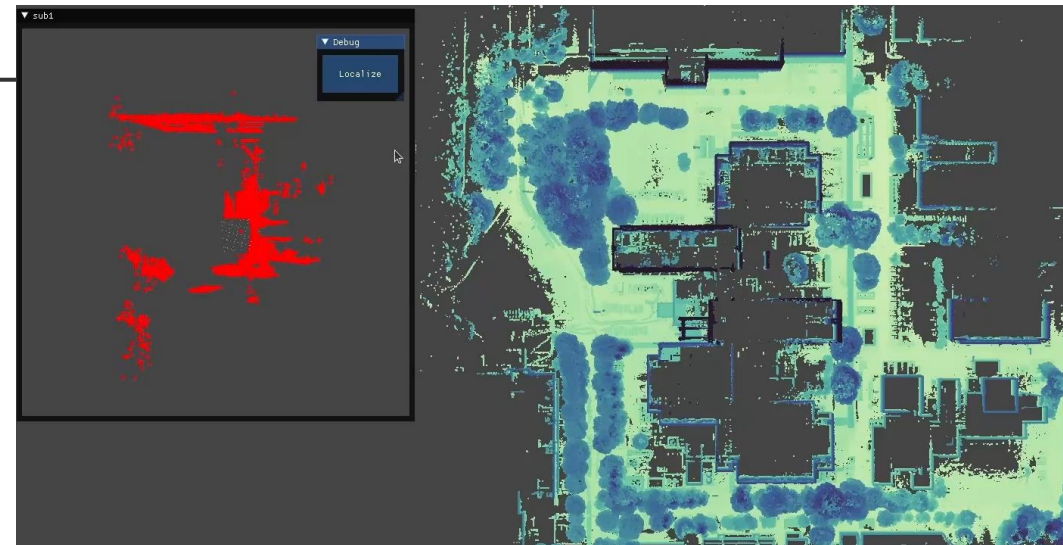
**5章: SLAMの実装技術とそのポイント**

6章: SLAMの実例・最新研究動向の紹介



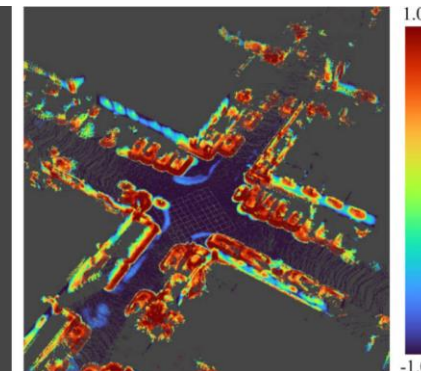
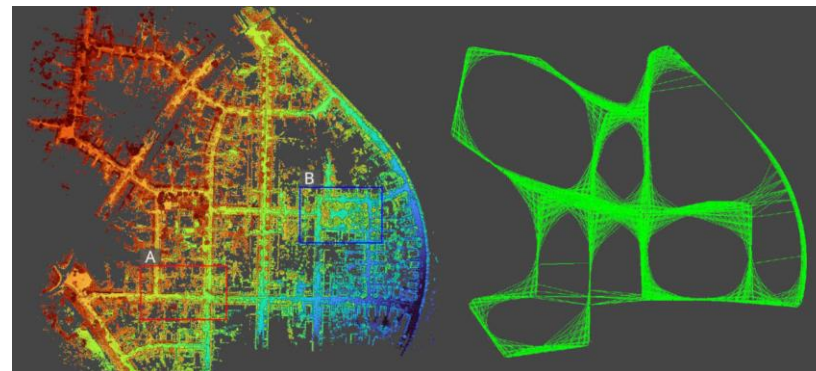
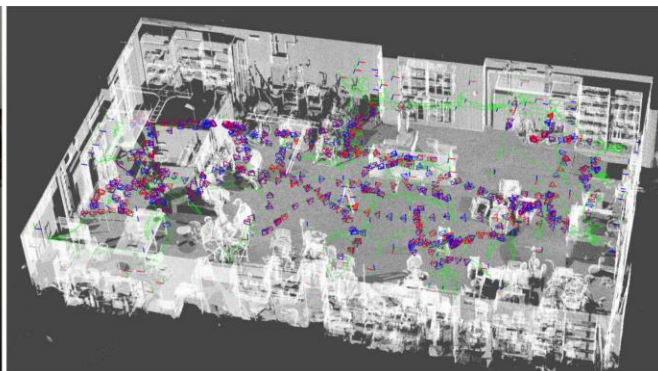
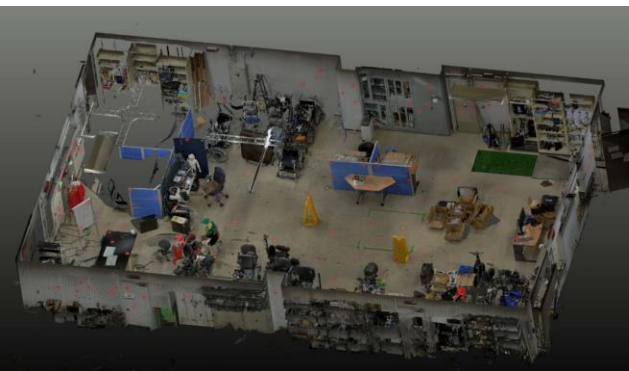
## 5. 実装技術: 可視化ライブラリ

- Iridescence (<https://github.com/koide3/iridescence>)
  - 本セミナーのSLAMのコードの可視化で使用
  - SLAMに関する技術に特化した可視化ツール
    - 軌跡や座標系、点群などを任意に配置可能
    - imgui(クリック動作に基づくインターフェース)
      - 自由度の高いインターフェース機能の実装
        - パラメータ調整バーや実行ボタンなど
    - ROSへの依存ナシ(topicを定義する必要がない)



imguiを用いたクリック動作による自己位置推定

<https://www.youtube.com/watch?v=QGxnDnwG1k>



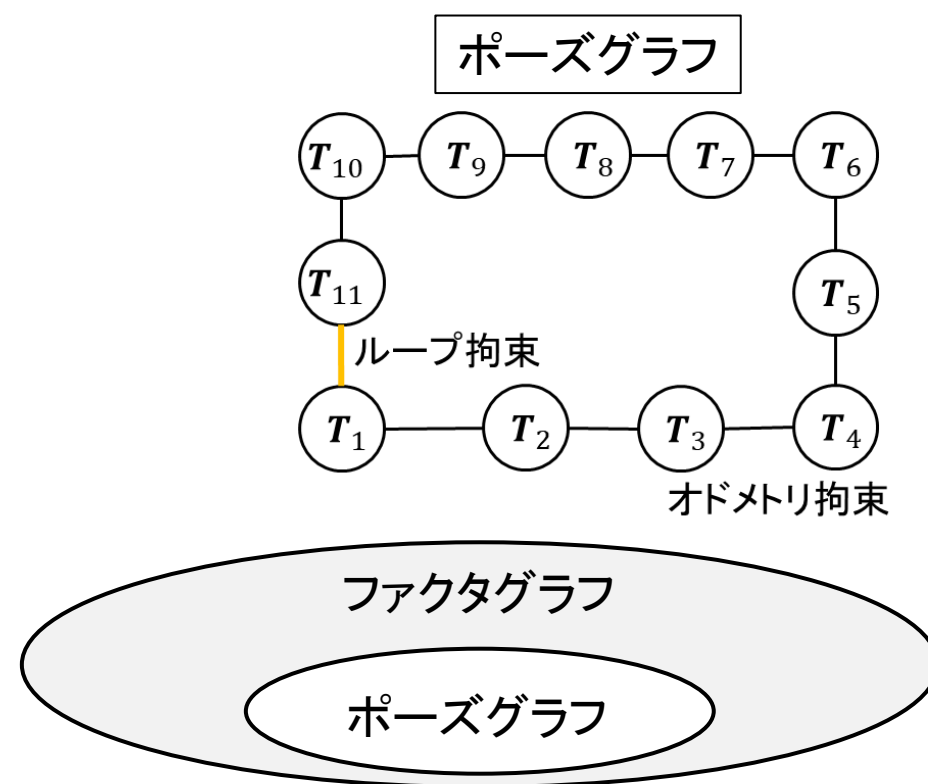
Iridescenceを用いた軌跡、地図

Aoki, Koki, et al. "3d-bbs: Global localization for 3d point cloud scan matching using branch-and-bound algorithm.", IEEE International Conference on Robotics and Automation (ICRA), 2024



## 5. 実装技術: ファクタグラフ最適化のライブラリ(GTSAM)

- GTSAM
  - ファクタグラフ最適化のためのライブラリ
- 【おさらい】ファクタグラフ
  - ポーズグラフを一般化させたグラフ
  - → GTSAMでは、ポーズグラフも処理可能
- サンプルプログラムを用いたデモの内容
  - GTSAMを用いてポーズ調整を実装
- 【おさらい】ポーズ調整
  - ポーズグラフ(オドメトリ拘束、ループ拘束から成るグラフ)を最適化
  - 局所的整合性、大域的整合性を保持した推定(SLAMの本質)が可能







## 5. 実装技術: ファクタグラフ最適化のライブラリ(GTSAM)

- GTSAMを使った実装
  - **GTSAM: ファクタグラフ最適化のためのライブラリ**
  - GTSAMに実装済みのもの
    - 基本的な最適化アルゴリズム (ガウスニュートン法、レーベンバーグマーカート法、ISAM2)
    - 基本的なファクタ (相対ポーズ、再投影誤差、IMUファクタなど)
    - 新しいファクタを実装する際の便利なフレームワーク (NoiseModelFactorなど)
  - GTSAMを使って自身でSLAMを実装する流れ (= 以降のデモの内容)
    - **ファクタグラフ(ポーズグラフ)の構築、最適化を実行するプログラムの実装**
    - 新しい拘束(ファクタ)を作る場合、提供されてるフレームワークに基づいてファクタの実装
      - 例: ICP、滑りなどの誤差を考慮したホイールオドメトリ
  - 余談
    - GTSAMは、SLAMだけでなくファクタグラフ最適化のライブラリ
    - → SLAM以外の最適化問題にも応用可能
      - 例: ロボットアームの逆運動学の数値解法、行動計画

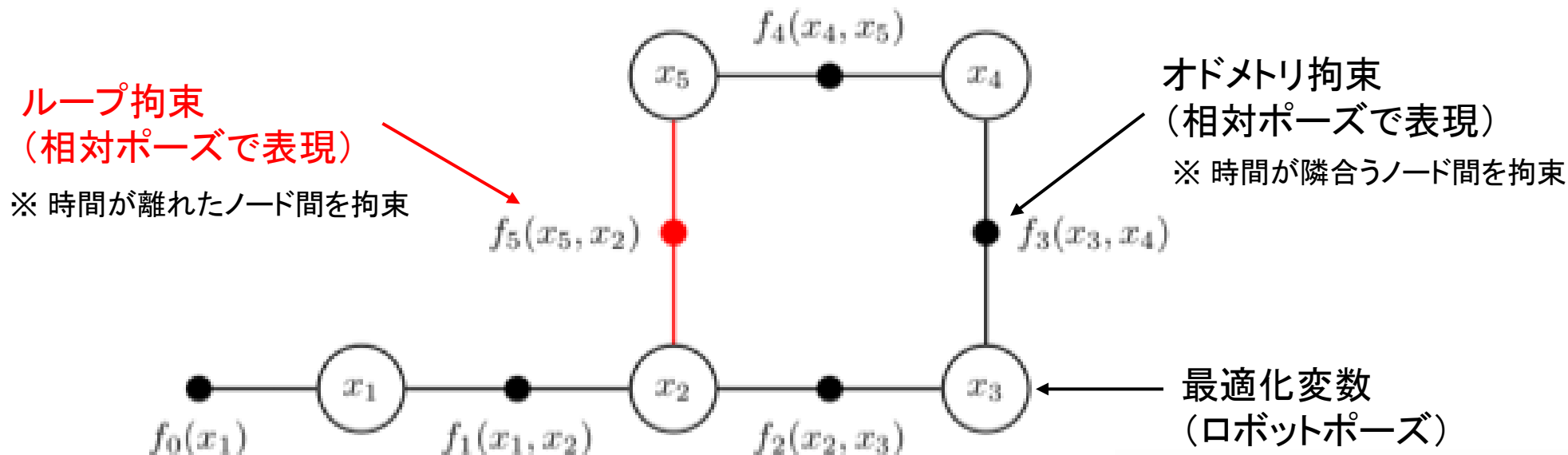




## 5. 実装技術: GTSAMでの簡単なポーズ調整

- 扱う題材: ポーズグラフによるSLAM
  - 概要
    - 多くのSLAMの手法で採用されている**ポーズ調整**(ポーズグラフ最適化)を実装
      - gtsamの[Pose2SLAMExample.cpp](#)を参照

### ポーズ調整のためのファクタグラフ(ポーズグラフ)



<https://gtsam.org/tutorials/intro.html>



## 5. 実装技術: GTSAMでの簡単なポーズ調整

右図のポーズグラフ・ポーズ調整を実装するプログラム

重要な箇所 @ [small\\_pose\\_adjustment\\_2d.cpp](#)

ファクタグラフ(ポーズグラフ)のクラスをインスタンス化

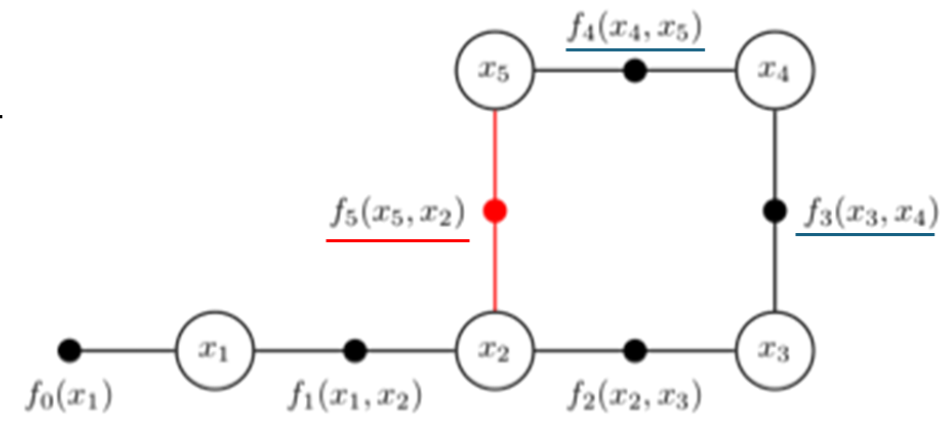
```
56 NonlinearFactorGraph graph;
```

ポーズグラフにオドメトリ拘束を追加

```
68 graph.emplace_shared<BetweenFactor<Pose2>> (<u>1</u>, <u>2</u>, Pose2(2, 0, 0), noise_model);
69 graph.emplace_shared<BetweenFactor<Pose2>> (<u>2</u>, <u>3</u>, Pose2(2, 0, M_PI_2), noise_model);
70 graph.emplace_shared<BetweenFactor<Pose2>> (<u>3</u>, <u>4</u>, Pose2(2, 0, M_PI_2), noise_model);
71 graph.emplace_shared<BetweenFactor<Pose2>> (<u>4</u>, <u>5</u>, Pose2(2, 0, M_PI_2), noise_model);
```

ポーズグラフにループ拘束を追加

```
77 graph.emplace_shared<BetweenFactor<Pose2>> (<u>5</u>, <u>2</u>, Pose2(2, 0, M_PI_2), noise_model);
```



ファクタを定義するノード

オドメトリの相対ポーズ

ノイズ(拘束は正規分布)

連続しない番号

## ポーズ調整

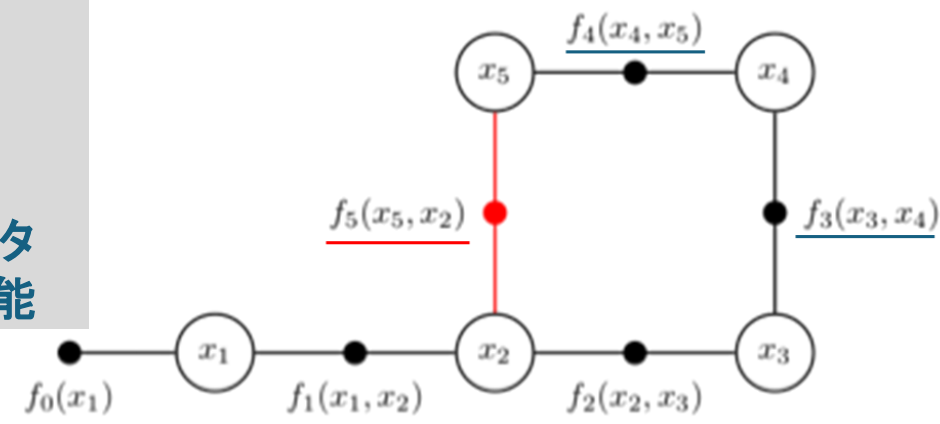
ポーズ調整の評価関数

$$E = \sum_i \underbrace{E_{i-1,i}^T \Sigma_{i-1,i}^{-1} E_{i-1,i}}_{\text{オドメトリ拘束}} + \sum_{s,t} \underbrace{E_{s,t}^T \Sigma_{s,t}^{-1} E_{s,t}}_{\text{ループ拘束}}$$

$E$ : 誤差ベクトル(同次変換行列にlog mapを適用)  
 $\Sigma$ : 共分散行列

BetweenFactor<Pose2>で、  
**オドメトリ拘束・ループ拘束を  
 実装可能**

オドメトリ拘束・ループ拘束を  
 graphに追加するだけで、**ファクタ  
 グラフ(ポーズグラフ)を実装可能**



ファクタグラフ(ポーズグラフ)のクラスをインスタンス化

```
56 NonlinearFactorGraph graph;
```

ポーズグラフにオドメトリ拘束を追加

```
68 graph.emplace_shared<BetweenFactor<Pose2> >(1, 2, Pose2(2, 0, 0), noise_model);
69 graph.emplace_shared<BetweenFactor<Pose2> >(2, 3, Pose2(2, 0, M_PI_2), noise_model);
70 graph.emplace_shared<BetweenFactor<Pose2> >(3, 4, Pose2(2, 0, M_PI_2), noise_model);
71 graph.emplace_shared<BetweenFactor<Pose2> >(4, 5, Pose2(2, 0, M_PI_2), noise_model);
```

ポーズグラフにループ拘束を追加

```
77 graph.emplace_shared<BetweenFactor<Pose2> >(5, 2, Pose2(2, 0, M_PI_2), noise_model);
```

ファクタを定義するノード

オドメトリの相対ポーズ

ノイズ(拘束は正規分布)

連続しない番号



## 5. 実装技術: GTSAMでの簡単なポーズ調整

重要な箇所 @ [small\\_pose\\_adjustment\\_2d.cpp](#)

最適化変数(2次元のポーズ)の初期値を設定

```
82     Values initialEstimate;  
83     initialEstimate.insert(1, Pose2(0.5, 0.0, 0.2));  
84     initialEstimate.insert(2, Pose2(2.3, 0.1, -0.2));  
85     initialEstimate.insert(3, Pose2(4.1, 0.1, M_PI_2));  
86     initialEstimate.insert(4, Pose2(4.0, 2.0, M_PI));  
87     initialEstimate.insert(5, Pose2(2.1, 2.1, -M_PI_2));
```

Final Result:

Values with 5 values:

Value 1: (gtsam::Pose2)  
(-1.27455483e-20, 8.10527674e-20, 3.07189987e-20)

Value 2: (gtsam::Pose2)  
(2, 1.78514217e-19, 4.34262714e-20)

Value 3: (gtsam::Pose2)  
(4, -3.42173202e-11, 1.57079633)

Value 4: (gtsam::Pose2)  
(4, 2, 3.14159265)

Value 5: (gtsam::Pose2)  
(2, 2, -1.57079633)

ガウスニュートン法を実行するクラスをインスタンス化、ポーズグラフと初期値をセット

```
92     GaussNewtonOptimizer optimizer(graph, initialEstimate);
```

ガウスニュートン法により、ポーズ調整(ファクタグラフ最適化)を実行

```
94     Values result = optimizer.optimize();
```

ポーズ調整の結果



## 5. 実装技術: GTSAMでの簡単なポーズ調整

- ポーズ調整実装の肝: `BetweenFactor<T>`
  - 任意の最適化変数の差分に基づく誤差関数を定義可能なファクタ
    - → `BetweenFactor<Pose2>`により、**相対ポーズの差分**(オドメトリ拘束、ループ拘束)を定義
      - 非常によく使われる
      - ルーズカップリングの場合は、`BetweenFactor`だけでグラフを構築できる
  - `T`には、任意の型を設定可能
    - `Pose2`ではなく`Pose3`にすれば、3次元の相対ポーズに基づく拘束を実装可能
    - → 2次元から3次元に拡張するのが容易
  - ポーズグラフとファクタグラフの違い(おさらい)
    - 拘束が相対ポーズのファクタグラフ = ポーズグラフ
    - → `BetweenFactor<Pose2>`や`BetweenFactor<Pose3>`のファクタグラフは、ポーズグラフと等価



## 5. 実装技術: GTSAMでの簡単なポーズ調整

- GTSAMの使い方 まとめ

- 使う前のSLAM実装に関する疑問

1. ファクタグラフってどのように実装するの？
2. 相対ポーズに基づく拘束(=ファクタ)ってどのように実装するの？
3. 最適化ってどのように実装するの？

- 解答: GTSAMがほとんどをカバーしてくれる

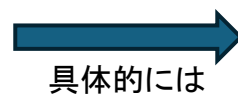
- ファクタグラフの実装 → 56 `NonlinearFactorGraph graph;`

- 相対ポーズに基づく拘束(ファクタ) → 68 `graph.emplace_shared<BetweenFactor<Pose2>>(1, 2, Pose2(2, 0, 0), noise_model);`  
77 `graph.emplace_shared<BetweenFactor<Pose2>>(5, 2, Pose2(2, 0, M_PI_2), noise_model);`

- 最適化の実装 → 92 `GaussNewtonOptimizer optimizer(graph, initialEstimate);`

- 自分で実装する箇所は？

- センサデータのハンドリング
  - そのセンサデータを、拘束(ファクタ)に入力



@ スライド 37ページ

- ポーズグラフによるSLAMの誤差関数の表現方法

- $E_{1,2} = T_{1,2}^{-1} \hat{T}_{1,2} = (T_1^{-1} T_2)^{-1} \hat{T}_{1,2}$

$\hat{T}_{1,2}$  相当、つまりはファクタの観測を取得すること

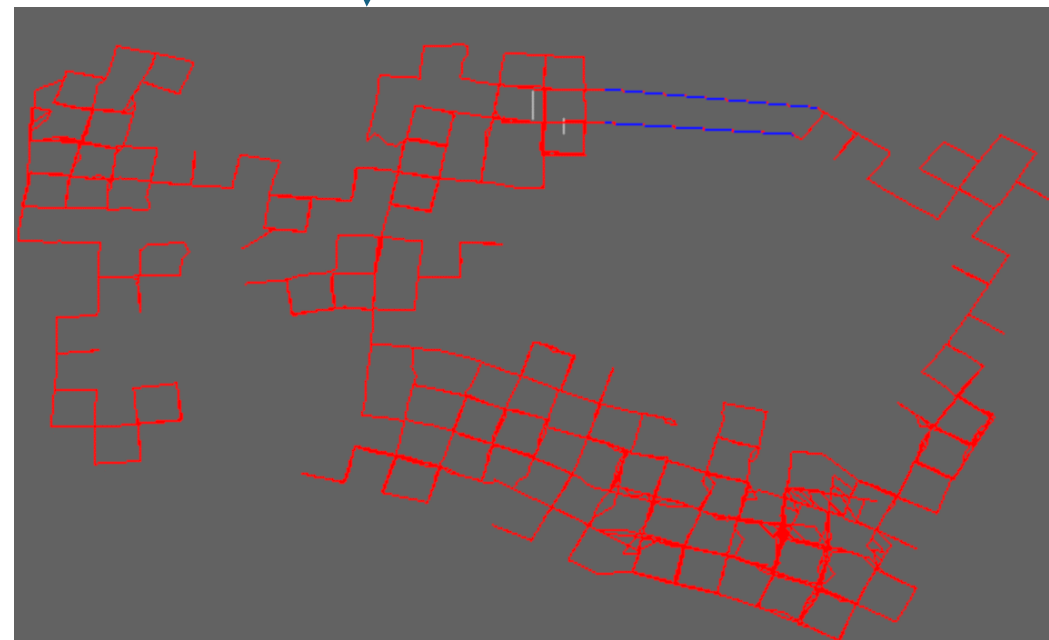




## 5. 実装技術: 2Dの実践的なポーズ調整

- マンハッタンデータセットを用いた例
  - 2次元のポーズ調整をテストするためのデータセット(.g2oファイル形式)
  - データセットの内容
    - 2次元のロボットポーズ(軌跡)の初期値
    - 上記のロボットポーズ間の相対ポーズとノイズ
      - オドメトリ拘束
      - ループ拘束
  - ゴール: 歪みが取れた軌跡を推定**

マンハッタンデータセットの軌跡の初期値





## 5. 実装技術: 2Dの実践的なポーズ調整

重要な箇所@ [pose\\_adjustment\\_2d.cpp](#)

ファクタグラフのgraphと初期値のinitialをインスタンス化する。

```
64         NonlinearFactorGraph graph;  
65         Values initial;
```

余談: GTSAMでSLAMを実装する際は、readG2OFile相当を実装するのがメイン。他は、GTSAMがやってくれる。

.g2oファイルから、初期値や拘束の情報を読み取る。

読み取った結果をファクタグラフのgraphと初期値のinitialに格納する(単なるデータの読み取りだけ)。

```
70         readG2OFile(filename, graph, initial);
```

ガウスニュートン法を実行するクラスをインスタンス化、ポーズグラフと初期値をセットする。  
ガウスニュートン法により、ポーズ調整を実行する。

```
91         GaussNewtonOptimizer optimizer(graph, initial);  
92         Values result = optimizer.optimize();
```

**最適化変数や拘束が増えたが、実装の基本は先ほどのシンプルな例とまったく同じ**



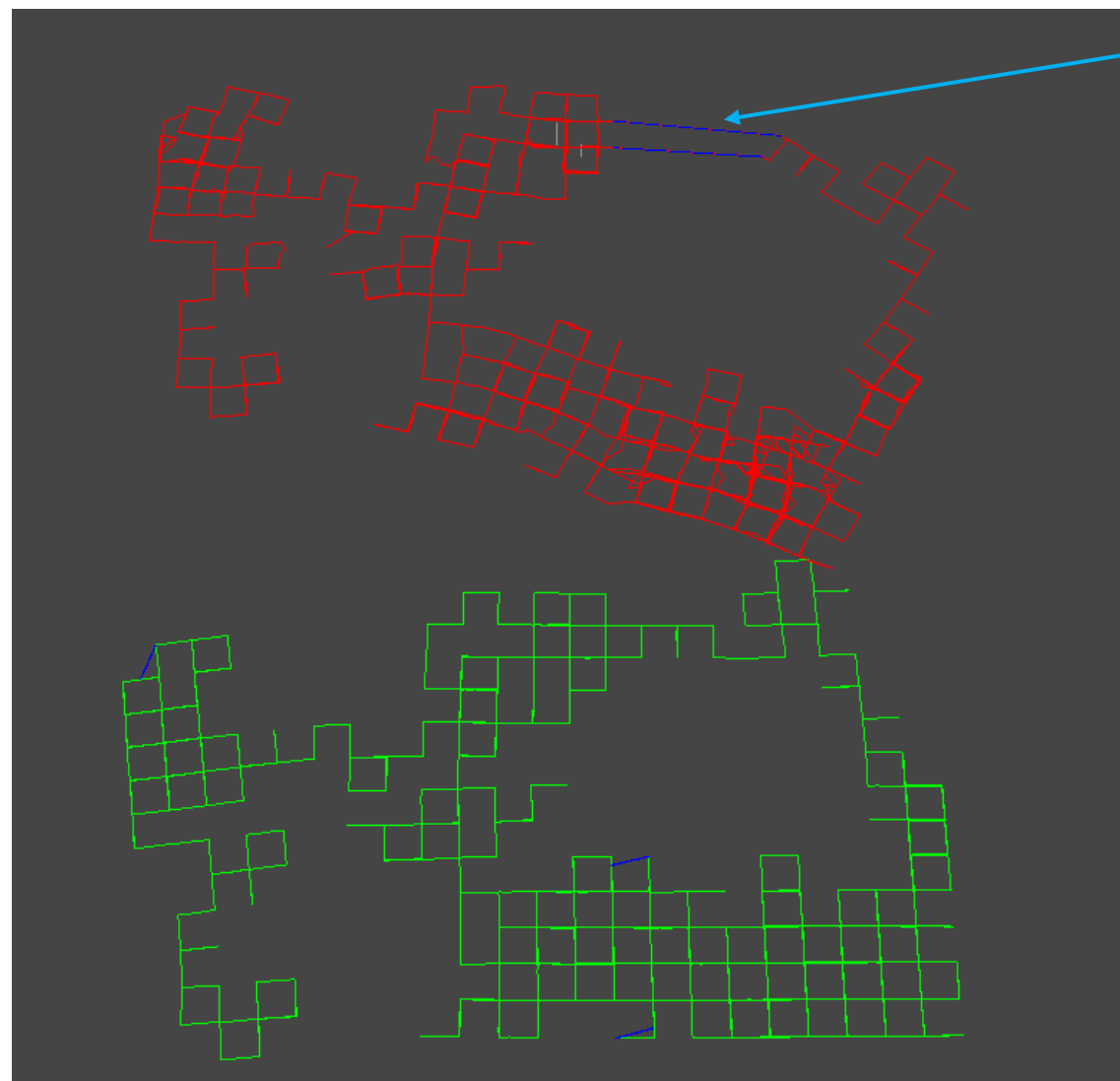
## 5. 実装技術: 2Dの実践的なポーズ調整

### 最適化前

軌跡が歪んでいる  
ループ拘束(例:右上の青い線)がある

### 最適化後

軌跡の歪みが解消されている



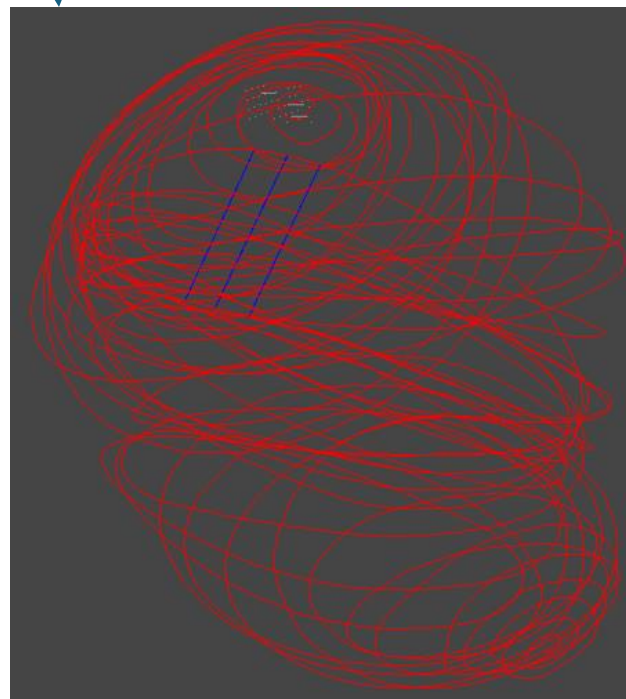
代表的な  
ループ拘束



## 5. 実装技術: 3Dの実践的なポーズ調整

- スフィアデータセットを用いた例
  - **3次元**のポーズ調整をテストするためのデータセット(.g2oファイル形式)
  - データセットの内容
    - **3次元**のロボットポーズ(軌跡)の初期値
    - 上記のロボットポーズ間の相対ポーズとノイズ
      - オドメトリ拘束
      - ループ拘束
  - **ゴール: 歪みが取れた軌跡(綺麗な球)を推定**

スフィアデータセットの軌跡の初期値





## 5. 実装技術: 3Dの実践的なポーズ調整

重要な箇所@ [pose\\_adjustment\\_3d.cpp](#)

ファクタグラフのgraphと初期値のinitialをインスタンス化する。

```
64         NonlinearFactorGraph graph;  
65         Values initial;
```

.g2oファイルから、初期値や拘束の情報を読み取り、ファクタグラフのgraphと初期値のinitialに格納する。

```
70         readG2OFile(filename, graph, initial);
```

**BetweenFactor<Pose3>で相対ポーズによる拘束、最適化変数を定義する。**

```
61         graph.emplace_shared<BetweenFactor<Pose3>>( ...
```

ガウスニュートン法を実行するクラスをインスタンス化、ポーズグラフと初期値をセットし、ポーズ調整を実行。

```
91         GaussNewtonOptimizer optimizer(graph, initial);  
92         Values result = optimizer.optimize();
```

**3次元でのポーズ調整だが、拘束や最適化変数をPose2からPose3に変更しただけ**

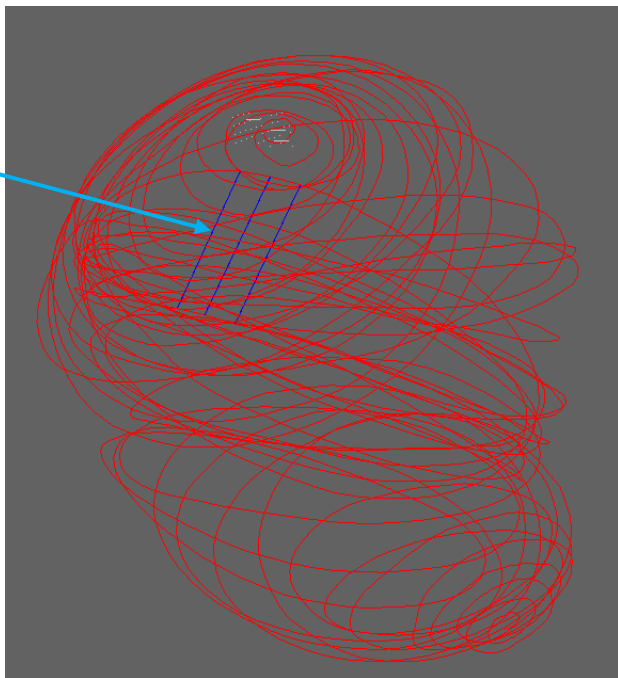


## 5. 実装技術: 3Dの実践的なポーズ調整

### 最適化前

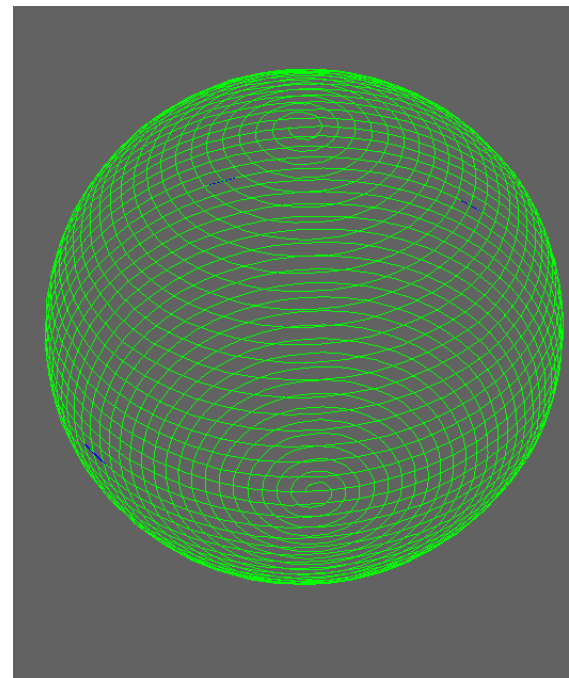
軌跡が歪んでいる  
ループ拘束(例: 真ん中付近の青い線)がある

代表的な  
ループ拘束



### 最適化後

軌跡の歪みが解消されている







## 5. 実装技術: GTSAMを用いて新しいSLAMを実装するには

### • オリジナルの拘束の作成のデモ

- BetweenFactorなどのようにGTSAMで既の実装された拘束ではなく、  
**自身でオリジナルの拘束(ファクタ)を作成**

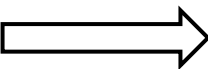
### • オリジナルの拘束を作成するモチベーション

- GTSAMでは、実装されている拘束(ファクタ)には限りがあった
- **GTSAMで実装されていないファクタは、自身で実装する必要があった**
  - 例: ICP、滑りなどの誤差を考慮したホイールオドメトリ

### • デモの内容

- BetweenFactor<Pose3>と等価なファクタを自身で実装
  - **オリジナルの拘束を実装する際の基本がつまっている**
  - 正しい実装か否かが判別しやすい
    - 正しく実装されていれば、スフィアデータセットでのポーズ調整において、BetweenFactorと同じ結果

ポーズ調整の理論の理解が必要





## 5. 実装技術: オリジナルな拘束の実装

- BetweenFactor<Pose3>と等価なファクタの実装に必要な知識(復習)
  - ポーズ調整の評価関数(再掲)

$$E = \underbrace{\sum E_{i-1,i}^{-1} \Sigma_{i-1,i}^{-1} E_{i-1,i}}_{\text{オドメトリ拘束}} + \underbrace{\sum E_{s,t}^{-1} \Sigma_{s,t}^{-1} E_{s,t}}_{\text{ループ拘束}}$$

$E$ : 誤差ベクトル(同次変換行列にlog mapを適用)  
 $\Sigma$ : 共分散行列

- Log map による誤差の表現方法(再掲)
  - $E_{1,2} = \log(T_1^{-1}T_2)^{-1}\hat{T}_{1_2}$
  - → Log map により、同次変換行列  $(T_1^{-1}T_2)^{-1}\hat{T}_{1_2}$  で記述された誤差をベクトルとして表現
    - この例だと、最適化変数は  $T_1$  と  $T_2$
- オドメトリ拘束もループ拘束も、どちらも相対ポーズによる拘束
  - 違いは、時間的に連続したポーズ間、離れたポーズ間を拘束するかという点のみ
  - → オドメトリ拘束・ループ拘束ともに統一したファクタで実装可能



## 5. 実装技術: オリジナルな拘束の実装

- BetweenFactor<Pose3>と等価なファクタの実装するデモ
  - オリジナルの拘束をクラス(CustomBetweenFactor)として、専用のhppとcppを実装

重要な箇所@ [customfactor.hpp](#)

```

20  class CustomBetweenFactor : public gtsam::NoiseModelFactor2<gtsam::Pose3, gtsam::Pose3> {
21      private:
22          gtsam::Pose3 measured_;
23
24      public:
25          CustomBetweenFactor(
26              gtsam::Key key1,
27              gtsam::Key key2,
28              const gtsam::Pose3& measured,
29              const gtsam::SharedNoiseModel& model);
30
31          virtual ~CustomBetweenFactor() {}
32
33          gtsam::Vector evaluateError(
34              const gtsam::Pose3 &pp, const gtsam::Pose3 &cp,
35              boost::optional<gtsam::Matrix&> H1 = boost::none, boost::optional<gtsam::Matrix&> H2 =
  
```

NoiseModelFactor2から継承

$$\text{例: } E_{1,2} = \log(\underline{T_1}^{-1} \underline{T_2})^{-1} \hat{T}_{1,2}$$

最適化変数のタイプを設定。  
相対ポーズの拘束なので、2つのポーズ。

重要

オリジナルな拘束の実装のゴール  
= evaluateError()の中身を実装



## 5. 実装技術: オリジナルな拘束の実装

- BetweenFactor<Pose3>と等価なファクタの実装
  - オリジナルの拘束を実装  $\approx$  evaluateError()の中身を実装
    - **ベクトルによる誤差**
      - つまりは、Log map によってベクトル化された誤差 (例:  $\log(T_1^{-1}T_2)^{-1}\hat{T}_{1_2}$ )
    - **ヤコビ行列 (H1, H2)**
      - H1, H2は、誤差ベクトルの最適化変数に関するヤコビ行列 (最適化計算で必要)
      - ほとんどのケースにて、GTSAMがヤコビ行列を計算してくれる
  - 残りの作業は、GTSAMが実行してくれる

重要な箇所@ [customfactor.hpp](#)

```
33     gtsam::Vector evaluateError(  
34         const gtsam::Pose3 &pp, const gtsam::Pose3 &cp,  
35         boost::optional<gtsam::Matrix&> H1 = boost::none, boost::optional<gtsam::Matrix&> H2 =
```

evaluateError()では、**ベクトルによる誤差**をreturn

**ヤコビ行列**



# 5. 実装技術: オリジナルな拘束の実装

重要な箇所@ [customfactor.cpp](#)

## • evaluateError()の中身

### 1. 下記の変数から誤差を計算

- 最適化変数:
  - pp: 直前のポーズ
  - cp: 現在のポーズ
- 観測値:
  - measured\_: 相対ポーズ

誤差の具体的な式

$$\frac{E_{1,2}}{\text{Error}} = \log(\frac{T_1^{-1}T_2}{\text{pp cp}})^{-1}\hat{T}_{1,2} \text{ Measured\_}$$

```
17  ▼ gtsam::Vector CustomBetweenFactor::evaluateError(  
18      const gtsam::Pose3 &pp, const gtsam::Pose3 &cp,  
19      boost::optional<gtsam::Matrix&> H1, boost::optional<gtsam::Matrix&> H2)  
20  
21      // d(T_pp_cp) / d(previous_pose)  
22      gtsam::Matrix66 H_dp_pp;  
23      // d(T_pp_cp) / d(current_pose)  
24      gtsam::Matrix66 H_dp_cp;  
25  
26      // delta pose between previous pose and current pose  
27      gtsam::Pose3 dp = pp.between(cp, H_dp_pp, H_dp_cp);  
28  
29  
30  
31      // d(Error) / d(dp)  
32      gtsam::Matrix66 H_E_dp;  
33      gtsam::Pose3 Error = dp.between(measured_, H_E_dp);  
34  
35      // d(log_Error) / d(Error)  
36      gtsam::Matrix6 H_logE_E;  
37  
38      // Error with logmap  
39      gtsam::Vector6 log_Error = gtsam::Pose3::Logmap(Error, H_logE_E);
```



# 5. 実装技術: オリジナルな拘束の実装

重要な箇所@ [customfactor.cpp](#)

## • evaluateError()の中身

### 1. 下記の変数から誤差を計算

- 最適化変数: pp, cp
- 観測値: measured\_

### 2. ヤコビ行列を計算

- ポーズの演算のヤコビ行列は、GTSAMが計算してくれる
  - 例:  $\frac{\partial(\Delta T)}{\partial(T_{pp})}, \frac{\partial(\Delta T)}{\partial(T_{cp})}$ 
    - 回転、逆行列など
- 最終的には、チェインルールにより、**所望のヤコビ行列を計算**

```
17  ✓ gtsam::Vector CustomBetweenFactor::evaluateError(  
18      const gtsam::Pose3 &pp, const gtsam::Pose3 &cp,  
19      boost::optional<gtsam::Matrix&> H1, boost::optional<gtsam::Matrix&> H2)  
20  
21      // d(T_pp_cp) / d(previous_pose)  
22      gtsam::Matrix66 H_dp_pp;  
23      // d(T_pp_cp) / d(current_pose)  
24      gtsam::Matrix66 H_dp_cp;  
25  
26      // delta pose between previous pose and current pose  $\Delta T = T_{pp}^{-1}T_{cp}$   
27      gtsam::Pose3 dp = pp.between(cp, H_dp_pp, H_dp_cp);  
35  
36      // d(log_Error) / d(Error)  
37      gtsam::Matrix6 H_logE_E;  
38  
39      // Error with logmap  
40      gtsam::Vector6 log_Error = gtsam::Pose3::Logmap(Error, H_logE_E);
```

誤差を計算する過程で、ほとんどのヤコビ行列はGTSAMが計算してくれる

$\Delta T = T_{pp}^{-1}T_{cp}$ に加えて、 $\frac{\partial(\Delta T)}{\partial(T_{pp})}, \frac{\partial(\Delta T)}{\partial(T_{cp})}$ も同時に計算される





## 5. 実装技術: オリジナルな拘束の実装

- チェインルールにより、所望のヤコビ行列の計算
  - 所望のヤコビ行列: 誤差ベクトルの最適化変数(今回はポーズ)に関するヤコビ行列
  - チェインルールが有効
    - 合成関数の微分に関するテクニック
      - 複雑な関数のヤコビ行列の計算に便利
- ヤコビ行列の計算結果
  - $\frac{\partial(\log E)}{\partial(T_{pp})} = \frac{\partial(\log E)}{\partial(E)} \frac{\partial(E)}{\partial(\Delta T)} \frac{\partial(\Delta T)}{\partial(T_{pp})}$
  - $\frac{\partial(\log E)}{\partial(T_{cp})} = \frac{\partial(\log E)}{\partial(E)} \frac{\partial(E)}{\partial(\Delta T)} \frac{\partial(\Delta T)}{\partial(T_{cp})}$
- evaluateError()の返回值
  - 最後に、ベクトルの誤差をreturn
- 実装終了

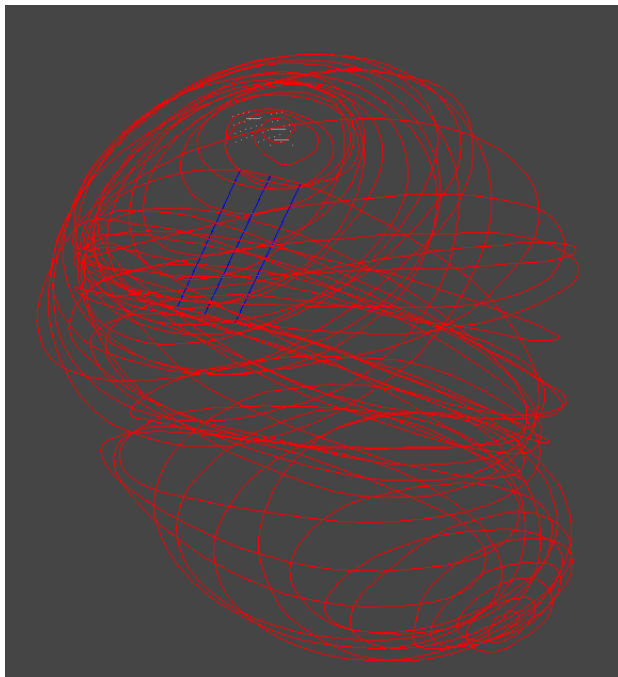
### 重要な箇所@ [customfactor.cpp](#)

```
42         if(H1) {  
43             // d(log Error) / d(previous pose)  
44             *H1 = H_logE_E * H_E_dp * H_dp_pp;  
45         }  
  
47         if(H2) {  
48             // d(log Error) / d(current pose)  
49             *H2 = H_logE_E * H_E_dp * H_dp_cp;  
50         }  
  
51  
52         return log_Error;  
53     }
```

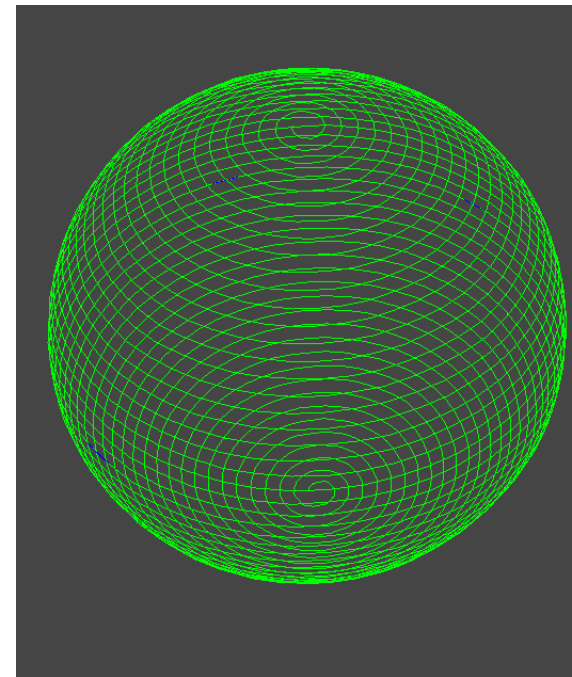


## 5. 実装技術: オリジナルな拘束の実装

最適化前



最適化後



**BetweenFactor<Pose3>と同じ結果が得られた**



## 5. 実装技術: GTSAMを用いて新しいSLAMを実装するには

### • オリジナルの拘束を実装 手順まとめ

- NoiseModelFactorを継承し、拘束の内容を実装
- → 具体的な作業内容は、evaluateError()の中身を実装し、下記2つを計算
  - 誤差ベクトル
  - 誤差ベクトルの最適化変数に関するヤコビ行列

### • 補足: ヤコビ行列とは

- 定義式は右の図
- イメージ

- $x_1$ を微小変化させたとき...

- $f_1$ はどう変化する?
- $f_2$ はどう変化する?
- $f_m$ はどう変化する?

これらの変化の割合を  
全ての組み合わせ分まとめたもの

$$\frac{\partial f}{\partial x} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix} \quad \left( f = \begin{bmatrix} f_1(x_1, \dots, x_n) \\ \vdots \\ f_m(x_1, \dots, x_n) \end{bmatrix} \right)$$

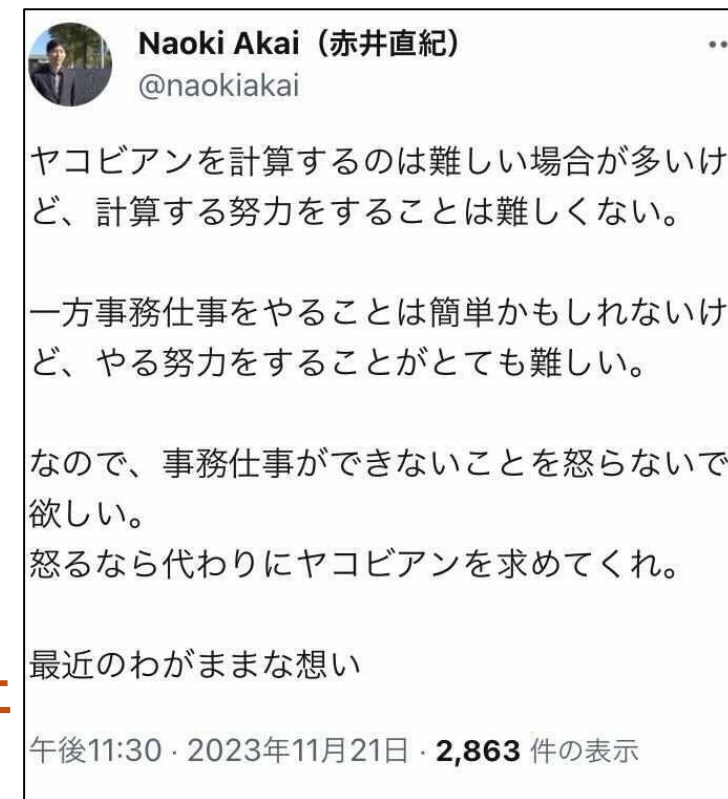
$f$ は $x$ の関数

- ガウスニュートン法などの最適化計算の必要不可欠な材料



## 5. 実装技術: GTSAMを用いて新しいSLAMを実装するには

- 新しいSLAMの実装で手間がかかるもの
  1. そもそも、新しい理論を作る
  2. 新しい理論に関するヤコビ行列を導出、実装する
- ヤコビ行列の計算が大変な場合
  - GTSAMで手軽に計算できない特殊な拘束
    - つまり、ヤコビ行列の一部を解析的に導出する必要がある
  - 例: 車輪の滑りの補正を考慮したオドメトリ拘束
    - 車輪の滑りが変化した時、オドメトリ拘束の誤差関数がどのように変化するかを記述するヤコビ行列の導出が必要になる
- 余談
  - 解析的に導出したヤコビ行列が本当に正しいかどうかを検証することもまた大変...



名古屋大学の赤井先生のXのポストより引用



## 5. 実装技術: 最適化計算・ロボット工学におけるヤコビ行列

- ヤコビ行列は、ロボット工学の分野で頻繁に出てくる
  - ロボットアームの逆運動学の数値計算、可操作性
  - ビジュアルサーボを用いたロボットの視覚フィードバック制御
  - 拡張カルマンフィルタ
  - 誤差伝搬
  - 機械学習の誤差逆伝播(学習＝最適化なので、学習の肝)
- なぜヤコビ行列はロボット工学で頻繁に出てくるか？
  - ロボット工学では、何かを**最適化**した上で推定、行動計画、制御を行う場合が多い
  - ヤコビ行列は最適化における重要な要素のため、頻繁に出てくる
  - → **ヤコビ行列の理解は、ロボット工学を使いこなすために非常に重要**



## 5. 実装技術: SLAMの高性能化に必要なこと

---

- SLAMの各要素の改善
  - SLAMは、様々な要素の組み合わせ
    - 画像処理    もまた、様々な要素の組み合わせ
      - ...
    - 点群処理    もまた、様々な要素の組み合わせ
      - ...
    - 最適化計算
      - ...
- SLAMの新たな拘束の追加（特に、センサ融合）
  1. SLAMの新たな拘束に関する理論を構築
  2. その理論を実現するための最適化計算を実装
    - GTSAMで複雑な拘束を使う場合、NoiseModelFactorを継承し、evaluateError()を実装するだけ
      - 具体的には、誤差・ヤコビ行列を導出・実装

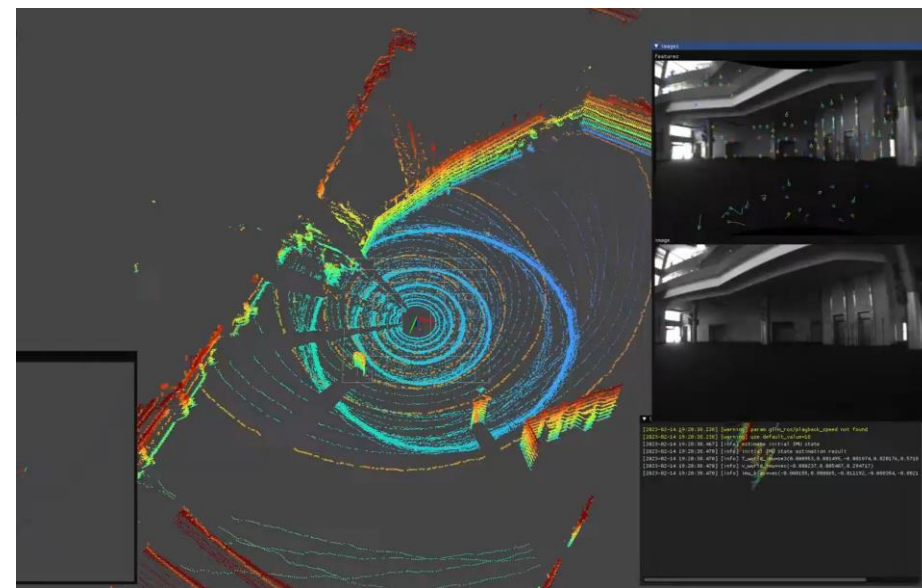




## 5. 実装技術: SLAMに新たな拘束を実装するには

- 新たな拘束を加えたSLAMを作る方法
  1. 新たな拘束を含め、全て1から作る
  2. 既存のオープンソースのSLAMに新たな拘束を追加
- オススメのオープンソースのSLAM: **GLIM** 🔥
  - 最新(2024年7月に公開)のLiDAR-IMU SLAM
  - 任意の距離センサ(LiDAR, デプスカメラ)で動作
  - 他のセンサを統合できるようなフレームワーク設計
    - カメラ、エンコーダによるオドメトリ、GPSなどを容易に統合
  - GTSAMでの実装
    - NoiseModelFactorの継承、evaluateError()の中身の実装
  - 開発者が日本の研究者
    - 企業との共同研究を多く行い、技術アドバイスの経験

<https://www.youtube.com/watch?v=9XG8pTI4Up0>



カメラ画像の拘束を統合したGLIM

Koide, Kenji, et al. "GLIM: 3D range-inertial localization and mapping with GPU-accelerated scan matching factors." Robotics and Autonomous Systems (2024): 104750.



## 5. 実装技術: GLIM + ホイールオドメトリ

機構パラメータ(車輪半径など)や環境項(滑りなど)をオンラインで補正する**ホイールオドメトリによる拘束をGLIMに統合**  
→ **オープンソース化**されています。 [https://github.com/TakuOkawara/full\\_linear\\_wheel\\_odometry\\_factor](https://github.com/TakuOkawara/full_linear_wheel_odometry_factor)

1. **Online calibration** of kinematic models for a skid steering robot



**Robustness to kinematic model errors**



**All wheel radii are increased by 25%.**

2. **Tightly coupled LiDAR-IMU-Wheel odometry**



**Robustness to point clouds degeneracy**



**Narrow FOV LiDAR points to the wall of corridors**  
to imitate severe point cloud degeneracy (e.g., tunnels).



## 5章 まとめ

---

- GTSAMを用いたデモを通したポーズ調整の説明
  - BetweenFactorによる拘束を用いた場合
    - デモ #4 ([small\\_pose\\_adjustment\\_2d.cpp](#))
    - デモ #5 ([pose\\_adjustment\\_2d.cpp](#))
    - デモ #6 ([pose\\_adjustment\\_3d.cpp](#))
  - 自身で実装したオリジナルの拘束を用いた場合
    - デモ #7 ([customfactor.hpp](#)、[customfactor.cpp](#)、[pose\\_adjustment\\_3d\\_with\\_customfactor.cpp](#))
- ヤコビ行列の重要性
- SLAMの高性能化に関するまとめ



# 目次

---

- 1章: SLAMの概要
- 2章: SLAMの基礎理論
- 3章: オドメトリ推定手法
- 4章: センサ融合によるSLAMの高性能化
- 5章: SLAMの実装技術とそのポイント
- **6章: SLAMの実例紹介、最新研究動向**



## 6. 実例紹介: SLAMを活用した実例

---

- 自己位置や地図が分かると、どの分野で応用できるか？
  - 自律移動ロボット・自動運転
    - 「周囲に何があるか」、「ロボット自身が地図上のどこに位置するか」は自律移動の基盤
    - **自動運転を行うための地図作成**
  - 林業や農業、建築現場
    - 現場の自律移動に加え、畑や建物などの**3Dスキャンや測量**など
  - VR・AR
    - 「人間がどう動いているのか」、「カメラと現実世界の対応関係はどうなってるか」などを把握
- 実例の動向
  - 自律移動ロボットのための環境認識、環境上の自己位置の特定
    - 昔から変わらず重要
  - 高精度な地図のニーズの増加
    - 高精度(矛盾の無い)地図には、**局所的な整合性と大域的な整合性を保つSLAMの理論が必須**

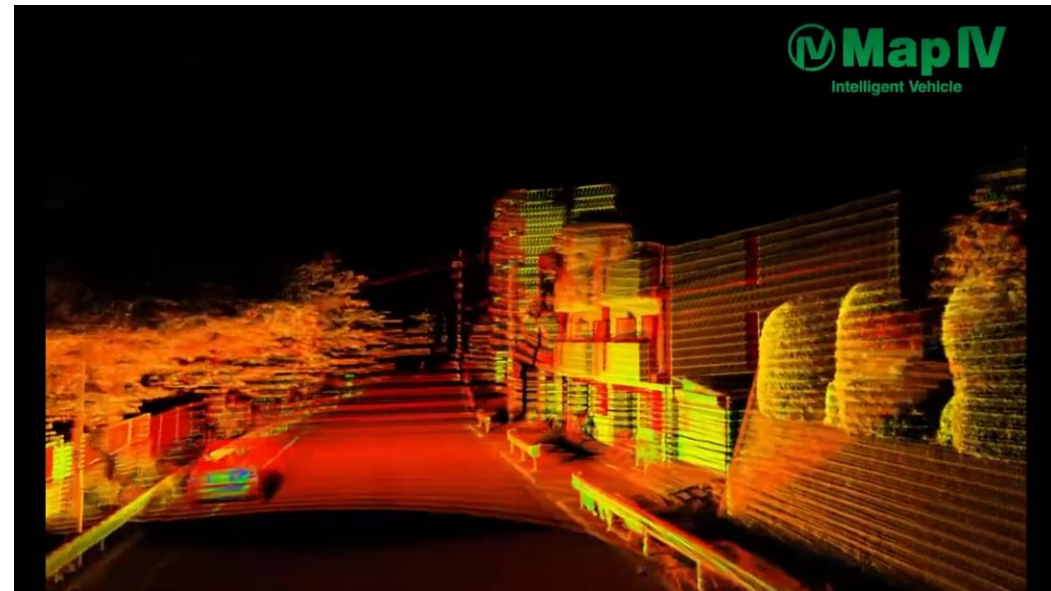




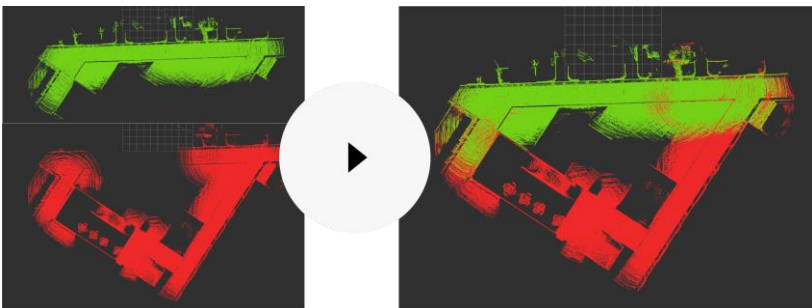
## 6. 実例紹介: MAP IV Engine

<https://www.youtube.com/watch?v=IONesfYdull>

- MAP IV Engine: 地図作成ソフトウェア
  - マップフォー独自のSLAM技術
    - 日本の企業です
  - IMUやGNSSとの統合可能
  - 専門知識ナシでも操作可能なUI
  - 下記で示す豊富な機能
    - → **自動運転、工場内の自律移動**などに使用



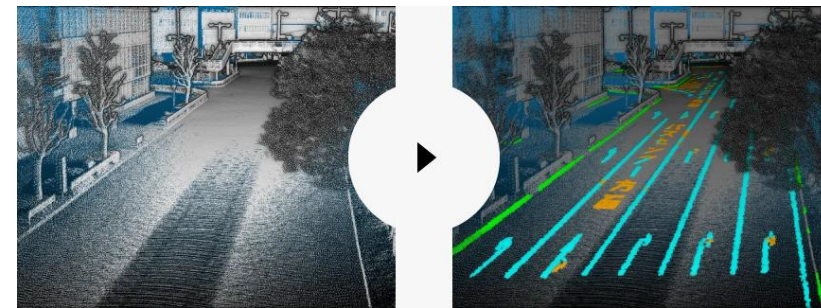
© MAP IV, INC.



地図の結合



点群の着色



路面マークなどの自動認識

<https://www.map4.jp/solutions/mapping-localization/map4-engine/>





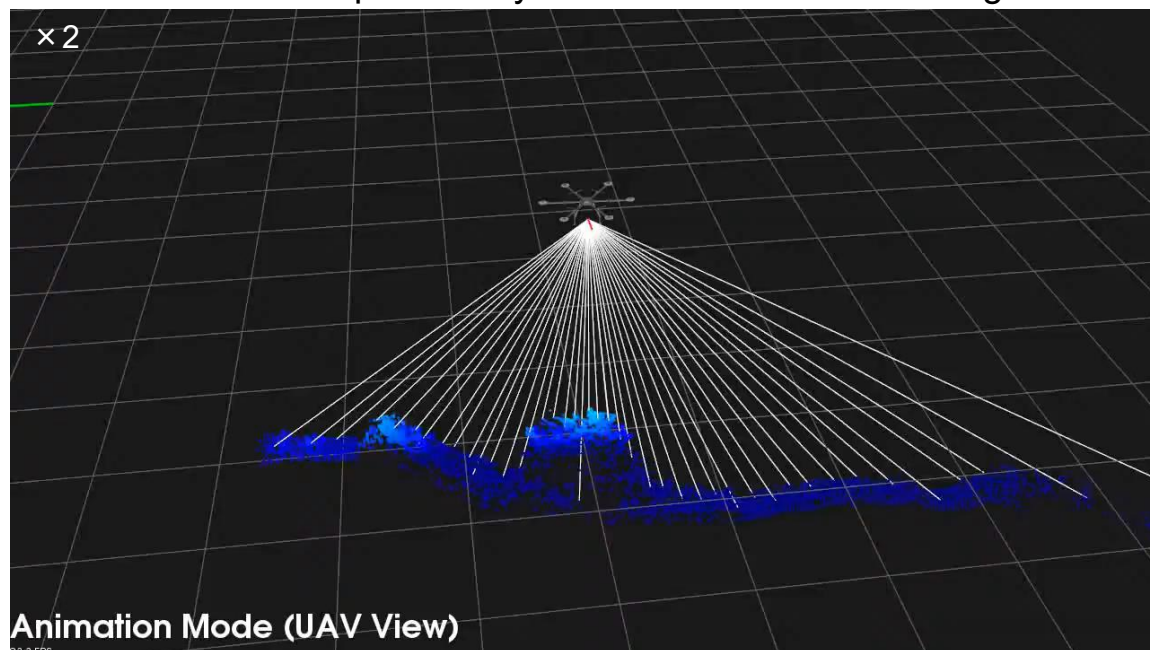
## 6. 実例紹介: ドローンを用いた地図構築

- ドローンを使用することのメリット
  - 人間が行きにくい箇所の地図構築が可能
  - 上空からの大規模な地図を容易に構築可能
    - 農業や災害現場、インフラ点検などで応用可能



© JEPICO

<https://www.youtube.com/watch?v=cbczfgH1x0s>



[https://www.youtube.com/watch?v=M\\_LIFtDuZVg](https://www.youtube.com/watch?v=M_LIFtDuZVg)





## 6. 実例紹介: 林業の自動化

© 労働新聞社

### • 林業は過酷

- 労働力不足
  - 従事者の高齢化
  - → 管理・整備が行き届いていない「未管理森林」が増大
- 労働災害
  - チェーンソーを用いた伐倒の際に死亡事故
- → 自律化の価値が高い



© 株式会社川上林業

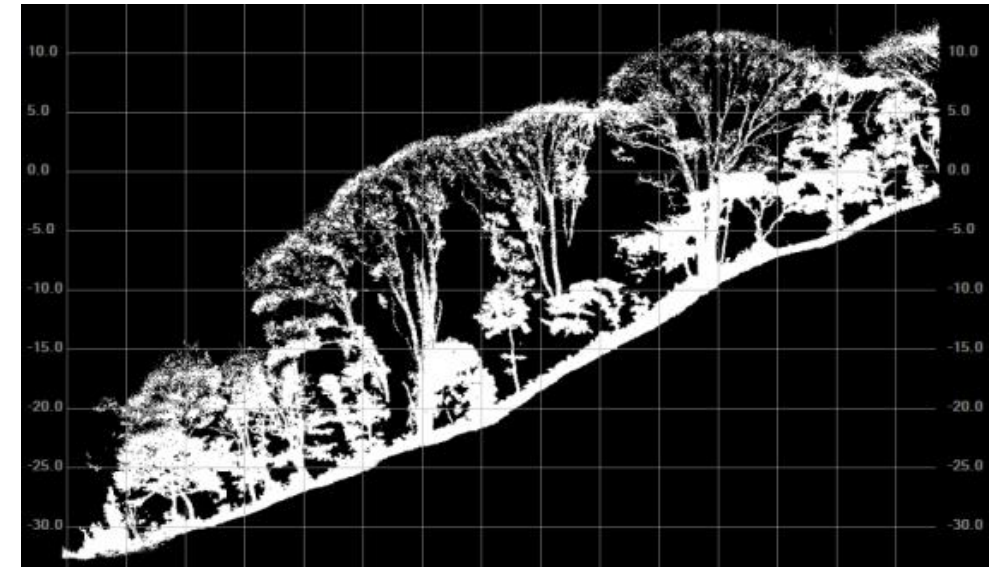
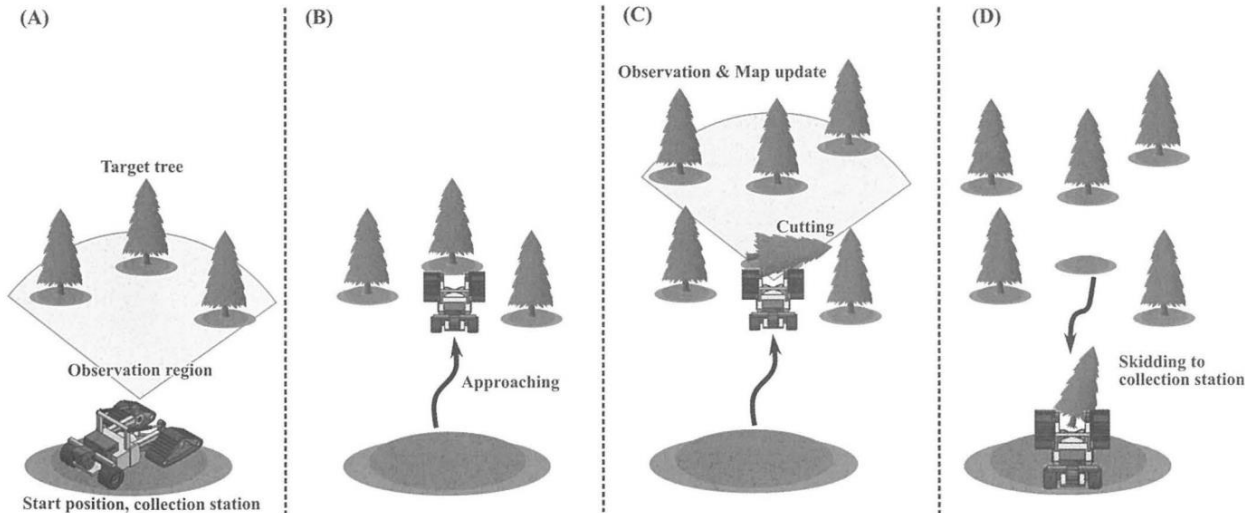
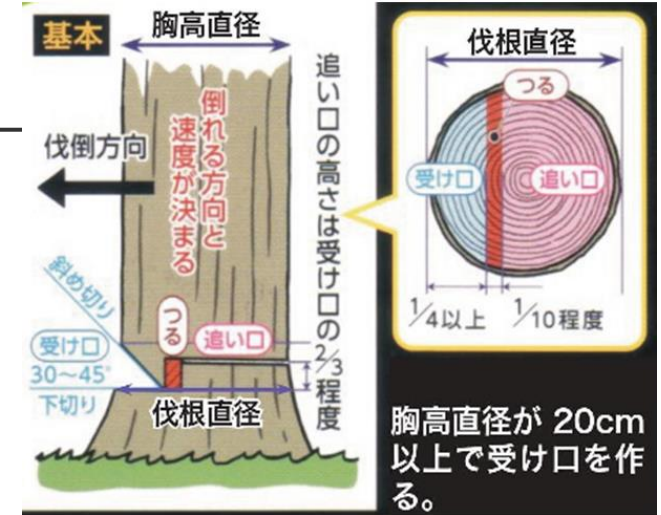






## 6. 実例紹介: 林業の自動化

- SLAMは林業の自律化にどう関わる？
  - 移動ロボットの自律移動・伐倒
  - 森林の地図構築(データベース化)
    - 木の位置
    - 木の胸高直径など
- → 伐倒の際の重要なパラメータ(人間が計測してる)

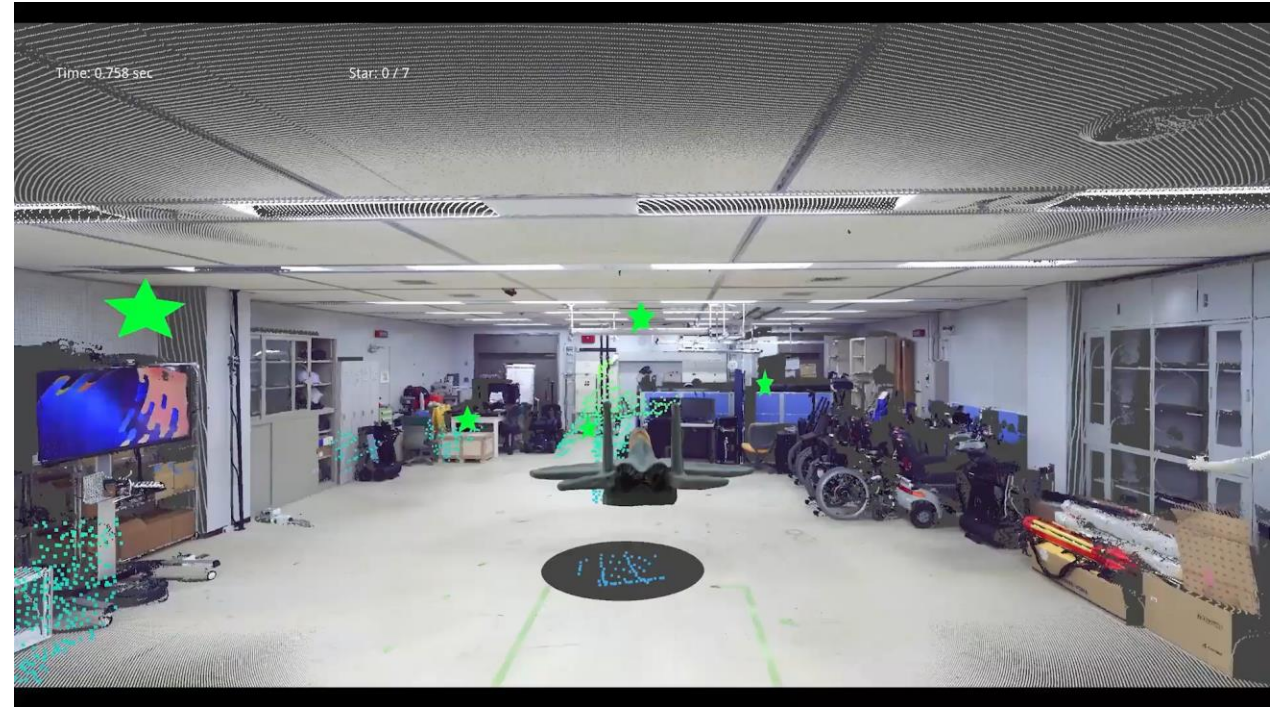


富永歩,ほか. "伐倒作業を行う林業機械の自律化に向けた自律移動システムの開発." 森林利用学会誌 36.2 (2021): 36-111.



## 6. 実例紹介：デジタルツイン

- <https://www.youtube.com/watch?v=EDoCQNJJGo>







## 6. センサ融合を用いたSLAMの最新動向



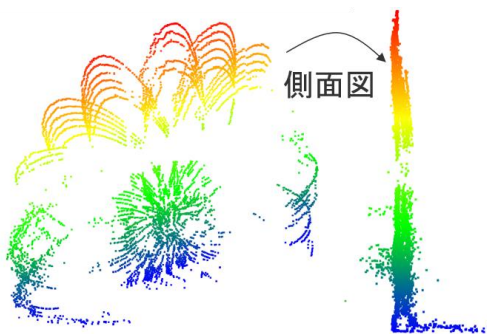
LiDAR SLAM, Visual SLAMが破綻する特徴が欠如した環境の例

- 紹介内容(大河原自身の**最新の研究内容**)
  - LiDAR-IMU-モデルベースオドメトリによるSLAM [[IEEE Access, 2024](#)]
  - LiDAR-IMU-機械学習ベースオドメトリによるSLAM
    - 車輪型ロボット [[Robotics and Autonomous Systems, 2025](#)]
    - 脚型ロボット [[Robotics and Automation Letters, 2025](#)]
  - ロボット身体への汎用性を有する機械学習ベースオドメトリ [[IROS, 2026](#)]



## 6. LiDAR-IMU-モデルベースオドメトリによるSLAM

走行環境



取得される点群

FOVの狭いLiDAR  
を壁面に向ける

全ての車輪半径  
を25%増加  
(機械的誤差)



地図構築と自己位置推定の結果

- 特徴が十分
- 特徴が欠如
- 点群が取得不可



× 50

ホイールオドメトリをそのまま使えない





## 6. LiDAR-IMU-モデルベースオドメトリによるSLAM

LiDAR-IMUによるSLAM: 特徴が豊富であれば非常に高精度



**SLAM研究の動向: 特徴の欠如に頑強なSLAM**

W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "Fast-lid2: Fast direct lidar inertial odometry," IEEE Transactions on Robotics, vol. 38, no. 4, pp. 2053–2073, 2022



## [再掲] 4. センサ融合: SLAMで使われるセンサ

- **センサ融合**: 複数のセンサを組み合わせ、各センサの長所を活かし、短所を補間

- 外界センサ (SLAMのメイン)

- LiDAR

- **長所**: 高精度な3次元位置を直接取得可能、360度視野
- **短所**: 色情報なし、点群の間隔がスパース、**幾何特徴が不十分だと破綻**



- 内界センサ (メインの補助)

- IMU

- **長所**: 高速な動作にも対応可能、**環境の特徴に依存せずに移動量推定が可能**
- **短所**: 並進方向の移動量には加速度の二階積分が必要、計測値のバイアスの考慮

- エンコーダ (ホイールオドメトリ)

- **長所**: IMUより安定した並進移動量の推定が可能、**環境の特徴に依存せずに移動量推定が可能**
- **短所**: 運動学モデルの誤差の影響



## 6. LiDAR-IMU-モデルベースオドメトリによるSLAM

### LiDAR-IMUによるオドメトリ推定の頑強性の課題

- 特徴が少ない環境下での点群の縮退

点群から移動量を一意に決定できない問題  
(長い廊下やトンネルなどで発生)

- IMUはどうか？
  - 長期間の点群の縮退には対応不可能
- ホイールオドメトリは？
  - 長期間の点群の縮退に対応可能
  - 車輪径や滑り(運動モデル)が正確であることが前提



~~× 事前に1度だけ~~

オンラインでのキャリブレーションが重要

- 経年劣化で車輪が摩耗
- 滑りは環境によって変化



点群の縮退が起きる環境の例



## 6. LiDAR-IMU-モデルベースオドメトリによるSLAM

LiDAR-IMU SLAM

高精度だが、幾何特徴の欠如に弱い



相補の関係



ホイールオドメトリ

パラメータ誤差に弱い、幾何特徴の欠如に強い

- LiDAR-IMU SLAMの高精度な推定結果によって、ホイールオドメトリのパラメータをオンラインキャリブレーション
- 幾何特徴の欠如がある環境では、ホイールオドメトリがLiDAR-IMU SLAMの推定を助ける

方針

自己位置に加えて、ホイールオドメトリのパラメータを求める = ポーズグラフではなく、ファクタグラフを使用  
高性能な融合を行いたい = タイトカップリングを使用 (LiDAR-SLAMとオンラインキャリブレーションの同時実行)

どんな運動モデルをオンラインキャリブレーションするのか？



## 6. LiDAR-IMU-モデルベースオドメトリによるSLAM

### • スキッドステアリングロボットの運動モデル

※ 車輪運動( $v = r\omega$ )を一般化した形

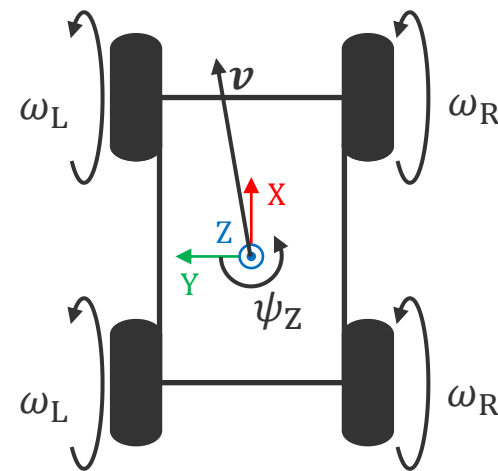
→ ステア機構を持たない二輪、四輪、六輪ロボット、クローラに適用可能

$$\begin{bmatrix} v_x \\ v_y \\ \psi_z \end{bmatrix} = J \begin{bmatrix} \omega_L \\ \omega_R \end{bmatrix}$$

ロボット速度      車輪速

$J$ :  $3 \times 2$ の行列

- ロボットの内部パラメータ(車輪径など)
- 環境に依存するパラメータ(滑りなど)



### • $J$ の表現方法: フルリニアモデルを使用

$$J = \begin{bmatrix} J_{11} & J_{12} \\ J_{21} & J_{22} \\ J_{31} & J_{32} \end{bmatrix}$$

$J$ の全ての要素を最適化変数として定義

$$K = [J_{11} \ J_{12} \ J_{21} \ J_{22} \ J_{31} \ J_{32}]^T$$

高精度な  $K$  が求まると。。。  
→ 高精度な自己位置を推定可能

モデル化が難しい環境依存のパラメータ・ロボットのモデル誤差を暗黙的に考慮可能



## 6. LiDAR-IMU-モデルベースオドメトリによるSLAM

### ・問題設定

LiDAR-IMU-wheel  
SLAM

### 提案手法

ホイールオドメトリのオンラインキャリブレーション  
＝高精度な  $K$  ( $= J$  行列) を求める

LiDAR-IMU-Wheel SLAMとオンラインキャリブレーションを同時に解く手法

### ・どうやって解くのか？

→ ファクタグラフ最適化に基づくタイトカップリング

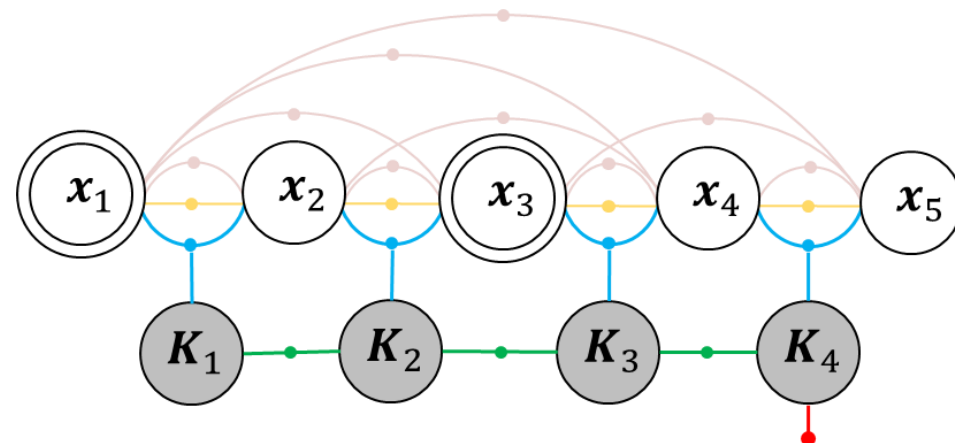
実装は複雑になるが  
高精度なセンサ融合を可能

最適化変数  $X_t = [x_t \quad K_t]$

自己位置など

ホイールオドメトリのパラメータ

同時に推定



自己位置とパラメータ  $K$  を同時推定するファクタグラフ

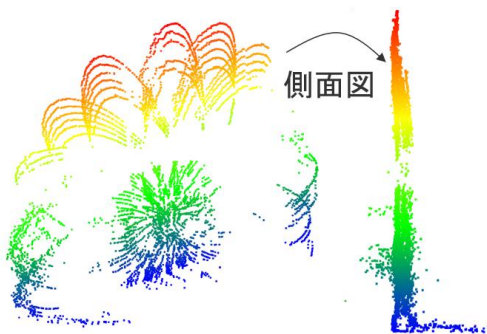
※ 各拘束の説明は論文を参照ください





# 6. LiDAR-IMU-モデルベースオドメトリによるSLAM

走行環境



取得される点群

FOVの狭いLiDAR  
を壁面に向ける

全ての車輪半径  
を25%増加  
(機械的誤差)



地図構築と自己位置推定の結果

- 特徴が十分
- 特徴が欠如
- 点群が取得不可



× 50

オンラインキャリブレーションで補正



## 6. LiDAR-IMU-モデルベースオドメトリによるSLAM

ホイールオドメトリのパラメータをオンラインで(その場で)キャリブレーション → 路面環境が変化しても適応可能



Taku Okawara\*, Kenji Koide\*\*, Shuji Oishi\*\*, Masashi Yokozuka\*\*,  
Atsuhiko Banno\*\*, Kentaro Uno\*, and Kazuya Yoshida\*

\* Tohoku University

\*\* National Institute of Advanced Industrial Science and Technology (AIST)

動画URL

<https://youtu.be/Vss86xUhU80?t=115>



## 6. SLAMの最新動向

### • 機械学習とSLAMの融合

- 巨大なネットワーク(基盤モデル)を活用

😊 多様なタスクをこなせる

😞 計算コストが高い

- 小さなネットワークを活用

- あるタスクに特化したネットワーク

- タスクごとにネットワークを作って組み合わせれば、多様なタスクもこなせる

- 特徴点マッチング

- ループ検出

- 移動量の推定

- etc.

😊 計算コストが小さい

→ 移動量推定のための小型のネットワークについての説明に焦点

ロボット運動の線形モデル(フルリニアモデル)  
を機械学習で表現する研究  
=先ほど紹介した研究の発展版





## 6. LiDAR-IMU-学習ベースオドメトリによるSLAM

### • 線形モデルの限界

- × 滑りを表現できるが、**滑りの挙動は基本的には非線形**
  - 低速では問題が薄い、高速走行では問題が顕著
- × 地面の変形などは、そもそも**定式化が困難**
- × 脚型ロボットのオドメトリは**非線形**(大量の三角関数)



- 線形モデルでは、**複雑な非線形項**を表現できない
- その非線形項は、挙動が複雑であるため、**明示的に定式化困難**



ホイールオドメトリをニューラルネットワークで表現し、機械学習で獲得する

$$v = \text{NN}(\text{センサデータ})$$

データから、複雑な項を高精度に表現可能



**ニューラル運動モデル**  
と呼ぶ

線形モデル(フルリニアモデル)

$$\begin{bmatrix} v_x \\ v_y \\ \psi_z \end{bmatrix} = J \begin{bmatrix} \omega_L \\ \omega_R \end{bmatrix}$$

ロボット速度      車輪速



## 6. LiDAR-IMU-学習ベースオドメトリによるSLAM

### ・従来のニューラル運動モデルの欠点: 路面環境への汎用性

#### 機械学習の課題

- ・ 事前データには含まれない状況での予測
  - ・ 路面環境への汎用性を獲得するには、無限種類の環境での事前データが必要
- ・ 汎用性と精度のトレードオフ
  - ・ 無限種類の環境の事前データがあったとしても、妥当なモデルを学習できない



1. 一つの条件にオーバーフィット



2. 平均的(どっちつかず)な性能に収束

従来研究では、上記の課題に対処した手法はほとんどなかった。

→ 事前学習で使った特定環境下でのみ、かなり高精度な推定が可能なニューラル運動モデルの研究がほとんど

事前学習時から環境が変化(例: 雨で路面が濡れる)、路面環境の遷移(例: 屋内→屋外)などの条件に対応困難





## 6. LiDAR-IMU-学習ベースオドメトリによるSLAM

提案手法が有効に働く状況・環境

### 車輪が滑る動作



極度の非線形車体挙動

### 路面環境の変化



環境反力 & 滑りの変化

### LiDARデータの欠損



幾何情報の欠如

従来手法では走破困難な状況・環境にも頑健に対処可能





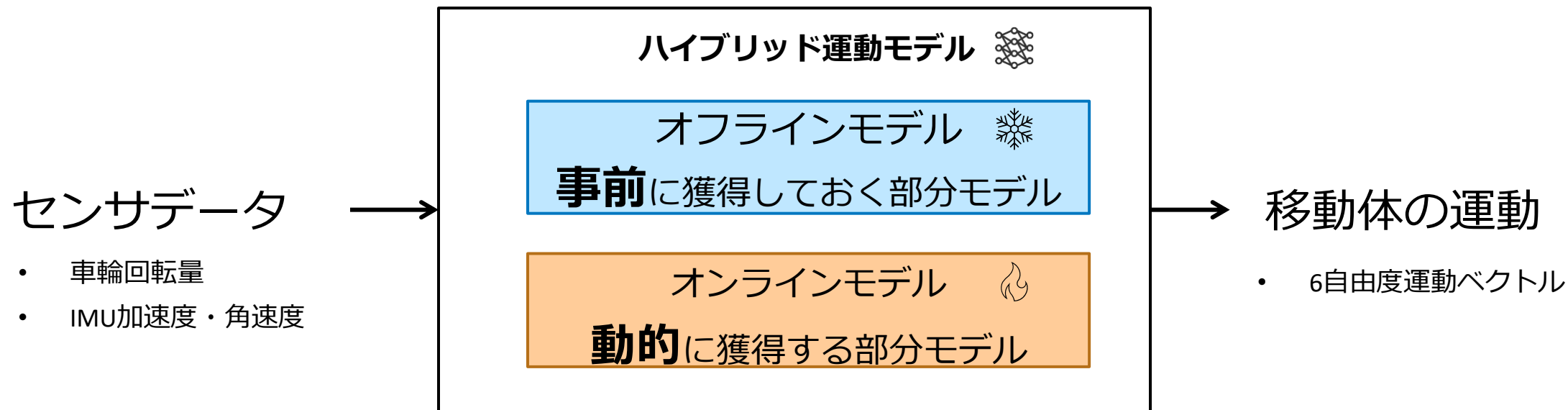
## 6. LiDAR-IMU-学習ベースオドメトリによるSLAM

- 提案手法: ニューラル運動モデルのオンライン学習
  - 路面に適応させるように、**ニューラル運動モデル**をリアルタイムで動的に学習する
- 問題設定: 実は、線形モデルのオンラインキャリブレーションの *拡張版*
  - ホイールオドメトリの線形モデル: **6次元**のパラメータのオンラインキャリブレーション
  - 今回: **数万次元**のパラメータのオンラインキャリブレーション
  - SLAMのシステムとしては、さきほどの研究例と全く同じ → SLAMと機械学習を同時に解く  
異なるのは、オンラインで学習する**パラメータの次元の数が膨大という点**のみ
- 提案手法の課題: **大規模なパラメータのキャリブレーションの高速計算**
  - 一般的な機械学習では、大量に時間をかけ、大量なデータで学習をする
  - 工夫をして、SLAMと同じ周期(≒ 0.1秒)で、**高速に学習をする必要がある**



## 6. LiDAR-IMU-学習ベースオドメトリによるSLAM

ニューラル運動モデルのパラメータを**オフライン学習**/**オンライン学習**併せたハイブリッド学習で獲得



- **オフラインモデル** (40000個のパラメータ) : 環境に依らない普遍的なパラメータ (ロボットパラメータ)
- **オンラインモデル** (100個のパラメータ) : 時々刻々と変化する動的なパラメータ (環境パラメータ)

の獲得に注力させることで, **学習の安定化・効率化**を図る

→ オンラインで学習するパラメータを100次元まで減らせ、**リアルタイムでの学習が可能**



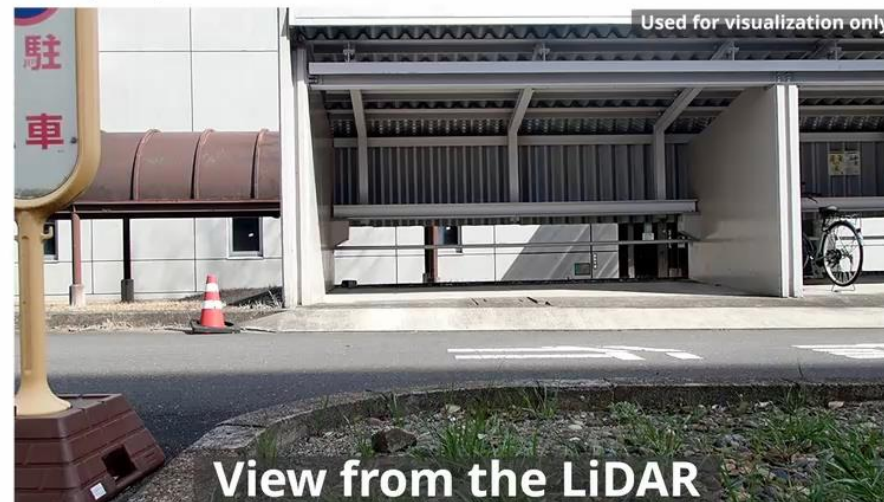
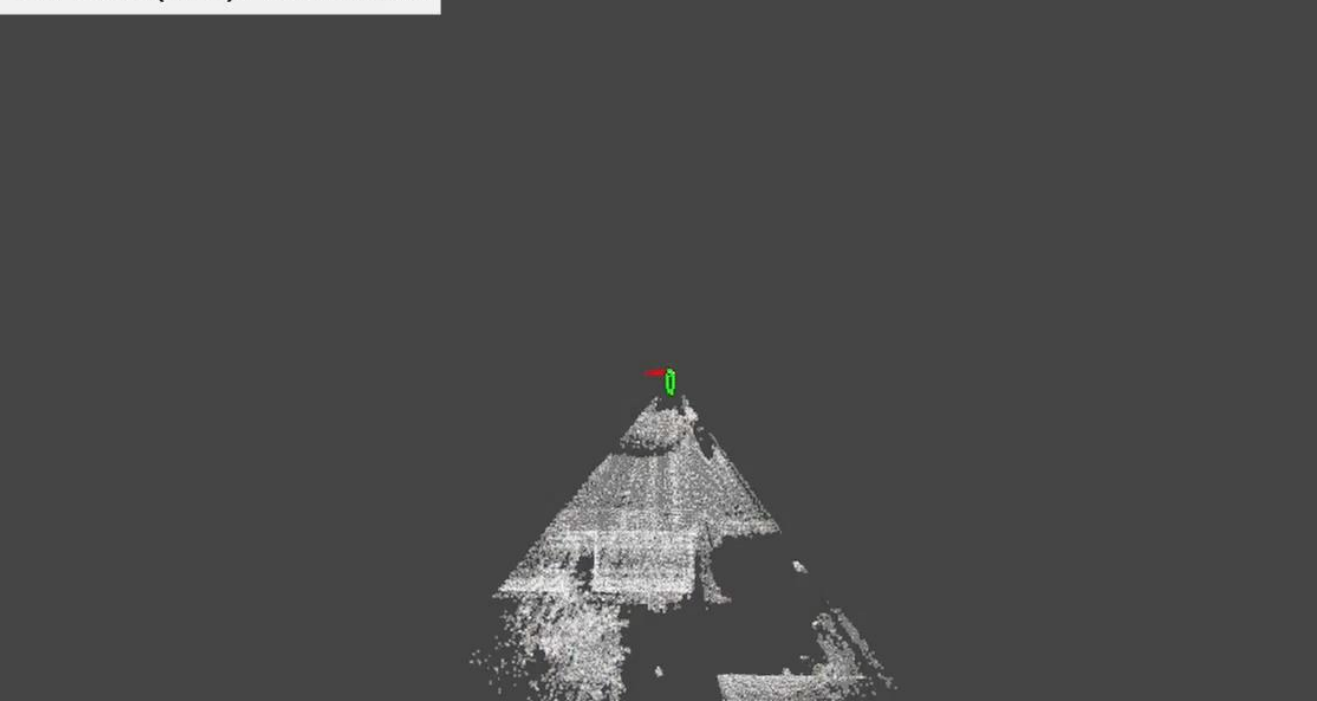
# 6. LiDAR-IMU-学習ベースオドメトリによるSLAM

Odometry estimation  
Sequence #3

— Ours — Ours w/o online learning — LIWO w/ linear model online calibration  
— FAST-LIO-Multi-Sensor-Fusion — FAST-LIO2

× 1 ▶

Terrain (1/2): Concrete



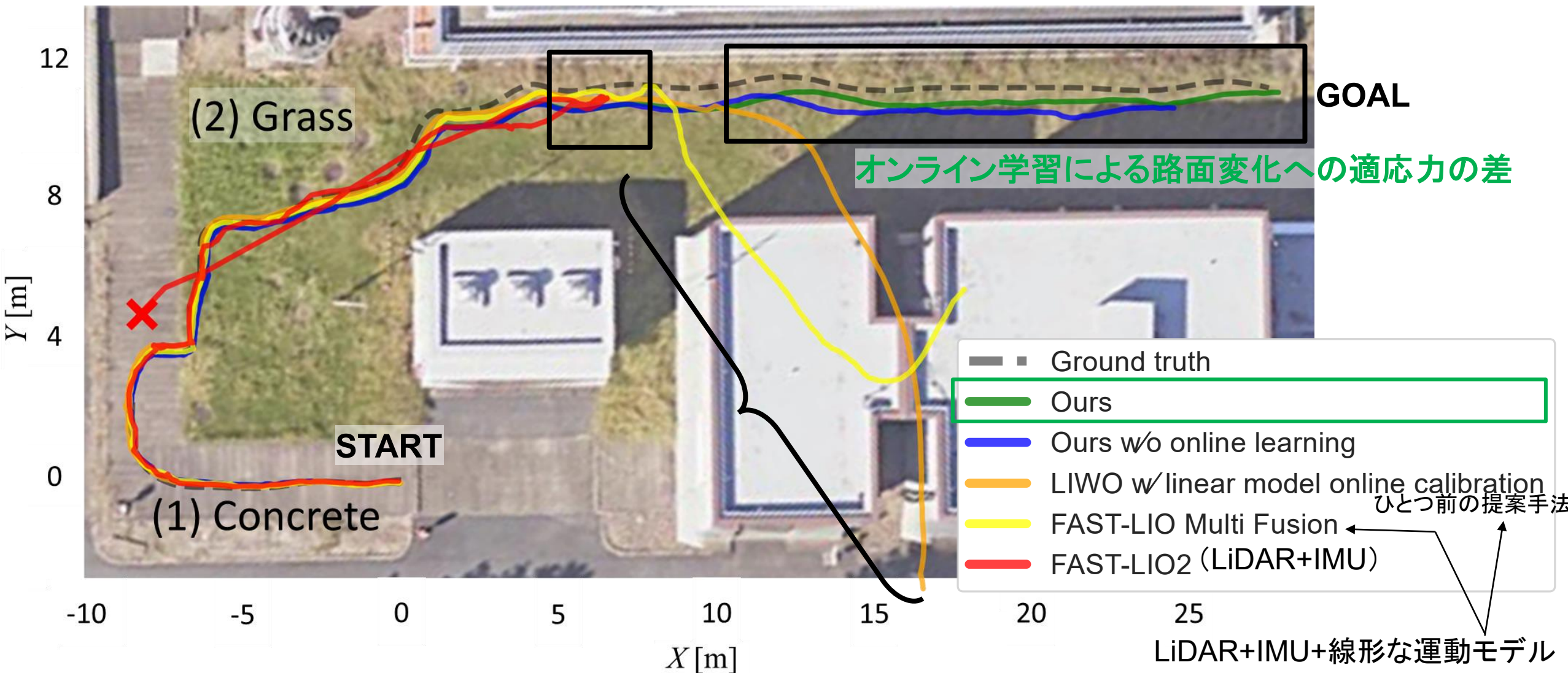
動画URL

<https://www.youtube.com/watch?v=CvRVhdda7Cw>





## 6. LiDAR-IMU-学習ベースオドメトリによるSLAM





## 6. LiDAR-IMU-学習ベース**レッグ**オドメトリによるSLAM



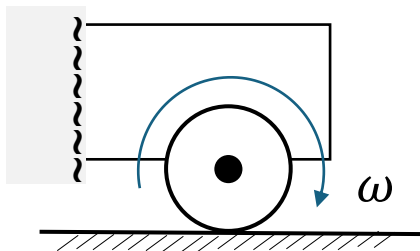
走破性・機動力の高さから脚ロボットへの注目が急速に高まっている → **脚型ロボット用のSLAMの需要**





## 6. LiDAR-IMU-学習ベース**レッグ**オドメトリによるSLAM

### 車輪の運動モデル



#### 入力データ

- 車輪角速度  $\omega$

#### 運動モデル

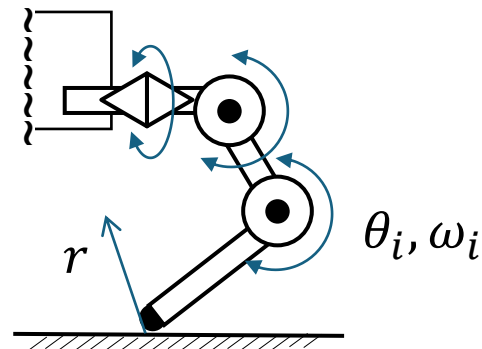
- 車輪回転

### 脚の運動モデル

- 車輪よりも複雑
- モデル化が大変



学習であれば、  
容易に獲得可能



#### 入力データ

- 関節角  $\theta_i, \omega_i$
- 脚先反力  $r$

#### 運動モデル

- 関節回転
- 脚の接地

- 車輪と脚では**運動モデルが異なる**が、提案手法は**脚型ロボットにも拡張可能**
  - 機械学習は、モデルフリーとも呼ばれる。脚型ロボット用に入出力を修正し、再学習するだけでモデル獲得
- **新たな入力モダリティ(脚先の触覚情報)の拡張**も容易



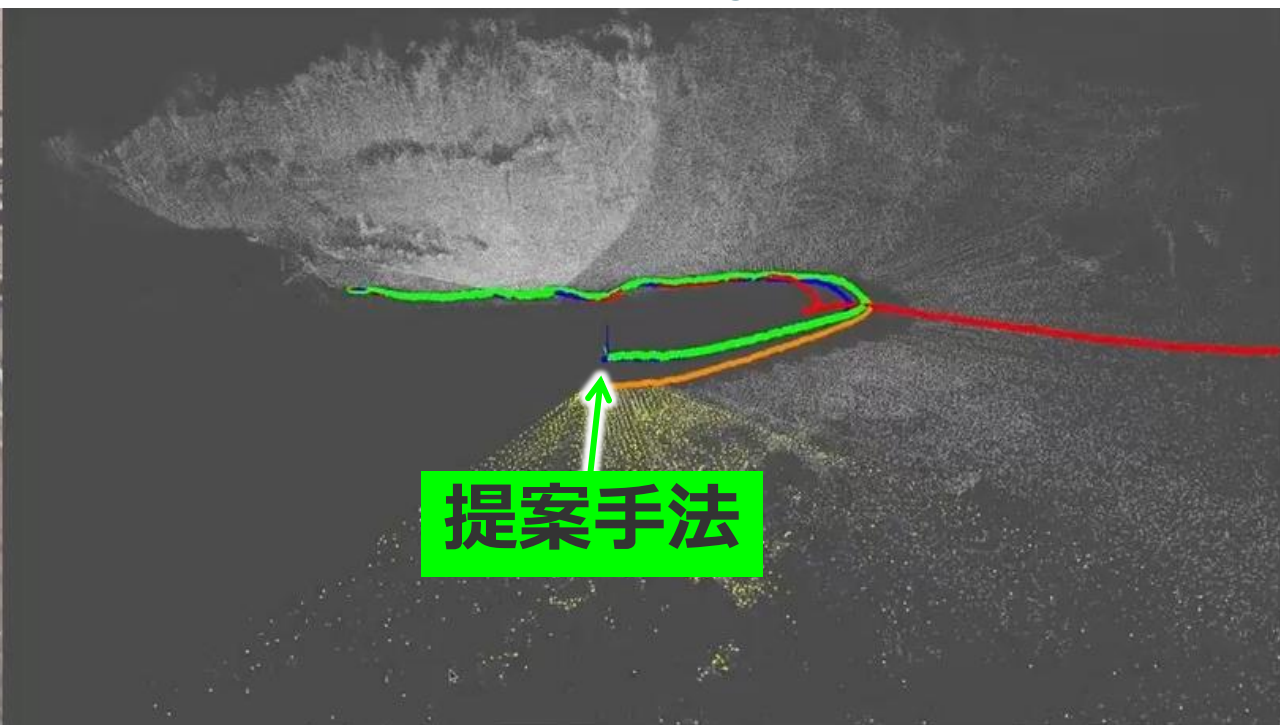


## 6. LiDAR-IMU-学習ベース**レグ**オドメトリによるSLAM

幾何特徴が極めて少ない環境(X、Y、Yawの三成分での位置合わせが不可能)での実験

動画URL

<https://youtu.be/ZD5cR1u8Nc0>



砂浜（**特徴の欠如 + 軟弱地盤**）でも**高精度なSLAM**



## 6. 身体への汎用性を有する機械学習ベースオドメトリ

- これまで提案したニューラル運動モデルの課題: **身体への汎用性**
  - 車輪型の研究成果: ~~四輪ロボット用モデル~~ rover robotics rover miniの専用モデル
  - 脚型の研究成果: ~~四脚ロボット用モデル~~ Unitree Go2の専用モデル
    - 既存のニューラル運動モデルは、**リンク長や車輪径が少しでも異なると、再学習が必要**
    - なぜか？ → 身体を考慮した学習をしていないから。
      - 余談: 「身体を考慮した学習」というのは、現在のPhysical AI/ロボット基盤モデルのキーワード
  - あるロボット専用のニューラル運動モデルを作られるケースがほとんど
    - これまでの自身の研究で、多様な環境用のニューラル運動モデルを単一モデルで表現できた
    - 身体に関しても汎用化できないか？ 脚と車輪のニューラル運動モデルを単一モデルで表現したい  
→ **ユーザーの手間(データの再収集、モデルの再学習)を可能な限り減らしたい**
- 機械学習で、多様な身体を考慮することの難しさ
  - 機械学習では、**入力のサイズが事前に定義されている**
    - **多様な車輪数・関節数**に対応するのは困難
    - **異なる形態である脚型と車輪型**を単一のニューラル運動モデルで表現するのはもっと困難

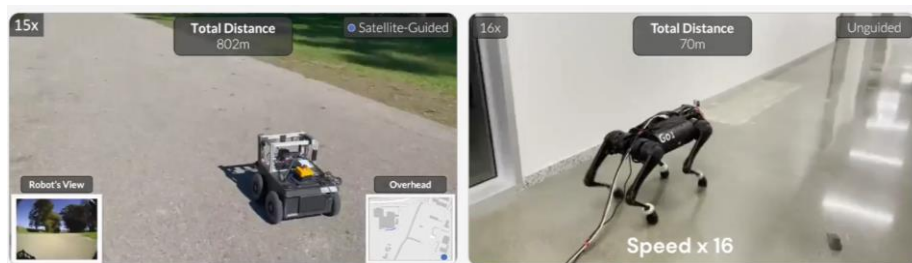


## 6. 身体への汎用性を有する機械学習ベースオドメトリ

### 学習データの収集 / 学習モデルの設計

近年のロボット基盤モデルと同様のアプローチ

- 大量のロボット身体へのデータ
- 可変長入力に対応可能なアーキテクチャ



General Navigation Model [Shah+, 2023]



RT-X [O'Neill+, 2024]

ロボット身体を考慮していないケースがほとんど

+ 身体を考慮したロボットの運動表現を統一するための工夫 (省略)

SLAMにあまり関係ないので。。

興味あれば、IROS2026で発表される  
KINGという論文を参照ください

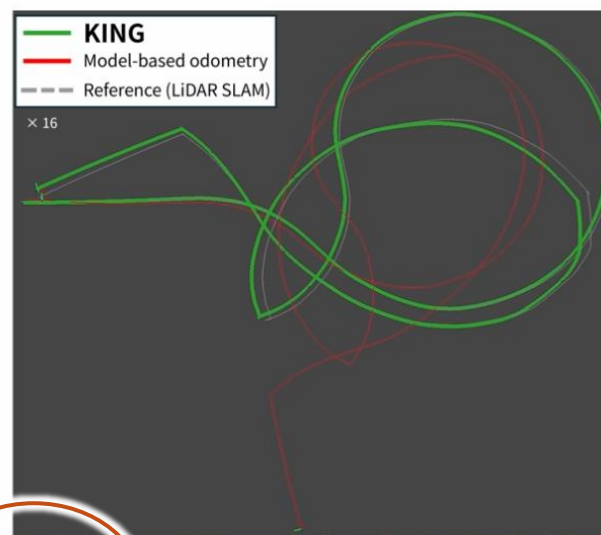




## 6. 身体への汎用性を有する機械学習ベースオドメトリ

異なる形態である四脚ロボットと四輪ロボットにおいて、単一モデルで高精度な自己位置推定が可能

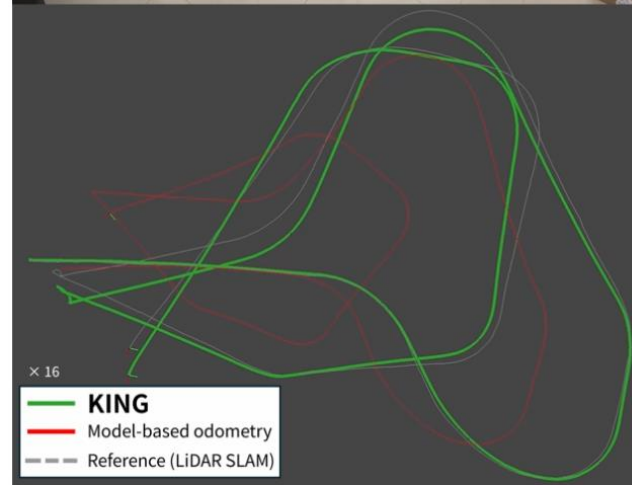
### Sim-to-Real: Odometry Experiments for *Legged* and *Wheeled* Robots



大規模な事前学習では、  
該当のロボットのデータは  
不使用



四脚ロボットの推定の際に、  
四脚ロボットのデータでの  
学習をほぼしていない  
→ 提案した学習モデルは、  
汎用的な身体運動表現を  
獲得している



even though this morphology data was not included in pretraining.



# 本セミナーのまとめ

---

- SLAMの基礎的な理論
  - グラフベースSLAM (グラフ理論のノードとエッジによってモデル化されたSLAM)
  - ファクタグラフ (グラフベースSLAMで使われる一般的なグラフ表現)
  - ポーズ調整 (拘束/最適化変数が全てポーズの場合となる、ファクタグラフにおける特殊形)
- SLAMの拘束を取得する様々な手法
  - ホイールオドメトリ/レッグオドメトリ (ロボット運動を用いた移動量の計算方法)
  - スキャンマッチング (LiDARを用いた移動量の計算方法)
  - バンドル調整 (カメラ画像を用いた移動量の計算方法)
- センサ融合を用いたSLAM
  - ルーズカップリング (実装は簡単だが、タイトカップリングほど性能は出ない)
  - タイトカップリング (実装は複雑だが、高精度かつ頑強)
- GTSAMを用いたSLAMの実装
  - BetweenFactor (ポーズグラフ・ポーズグラフ最適化を簡単に実装可能)
  - NoiseModelFactor (自身で新たな拘束(ファクタ)を実装する際に必要)





# デモのまとめ

---

デモ #1: オープンソースのICP [gtsam\\_points](#)

デモ #2: オープンソースのLiDAR SLAM [GLIM](#)

デモ #3: オープンソースのVisual SLAM [RTAB-Map](#)

デモ #4: シンプルなポーズ調整 [small\\_pose\\_adjustment\\_2d.cpp](#)

デモ #5: 実践的な2Dのポーズ調整 [pose\\_adjustment\\_2d.cpp](#)

デモ #6: 実践的な3Dのポーズ調整 [pose\\_adjustment\\_3d.cpp](#)

デモ #7: オリジナルの拘束の作成 [customfactor.hpp](#)、[customfactor.cpp](#)



# 参考になる資料

- 本セミナーでは、各要素の細かい理論や式展開は省略した
- 各理論の詳細を追う際に便利な資料を下記に示す
  - [slambook](#)
    - C++、コンピュータビジョン、最適化計算など**全てを網羅した無料の資料**
    - 上記の**全技術に関してのC++のサンプルコード付き**
      - 最終的には、Visual SLAMを1から実装する
    - 英語だが、日本人にとっては読みやすい記述
    - [書籍化](#)もされている
  - [SLAM handbook](#)
    - **機械学習とSLAMの融合**など、基礎から最新手法まで丁寧に解説
    - 英語で書かれているが、無料で入手できて、ボリュームもすごい
  - SLAM入門(著: 友納正裕)
    - LiDARを用いた基礎的なSLAMを**日本語**で解説
    - C++のサンプルコード付き
  - 詳細確率ロボティクス(著: 上田隆一)
    - 基礎的なSLAMに加えてフィルタリングについても**日本語**で解説
    - Pythonのサンプルコード付き

